

# Robust Distributed Arrays:

## Provably Secure Networking for Data Availability Sampling

Dankrad Feist<sup>1</sup>    Gottfried Herold<sup>1</sup>    Mark Simkin<sup>2</sup>    Benedikt Wagner<sup>1</sup>

April 24, 2025

<sup>1</sup> Ethereum Foundation

[{dankrad.feist,gottfried.herold,benedikt.wagner}@ethereum.org](mailto:{dankrad.feist,gottfried.herold,benedikt.wagner}@ethereum.org)

<sup>2</sup> Flashbots

[mark@univariate.org](mailto:mark@univariate.org)

### Abstract

Data Availability Sampling (DAS), a central component of Ethereum’s roadmap, enables clients to verify data availability without requiring any single client to download the entire dataset. DAS operates by having clients randomly retrieve individual symbols of erasure-encoded data from a peer-to-peer network. While the cryptographic and encoding aspects of DAS have recently undergone formal analysis, the peer-to-peer networking layer remains underexplored, with a lack of security definitions and efficient, provably secure constructions.

In this work, we address this gap by introducing a novel distributed data structure that can serve as the networking layer for DAS, which we call *robust distributed arrays*. That is, we rigorously define a robustness property of a distributed data structure in an open permissionless network, that mimics a collection of arrays.

Then, we give a simple and efficient construction and formally prove its robustness. Notably, every individual node is required to store only small portions of the data, and accessing array positions incurs minimal latency. The robustness of our construction relies solely on the presence of a minimal *absolute* number of honest nodes in the network. In particular, we avoid any honest majority assumption. Beyond DAS, we anticipate that robust distributed arrays can have wider applications in distributed systems.

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>3</b>  |
| 1.1      | Our Contribution . . . . .                               | 3         |
| 1.2      | Related Work . . . . .                                   | 4         |
| 1.3      | Organization of this Paper . . . . .                     | 5         |
| <b>2</b> | <b>Notation and Model</b>                                | <b>5</b>  |
| <b>3</b> | <b>Robust Distributed Arrays</b>                         | <b>8</b>  |
| 3.1      | Background: Data Availability Sampling . . . . .         | 9         |
| 3.2      | Defining Robust Distributed Arrays . . . . .             | 10        |
| <b>4</b> | <b>Our Construction</b>                                  | <b>11</b> |
| 4.1      | Overview . . . . .                                       | 12        |
| 4.2      | Subnet Discovery . . . . .                               | 13        |
| 4.3      | Construction . . . . .                                   | 14        |
| 4.4      | Robustness . . . . .                                     | 17        |
| 4.4.1    | Good Events and Admissible Schedules . . . . .           | 19        |
| 4.4.2    | Our Protocol Works, Provided Good Events Occur . . . . . | 21        |
| 4.4.3    | Good Events Occur for Admissible Schedules . . . . .     | 27        |
| 4.4.4    | Proof of <a href="#">Theorem 1</a> . . . . .             | 29        |
| 4.5      | Efficiency . . . . .                                     | 29        |
| <b>5</b> | <b>Extensions and Practical Considerations</b>           | <b>31</b> |
| <b>6</b> | <b>Benchmarks</b>                                        | <b>32</b> |
| 6.1      | Concrete Numbers from Analytical Results . . . . .       | 32        |
| 6.2      | Results from Simulation . . . . .                        | 34        |
| <b>7</b> | <b>More Related Work</b>                                 | <b>34</b> |
| 7.1      | Data Availability Sampling . . . . .                     | 34        |
| 7.2      | Distributed Hash Tables . . . . .                        | 35        |
| 7.3      | Further Related Work . . . . .                           | 36        |
| <b>A</b> | <b>Mathematical Tools</b>                                | <b>42</b> |
| <b>B</b> | <b>Subnet Discovery Protocol</b>                         | <b>42</b> |

# 1 Introduction

Blockchains are distributed systems that implement replicated state machines, whose states are regularly updated through distributed consensus protocols. In a nutshell, these systems provide two main guarantees called *liveness* and *safety*. The former ensures the continuous progression of the machine's state, while the latter guarantees that the machine never transitions into an invalid state.

**Strong Safety.** Popular blockchains, such as Bitcoin and Ethereum, are designed to offer a strong form of safety. They maintain the integrity of their state even when the majority of parties running the consensus protocol are acting maliciously. To achieve such a strong form of safety, *all* network participants need to individually verify every state transition proposed by the consensus protocol. Invalid state transitions are rejected by the network participants, regardless of whether they have been approved by consensus. While this approach provides strong safety guarantees, it also imposes heavy requirements on all network participants in terms of bandwidth, computing power, and storage, since every participant needs to download and verify every single state transition.

**Scaling Blockchains.** As blockchains grow in popularity and adoption, the size of their state continues to increase and requiring full replication of the state machine among all network participants is becoming a performance bottleneck. Towards building scalable blockchains, we would like to reduce the burden placed on the network participants, while maintaining the strong safety guarantees blockchains currently provide. Ideally, one would like a solution which does not require network participants to download all state related data to verify the validity of each state transition.

At first glance, succinct non-interactive arguments of knowledge (SNARKs) [Kil92, Mic00, Gro16] appear to provide a solution to the above problem. Whenever consensus proposes a new state transition, it can be accompanied by a succinct proof, attesting to the validity of the transition. Network participants can verify the succinct proof, without needing to download all data associated with the state transition itself. While this solution addresses the question of how to ensure the validity of state transitions, it does not address the question of how to ensure *availability* of all state related data. In addition to succinct proofs, the outlined approach requires a way for network participants to collectively check that all data is in principle available, preferably without requiring them to download the data in full.

**Data Availability Sampling.** Towards addressing the above issue, Al-Bassam et al. [ASBK21] have introduced the concept of *data availability sampling* (DAS), which was subsequently formalized by Hall-Andersen, Simkin, and Wagner [HASW23] and then studied by several other works [HASW24, WZ24, EMA25, EA25]. In the DAS setting, a potentially malicious party encodes a data block and provides oracle access to the encoding. A set of independent verifiers randomly probe the encoding through the provided oracle. If the verifying parties independently accept the responses they receive from the adversarially controlled oracle, then the security properties of DAS ensure that the parties could pool their individual transcripts together to reconstruct some well-defined data.

The works on DAS mentioned above focus only on *how to encode* the data, but do not study *how to store it*. In a real-world setting, the encoded data would not be stored inside a hypothetical oracle, but rather within a distributed system. Network participants should be able to store new encodings or individual symbols that they have reconstructed, and query individual symbols of already existing encodings. Understanding how to construct efficient and secure distributed storage networks is of crucial importance for making DAS a viable approach for scaling real-world blockchains.

## 1.1 Our Contribution

In this work, we formally define and construct secure distributed storage systems, which we call *Robust Distributed Arrays* (RDAs). In combination with DAS encodings, RDAs enable a full-fledged approach for implementing DAS in the real world.

**Formal Model.** Our formal model for RDAs precisely defines the security guarantees that should be provided in the presence of an adversary who controls some subset of parties in the network. A particular focus is placed on avoiding idealizing or simplifying assumptions that are difficult to implement in reality. Among other things, our formal model encapsulates both honest and malicious parties joining and leaving the network in an unpredictable, maliciously scheduled fashion and it does not assume that all honest parties can directly communicate with each other by default.

**Construction.** In our model, we then construct concretely efficient RDAs and rigorously prove their security. Our construction remains responsive and secure even if a majority of participants are malicious. Concretely, we only assume that a sufficiently large *absolute* number of honest parties is online in sufficiently large time intervals, irrespective of how many malicious parties there are in the network. Our solution incurs minimal latency overheads for reading or writing the data stored within the system.

From an engineering perspective, our construction is conceptually simple, making it easier to implement securely. It can be adapted to different trade-offs between the required data storage per party, the required bandwidth per party, and the required number of honest parties in the system. We evaluate the practical performance of our system across different parameter settings and demonstrate that it achieves provable security, while maintaining high efficiency in many realistic scenarios. As an example, we show that 5000 honest network participants, each storing 1% of the data and being connected to 10% of the other peers, allow for provably ensuring that 90% of the data are available at all points in time.

## 1.2 Related Work

Several existing lines of research works are related to what we do here. All of them fall short of providing a networking solution for DAS in one or multiple ways. In the following, we will provide a short overview of the main related works and defer a more in-depth comparison to [Section 7](#).

**Distributed Hash Tables.** Conceptually, the systems we are aiming to build in this work are strongly reminiscent of distributed hash tables (DHTs). These are data structures, maintained by a network of parties, that allow for storing key/value pairs and searching for values corresponding to given keys. A large body of research works has studied how to construct asymptotically and concretely efficient DHTs, with the most notable constructions being CAN [\[RFH<sup>+</sup>01\]](#), Pastry [\[RD01\]](#), Chord [\[SMK<sup>+</sup>01\]](#), and Kademlia [\[MM02\]](#). Commonly, DHTs prioritize minimizing the number of peers that each network participant needs to be connected to, while still being able to store and retrieve data efficiently. The above works show that, in a network with  $n$  parties, each party only needs to maintain  $\text{polylog}(n)$  peer-to-peer connections to store and retrieve data with latency  $\text{polylog}(n)$ . All of the above works only consider benign settings without the presence of an adversarial power that may try to maliciously disturb the network.

**Robust Distributed Hash Tables.** Motivated by the lack of security guarantees for DHTs in the presence of malicious behavior, several works have studied DHTs in the context of various adversarial influences [\[FS02, NW03, AS04, FSY05, Sch05, AS06b, AS06a, AS07, BM07, SF12, YKGK13, GHK13\]](#). Many of these works rely on quorum-based approaches for providing security guarantees. From a high-level perspective, the idea behind these approaches is to take a data structure that works in the benign setting and to then replace individual parties by sets of parties called quorums. One then argues that the data structure works correctly in combination with the quorums, as long as each quorum contains an honest majority of parties. As the benign DHTs, these approaches all incur polylogarithmic latencies per operation.

In contrast to the above works on DHTs, we consider a highly adversarial setting, where a large number of network participants may be corrupted. Whereas the above approaches require (at least) an honest majority among all network participants to ensure honest majorities in all quorums, our RDAs remain secure and functional in the presence of a dishonest majority. Furthermore, our construction only incurs small *constant latencies* per operation, rather than polylogarithmic ones. On the downside, our construction requires each network participant to maintain approximately  $\sqrt{n}$  peer-to-peer connections.

**Networking for DAS.** A few works have explored the networking in the specific context of DAS. Król et al. [\[KAR<sup>+</sup>23\]](#) informally identify functionality and efficiency requirements for networks underlying DAS and discuss why classical solutions, like DHTs, are not directly applicable. Cortes-Goicoechea et al. [\[CKR<sup>+</sup>24\]](#) analyze the suitability of Kademlia [\[MM02\]](#) in the context of DAS in terms of efficiency, but not security. Ascigil et al. [\[AKP<sup>+</sup>24\]](#) propose a networking solution for DAS, which has similarities with our approach. Unfortunately their approach requires each network participant to have almost complete knowledge of all other parties. In addition, their work lacks both formal security definitions and proofs.

In our work, we provide security notions for the networking layer and prove that our solution provides the stated security guarantees. Our approach does not require network participants to have complete or close to complete knowledge of the network.

### 1.3 Organization of this Paper

In [Section 2](#), we introduce some notation and our model, including network and adversarial model. In [Section 3](#), we give our definition of robust distributed arrays and explain how it relates to data availability sampling. We present and analyze our construction in [Section 4](#). This section also contains an informal overview of our construction and the definition of an abstract subprotocol, namely, (robust) subnet discovery protocols. We give an example implementation of such a protocol in [Appendix B](#). We discuss several extensions of our protocol that are useful in practice in [Section 5](#), give benchmarks in [Section 6](#), and discuss related work more extensively in [Section 7](#).

## 2 Notation and Model

Here, we specify notation and our model. We recall relevant mathematical tools in [Appendix A](#).

**Notation.** We denote the set of the first  $N$  positive integers by  $[N] = \{1, \dots, N\}$ . From time to time, we use standard asymptotic notation like  $\Omega$ ,  $O$ ,  $\Theta$ ,  $\omega$ , where the variable over which it is defined shall be clear from the context. We usually use typewriter font (e.g., `msg` instead of  $\text{msg}$ ) to distinguish explicit strings from variables. For example, we may say that a party sets  $x := 5$  and sends  $(\text{store}, x)$  to another party, which means that `store` is a string and the string  $(\text{store}, 5)$  is actually sent.

**Pseudocode.** When writing our protocols, we give both a verbal description as well as pseudocode. The pseudocode specifies both instructions that parties execute when interfaces of the protocol are called by applications, as well as message handlers that are executed when receiving certain messages. We use a standard imperative syntax for our pseudocode, including constructs like `for`, `if`, `parse`. We also assume instructions to send messages (`send . . . to . . .`), to wait for an amount of time (`wait for . . .`), and to receive messages within an amount of time (`on receiving . . . within . . .`). Unless specified otherwise, all maps and sets are implicitly initialized as being empty and all numerical variables are initialized as zero.

**Random Oracle Model.** We will analyze our construction in the random oracle model [\[BR93\]](#). To recall, all parties in this model have oracle access to a function  $H: \mathcal{X} \rightarrow \mathcal{Y}$ , which is uniformly random. That is, we assume that initially, for all  $x \in \mathcal{X}$ , we have  $H(x) = \perp$ . This holds until the first party queries  $H(x)$ . At this point, a random  $y \in \mathcal{Y}$  is sampled uniformly and we define  $H(x) := y$ , before returning  $H(x)$  to the party. Any subsequent call  $H(x)$  then returns  $y$ . In particular, all outputs are independently sampled from  $\mathcal{Y}$ . In other words, the random oracle is a uniform random function from  $\mathcal{X}$  to  $\mathcal{Y}$ . Any randomness in our experiments is also taken over the random coins used to determine the values of  $H$ , i.e., over the random oracle.

**Network Model.** Throughout, we use the terms *node* and *party* interchangeably. We consider a synchronous network, i.e., any message is delivered within a finite and known delay, and all parties have synchronized clocks. This allows protocols and our security experiment to proceed in rounds, which we also refer to as points in time. Precisely, if a party sends a message in round  $\tau$ , then it is received at some point in round  $\tau + 1$ . Strictly speaking, the message could already arrive in round  $\tau$ , but we assume for convenience that it arrives exactly in round  $\tau + 1$ . This is without loss of generality, as the receiving party can simply wait until  $\tau + 1$  before processing the message. The network we consider is open (aka permissionless), i.e., parties can join and leave over time. Parties cannot automatically communicate with each other. Instead, a party  $\mathcal{P}$  can only send a message to  $\mathcal{P}'$  if it knows their address, which it could have learned from some third party. As soon as  $\mathcal{P}$  knows the address of  $\mathcal{P}'$ , we assume that  $\mathcal{P}$  can initiate communication and the two parties can communicate via an (not necessarily private) authenticated point-to-point channel. A party always knows its own address. Of course, if a party does not already know any other party when joining, it could never reach any other party. Therefore, we will assume that when a party joins, it gets as input a list of *bootstrap nodes*. We discuss below how this list is specified. In our concrete protocol, only a subset of the parties plays the role of bootstrap nodes and nodes are aware of their own role.

**Protocols.** A protocol  $\Pi$  is specified by an ensemble of interfaces and corresponding instructions. Applications running on a node can call such an interface with some inputs, which results in the node executing the instructions specified by the protocol, e.g., sending messages to other nodes. A protocol must also specify instructions for joining, and instructions to react to incoming messages. We call the

parties that are currently online and take part in a protocol *active* for  $\Pi$ . We assume that  $\Pi$  also specifies initialization instructions  $\Pi.\text{INIT}$  to be executed in a coordinated setup<sup>1</sup> by a set  $\mathbf{P}$  of initially active honest parties that know each other. This allows protocols to build some initial topology among these parties.

**Adversarial Model.** We consider an adversary that is in full control of a subset of all parties, and we call these parties *malicious*, *adversarial*, or *byzantine*. The set of malicious parties is assumed to be fixed and disjoint from the honest parties, but not necessarily bounded. That is, we consider some form of *static* corruptions in which honest parties remain honest. For security, we will tolerate arbitrarily many malicious parties, i.e., we do not assume any protection against Sybil attacks. On the other hand, efficiency will depend on the total number of parties, so that adding Sybil protection on top is a good idea. Malicious parties can deviate arbitrarily from the protocol and collude at all times, and our adversary is rushing. This means that it sees all messages of honest parties in a certain round before deciding on its own actions. We will assume that the adversary is *computationally bounded*, and treat cryptographic primitives as idealized objects, as common in distributed systems literature. When it comes to honest parties, we let the adversary adaptively schedule their interface calls and inputs.

**Join and Leave Schedules.** While joining and leaving of malicious parties is under full control of the adversary, we assume that honest parties join and leave according to an arbitrary schedule, which is not controlled by the adversary. More precisely, a *join-leave schedule* for a protocol  $\Pi$  is given by a function<sup>2</sup>  $S$  that outputs the following data  $S(\tau)$  for each point in time  $\tau \geq 0$ :

- A list of honest parties  $\mathbf{P}$  that are supposed to join the protocol in round  $\tau$ , with special considerations at time  $\tau = 0$ , see below.
- For each such party  $\mathcal{P} \in \mathbf{P}$ , a set  $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}$  of *bootstrap* nodes (depending on  $\mathcal{P}$ ) via which  $\mathcal{P}$  should join.
- For each such party  $\mathcal{P} \in \mathbf{P}$ , some auxiliary data *role* (depending on  $\mathcal{P}$ ).
- A list of honest parties  $\tilde{\mathbf{P}}$  that are supposed to leave in round  $\tau$ .

Without loss of generality, we only consider schedules in which only parties are instructed to leave that are currently active<sup>3</sup>. Otherwise, if an honest party did not yet join (or already left again) but should leave according to the schedule, this can simply be ignored. The list  $\mathbf{P}$  of honest parties output at time  $\tau = 0$  are the parties that run the coordinated setup. Each of those parties runs  $\Pi.\text{INIT}(\mathbf{P}, \text{role}_1, \dots, \text{role}_\ell)$ , where the  $\text{role}_i$  are their respective auxiliary data as above. See [Definition 1](#) for a more precise description. For time  $\tau > 0$ , if the join-leave schedule specifies that  $\mathcal{P}$  should join via  $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}$  with auxiliary data *role*, the party  $\mathcal{P}$  instead calls  $\Pi.\text{JOIN}(\{\mathcal{P}_1, \dots, \mathcal{P}_t, \mathcal{P}'_1, \dots, \mathcal{P}'_{t'}\}, \text{role})$ , where  $\mathcal{P}'_1, \dots, \mathcal{P}'_{t'}$  are additional (potentially malicious) bootstrap nodes adaptively chosen by the adversary. Honest parties never join a second time after they have left. Equivalently, we assume that honest nodes get a fresh identifier and forget all previous state if they were to re-join. Also, we require that any honest node  $\mathcal{P}_i$  that is designated as a bootstrap node for the joining of  $\mathcal{P}$  by  $S(\tau)$  at time  $\tau$  is actually active at time  $\tau - 1$ , i.e.  $\mathcal{P}_i$  joined itself *before*  $\mathcal{P}$  and did not yet leave. Note that  $\text{JOIN}$  might take multiple rounds to complete<sup>4</sup> and other interface calls are not meaningful until then; we model this by having our security definitions provide no guarantees for interfaces that are called before  $\text{JOIN}$  has completed.

Our security definitions will universally quantify over join-leave schedules  $S$ . Importantly, we remark that this means joining and leaving of honest parties is *independent* of the protocol execution

<sup>1</sup>For this work, we only need protocols where the coordinated setup completes within a single round at time  $\tau = 0$ . We use this to simplify the definitions.

<sup>2</sup>We do not allow the join-leave schedule to depend on the random oracle, i.e., the random oracle is instantiated after the join-leave schedule (as a function) is fixed.

<sup>3</sup>Due to how our experiment works, “active” at time  $\tau$  means that the join-leave schedule specified it as a party that should join at some time  $\tau' \leq \tau$ , but did not specify it as a party that should leave at any time  $\tau'' \leq \tau$ . Note that we consider parties active in the time slots where they join and leave.

<sup>4</sup>There are multiple ways to formally model algorithms spanning multiple rounds (e.g. via a “sleep functionality” or sending messages to yourself and stop, then resume upon receiving that message). For this work, we assume that each interface call and processing each received message each starts a new thread on the affected party. We then have a “sleep and wake up the next round (but in the same thread)” operation available. This is solely to make it easier to precisely talk about terminating calls or returning values from calls: interface calls terminate (possibly with a returned value) iff the thread that was started by that call finishes.



and random coins sampled during the protocol. In particular, parties that decide to join or leave based on the status of the protocol but otherwise behave honestly are treated as malicious. Further, we allow protocols to make assumptions on the schedule: our security notions require us to define a class  $\mathcal{S}$  of *admissible* schedules and we then only quantify over all admissible schedules  $S \in \mathcal{S}$ . One example may be the class of all schedules in which honest parties always stay active for at least time  $\Delta$ , or the class of all schedules such that there is always at least one active honest node. The admissibility condition might also put requirements on the number of parties involved in the coordinated setup.

With these assumptions, we have now implicitly specified a security experiment, and our security definitions for protocols will make use of this experiment. We make the experiment explicit in the following definition.

**Definition 1** (Security Experiment). The experiment proceeds in rounds, starting at  $\tau = 0$ . For a given adversary  $\mathcal{A}$  and a join-leave schedule  $S$ , it works as follows:

1. *Initialization.* At  $\tau = 0$ , we consider the list of honest parties  $\mathbf{P} = (\mathcal{P}_1, \dots, \mathcal{P}_\ell)$  determined by  $S(0)$ . For each such party  $\mathcal{P}_i \in \mathbf{P}$ , we have some auxiliary data  $\text{role}_i$  that depends on  $i$ . Note that the set of bootstrap nodes is necessarily empty (because those would need to have joined strictly before time 0). Each of those initially active parties  $\mathcal{P}_i$  runs  $\Pi.\text{INIT}(\mathbf{P}, \text{role}_1, \dots, \text{role}_\ell)$ . We require this to not span multiple rounds. Next, the adversary  $\mathcal{A}$  is started. It has oracle access to  $S$ . We skip Steps 2–3 for  $\tau = 0$ .
2. *Honest Parties Join.* In each round, honest parties join according to  $S(\tau)$ . For each joining honest party  $\mathcal{P}$ , call  $\Pi.\text{JOIN}(\{\mathcal{P}_1, \dots, \mathcal{P}_t, \mathcal{P}'_1, \dots, \mathcal{P}'_{t'}\}, \text{role})$ , where  $\mathcal{P}_1, \dots, \mathcal{P}_t$  and  $\text{role}$  are specified by  $S(\tau)$  for  $\mathcal{P}$  and  $\mathcal{P}'_1, \dots, \mathcal{P}'_{t'}$  are specified by the adversary as in Step 5(c)iv. Also, continue any  $\text{JOIN}$ -calls that were started in past rounds, but did not finish yet.
3. *Honest Party Actions.* In each round, a list of events (specified in Step 5c) is executed as follows
  - (a) The event specifies an honest party  $\mathcal{P}$ , an interface of the protocol, and a set of inputs.
  - (b) The event is triggered by calling the interface with the respective inputs on party  $\mathcal{P}$ , provided  $\mathcal{P}$  is active.

Furthermore, active honest parties process incoming messages and perform actions based on them. Also, active honest parties continue processing actions from previous rounds if the protocol specifies that those actions span multiple rounds;

typically, the latter happens due to waiting for some other processing to complete or waiting for incoming messages. We stress that, unless specified otherwise, this means that waiting does *not* block parties from processing new event or handling other messages.

4. *Leaving.* At the end of each round  $\tau$ , honest parties leave according to  $S(\tau)$ .
5. *Adversarial Actions.* In each round,  $\mathcal{A}$  can take the following actions, after observing the messages that honest parties send:
  - (a) Create malicious parties.
  - (b) Determine the behavior (e.g., messages) of all malicious parties in this round;
  - (c) Determine the events for the next round (cf. Step 3); this means
    - i. Determine which interface calls (excluding  $\text{INIT}$  and  $\text{JOIN}$ ) are executed on honest parties and their inputs.
    - ii. Send messages originating from any malicious party to any honest party (thereby adding message processing events).
    - iii. Determine the order in which interface calls and messages are processed by active honest parties (including continuations from previous rounds, if processing spans multiple rounds).
    - iv. Determine the additional bootstrap nodes used by the parties that join in the next round. The adversary can query  $S$ , so it knows which honest parties will join in the future.

Recall that we assume authenticated channels and we assume a fixed (but unbounded) set of malicious parties, so creating a malicious party really means creating communication channels to honest parties. Also, since the adversary is byzantine, there is no need for its parties to join the protocol via  $\Pi.\text{JOIN}$ ; it just sends appropriate messages.

We assume that the experiment does not terminate, but we only require that our security properties hold for a certain lifetime  $T \in \mathbb{N}$ .

*Remark 1 (Setup).* In our model, we assume that the initial parties are all honest. This is mostly for simplicity because we want to model that our protocol starts with a defined state. The assumption can be weakened to only require that the set  $\mathbf{P}_{\text{hon}}$  of initial honest parties know each other. In this case, every honest initial party  $\mathcal{P} \in \mathbf{P}_{\text{hon}}$  is initialized with  $\Pi.\text{INIT}(\mathbf{P}_{\text{hon}} \cup \mathbf{P}_{\text{adv}}^{\mathcal{P}}, \text{role}_1, \dots)$  where  $\mathbf{P}_{\text{adv}}^{\mathcal{P}}$  are adversarially chosen parties. We remark that the protocol we define in [Section 4](#) is also secure in this more general model, provided we instantiate it with an appropriate subnet protocol, such as the concrete subnet protocol in [Appendix B](#).

Our robustness notions defined later (e.g., [Definition 4](#)) and our security proofs will heavily make use of certain events that may happen during a run of this experiment. For ease of reference, we provide a table on [Page 47](#) of all such events defined throughout the document. Let us start by defining some events related to activity of parties that we will need very often, and state a trivial proposition that follows from the definition and our assumptions that parties never re-join.

**Definition 2 (Activity Events).** Let  $0 \leq \tau \leq \tau'$  be time slots and let  $\mathcal{P}$  be an honest party. Consider a run of our experiment from [Definition 1](#) for some protocol and some join-leave schedule  $S$ . Then, we define the following events.

- Event  $\text{Active}[\mathcal{P}, \tau]$ : This event occurs if at time  $\tau$ , honest party  $\mathcal{P}$  is active.<sup>5</sup>
- Event  $\text{Active}[\mathcal{P}, \tau .. \tau']$ : This event occurs if honest party  $\mathcal{P}$  is active (at least) from time  $\tau$  to  $\tau'$  (both inclusive).
- Event  $\text{FullyJoined}[\mathcal{P}, \tau .. \tau']$ : This event occurs if the honest party  $\mathcal{P}$  is active (at least) from  $\tau$  to  $\tau'$  and either is one of the initial parties or its  $\text{JOIN}$ -call has finished by time slot  $\tau$ .<sup>6</sup>
- Event  $\text{FullyJoined}[\mathcal{P}, \tau]$ : This is short for  $\text{FullyJoined}[\mathcal{P}, \tau .. \tau]$ .

**Proposition 1 (Activity is an Interval).** Let  $\mathcal{P}$  be an honest party and  $0 \leq \tau_0 \leq \tau_1$  be points in time. Then, for any point in time  $\tau$  with  $\tau_0 \leq \tau \leq \tau_1$ , we have

$$\begin{aligned} \text{Active}[\mathcal{P}, \tau_0] \wedge \text{Active}[\mathcal{P}, \tau_1] &\implies \text{Active}[\mathcal{P}, \tau], \text{ and} \\ \text{FullyJoined}[\mathcal{P}, \tau_0] \wedge \text{Active}[\mathcal{P}, \tau_1] &\implies \text{FullyJoined}[\mathcal{P}, \tau] . \end{aligned}$$

Consequently, we have

$$\begin{aligned} \text{Active}[\mathcal{P}, \tau_0] \wedge \text{Active}[\mathcal{P}, \tau_1] &\iff \text{Active}[\mathcal{P}, \tau_0 .. \tau_1], \text{ and} \\ \text{FullyJoined}[\mathcal{P}, \tau_0] \wedge \text{Active}[\mathcal{P}, \tau_1] &\iff \text{FullyJoined}[\mathcal{P}, \tau_0 .. \tau_1] . \end{aligned}$$

We will use these properties without explicitly referring to this proposition.

*Proof.* This follows from the definitions and the fact that parties never re-join.  $\square$

### 3 Robust Distributed Arrays

Now, we present the definition of robust distributed arrays motivated by data availability sampling.

<sup>5</sup>In our experiment that we introduced in [Definition 1](#), an honest party  $\mathcal{P}$  always joins at the *beginning* of a round  $\tau_1$  (or is one of the initial parties, in which case  $\tau_1 = 0$ ) and leaves at the *end* of a round  $\tau_2$  (or never leaves, in which case  $\tau_2 = \infty$ ). This means that  $\mathcal{P}$  is active precisely from time  $\tau_1$  until  $\tau_2$ . Since  $\mathcal{P}$  will not join/leave in the middle of a round, we are guaranteed that interfaces can be called and all messages are processed by  $\mathcal{P}$  in any round during which  $\mathcal{P}$  is active.

<sup>6</sup>Recall that in our experiment [Definition 1](#), the  $\text{JOIN}$ -call will finish at the *beginning* of some round  $\tau_0$ , even if that  $\text{JOIN}$ -call takes multiple rounds. For that round  $\tau_0$  itself, we consider  $\mathcal{P}$  already fully joined, so  $\text{FullyJoined}[\mathcal{P}, \tau_0]$  holds.



### 3.1 Background: Data Availability Sampling

To motivate our definition of robust distributed arrays, we recall the basic setting of data availability sampling (DAS). Robust distributed arrays are meant to implement the networking layer of DAS.

**Data Availability Sampling.** DAS [ASBK21, HASW23] allows clients to confirm whether data (such as a transaction list) is available on a peer-to-peer network. We can think of this process as follows:

1. A proposer holding the data redundantly encodes the data and computes a succinct commitment of this encoding.
2. The proposer distributes this encoding on a peer-to-peer network, such that it is stored across various nodes.
3. Clients download the concise commitment.
4. To verify the availability of the associated data on the network, clients randomly request specific portions of the encoded data from the network and check consistency with the commitment. If enough of these random checks are successful, the clients can infer that the data is indeed available.

Hall-Andersen, Simkin, and Wagner [HASW23] have defined a formal cryptographic framework for DAS and have analyzed several constructions. Importantly, their model and analysis concerns the encoding and verification part of DAS, but *not how the network is implemented*. That is, in their model they treat the network as a black-box by providing an oracle to the clients:

- *Correctness Case.* When modeling the requirement that all clients accept if the encoding is indeed available and provided by an honest proposer, clients are assumed to have oracle access to the encoding. That is, a client can input  $i$  to its oracle and obtains the  $i$ th symbol of the encoding.
- *Adversarial Case.* When defining security properties, the oracle is instead under full control of the adversary. In particular, the adversary can decide for each query what to return, and it can make its decisions adaptive, even taking into account which client makes the query.

The natural next question after the work of Hall-Andersen, Simkin, and Wagner is how the network would be implemented in reality, and what kind of properties it should achieve.

**Basic Network Interface.** From the description above it is clear that any implementation of the network should provide two services:

- *Storing The Encoding.* The proposer has to have a way to distribute the encoding somehow in the network.
- *Making Queries.* The clients need to be able to specify indices  $i$  and get back the  $i$ th symbol of the encoding from the network.

We will make this interface more precise when giving our formal definition, but one may already notice that the (non-distributed) data-structure that provides this kind of interface is simply an *array*.

**What Can't Go Wrong.** As Hall-Andersen, Simkin, and Wagner show, constructions of DAS remain *secure* even in case the adversary is in full control of the oracle, i.e., in full control of the network. Therefore, *no matter how* the network is implemented, it is guaranteed that (enough) clients only accept if data is indeed available. One may be tempted to think that this means we are done, and that we could just use any network implementation, but as we see next, this thought is misleading.

**What Can Go Wrong.** It turns out that the model of Hall-Andersen, Simkin, and Wagner alone does not give us *liveness*. More specifically, consider the case where we have an honest proposer that encodes the data and makes it available on the network. But consider the case that a certain portion of peers on the network acts maliciously. For example, these peers may delete the symbols of the encoding that they receive, forward incorrect symbols, or try to prevent honest peers from receiving correct data. Then, a bad implementation of the network would make clients think that data is not available (e.g., because they do not get responses for their queries), even though the proposer made it available initially. This is what our robustness definition should rule out.

Another relevant scenario that arises is that some encoded symbols are initially unavailable but later recovered by honest parties through the properties of the underlying erasure code. In this case, these

honest parties should be able to reintroduce the recovered symbols to the network, ensuring that other honest peers can access them. This mechanism is crucial for eventually reaching agreement on data availability.

### 3.2 Defining Robust Distributed Arrays

We now turn to a formal definition of robust distributed arrays, using the network and adversarial model that we have introduced in Section 2. First, we define the syntax of such protocols. As already outlined above, we require two interfaces. Namely, to allow parties to distribute encodings, we require an interface `STORE`, which takes an index  $i$  and a symbol  $x$ , and intuitively should set the  $i$ th position of the array to  $x$ . Parties can read from the array by specifying the position  $i$  in a call to an interface `GET`, which intuitively should return the  $i$ th symbol of the array. To allow for a collection of arrays, we also add a *handle*  $h$  as input to the interfaces, with the intuition that different  $h$ 's access different arrays. In the setting of DAS, we think of  $h$  as being the concise commitment that all clients download. As distributed arrays could have other applications, we refer to the encoding as a *file*.

**Definition 3** (Distributed Array). A distributed array protocol with file space  $\Sigma^m$  and handle space  $\mathcal{H}$  is a protocol  $\Pi$  offering the interfaces `STORE`, `GET`, with the following syntax:

- Interface `STORE`( $h, i, x$ ): This interface can be called by any party  $\mathcal{P}$ . It takes as input a handle  $h \in \mathcal{H}$ , an index  $i \in [m]$ , and a symbol  $x \in \Sigma$ . It does not return anything to  $\mathcal{P}$ .
- Interface `GET`( $h, i$ ): This interface can be called by any party  $\mathcal{P}$ . It takes as input a handle  $h \in \mathcal{H}$  and an index  $i \in [m]$ . It returns a symbol  $x \in \Sigma$  to  $\mathcal{P}$ .
- Interfaces `JOIN` and `INIT` as required by Section 2.

We have yet to provide a *semantic* definition of how `STORE` and `GET` should behave. To establish a meaningful definition, first consider the simplified scenario in which all parties behave honestly and what would constitute the correctness property of a (non-distributed) array: if a symbol  $x$  is stored at position  $i$  and this position is later accessed, the value  $x$  should be returned, assuming that  $x$  has not been overwritten in the interim. To simplify our lives, we will disregard this overwriting assumption and instead focus on position-symbol pairs satisfying a predicate  $\text{Pred}(h, i, x)$ , where  $\text{Pred}$  models the *verification* of responses in DAS. Crucially, we assume that  $\text{Pred}$  is position-binding (see Remark 2).

Our goal is to define a robustness property that mirrors the correctness property of traditional, non-distributed arrays, but within a distributed and adversarial context. Specifically, we want to ensure that if an honest party calls `STORE`( $h, i, x$ ) and another honest party subsequently calls `GET`( $h, i$ ), the latter retrieves  $x$ . Naturally, this requirement must be adapted to accommodate the distributed and adversarial nature of our setting.

First, the fact that we are in a distributed setting necessitates several changes. For instance, a symbol stored via `STORE` might not be immediately accessible through `GET`, as time is required for the symbol to propagate through the system. To address this, we introduce two delay parameters:

- $\Delta_{\text{get}}$ : the delay between invoking `GET` and receiving a response.
- $\Delta_{\text{store}}$ : the delay required after an `STORE` call before the stored symbol can be reliably accessed.

Also, we may need to make some assumptions on how (many) parties join and when, so we parameterize the definition by a set of admissible schedules  $\mathcal{S}$ .

Second, in the presence of adversaries, we cannot expect correctness to hold universally. Instead, we allow for some tolerance of adversarial influence:

- For a small  $\varepsilon$ -fraction of indices  $\text{Corr}_{\mathcal{P}}^{\tau} \subseteq [m]$ , correctness may fail. These corrupted indices may depend on the specific parties  $\mathcal{P}$  calling the interface and the current time  $\tau$ .
- All of this is then required to hold with high probability  $1 - \delta$  over the protocol's lifetime,  $T$ .

We now present the formal robustness definition. To summarize, it states that for most indices  $i$ , if a party  $\mathcal{P}_{\text{store}}$  stored the symbol  $x$  at position  $i$  at time  $\tau_{\text{store}}$ , and after some minimum delay  $\Delta_{\text{store}}$  a party  $\mathcal{P}_{\text{get}}$  calls `GET`( $h, i$ ), then this call returns  $x$  after delay  $\Delta_{\text{get}}$ .

**Definition 4** (Robustness). Let  $\Pi$  be a distributed array protocol with file space  $\Sigma^m$ , handle space  $\mathcal{H}$ , and predicate  $\text{Pred} : \mathcal{H} \times [m] \times \Sigma \rightarrow \{0, 1\}$ . Consider the experiment specified by our network and adversarial model for an adversary  $\mathcal{A}$  and a join-leave schedule  $S$ . In this experiment, consider the following events:

- Event  $\text{CalledStore}_{h,i,x}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}]$ : This event occurs if at time  $\tau_{\text{store}}$ , honest party  $\mathcal{P}_{\text{store}}$  calls  $\text{STORE}(h, i, x)$ .
- Event  $\text{CalledGet}_{h,i}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$ : This event occurs if at time  $\tau_{\text{get}}$ , honest party  $\mathcal{P}_{\text{get}}$  calls  $\text{GET}(h, i)$ .
- Event  $\text{GotResult}_{h,i,x}^{\Delta}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$ : This event occurs if  $\text{CalledGet}_{h,i}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$  occurs and at some point  $\tau \in [\tau_{\text{get}}, \tau_{\text{get}} + \Delta]$ , the call to  $\text{GET}$  returns  $x$ .

Then, we say that  $\Pi$  is  $(\text{Pred}, \varepsilon, \delta, T, \Delta_{\text{store}}, \Delta_{\text{get}}, S)$ -robust, if for every adversary  $\mathcal{A}$  and  $S \in \mathcal{S}$ , the following holds except with probability at most  $\delta$ :

There is a family of subsets  $\text{Corr}_{\mathcal{P}}^{\tau} \subseteq [m]$ , indexed by honest parties and time, such that:

- *Bounded Size*. For every honest party  $\mathcal{P}$  and any point in time  $0 \leq \tau \leq T$ , we have  $|\text{Corr}_{\mathcal{P}}^{\tau}| \leq \varepsilon m$ .
- *Correctness if not Corrupted*. For all honest parties  $\mathcal{P}_{\text{store}}, \mathcal{P}_{\text{get}}$ , all points in time  $0 \leq \tau_{\text{store}}, \tau_{\text{get}} \leq T$  with  $\tau_{\text{get}} \geq \tau_{\text{store}} + \Delta_{\text{store}}$ , all handles  $h \in \mathcal{H}$ , symbols  $x \in \Sigma$ , and indices  $i \in [m] \setminus (\text{Corr}_{\mathcal{P}_{\text{store}}}^{\tau_{\text{store}}} \cup \text{Corr}_{\mathcal{P}_{\text{get}}}^{\tau_{\text{get}}})$ , we have

$$\left( \begin{array}{l} \text{FullyJoined}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}] \\ \wedge \text{CalledStore}_{h,i,x}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}] \\ \wedge \text{Pred}(h, i, x) \\ \wedge \text{FullyJoined}[\mathcal{P}_{\text{get}}, \tau_{\text{get}} \dots (\tau_{\text{get}} + \Delta_{\text{get}})] \\ \wedge \text{CalledGet}_{h,i}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}] \end{array} \right) \implies \text{GotResult}_{h,i,x}^{\Delta_{\text{get}}}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}].$$

*Remark 2* (Position-Binding). In the context of DAS, the predicate will always be *position-binding* (see, e.g., [HASW23]). That is, no efficient adversary can output a handle  $h \in \mathcal{H}$ , a position  $i \in [m]$ , and two symbols  $x, x' \in \Sigma$  with  $x \neq x'$  and  $\text{Pred}(\text{handle}, i, x) = \text{Pred}(\text{handle}, i, x') = 1$ .

*Remark 3* (No Secrecy). We do not require any secrecy from the distributed array protocol. The adversary is allowed to immediately learn any symbol that is stored, and any query that is made.

*Remark 4* (Agreement on the Handle). The robust distributed array protocol does not specify how parties agree on the handle  $h$  (the commitment in DAS). One can assume that another protocol (e.g., a broadcast channel, realized via the blockchain) takes care of that.

**Efficiency Metrics.** We aim to minimize the following metrics:

- *Latency and Round Complexity*. What is the latency of the  $\text{STORE}$  and  $\text{GET}$  interfaces? Specifically, what are the delays  $\Delta_{\text{store}}$  and  $\Delta_{\text{get}}$ , and how many communication rounds are required when these interfaces are invoked?
- *Communication Complexity*. How much data is exchanged among honest parties during protocol execution?
- *Bandwidth per Node*. What is the data transmission and reception rate for an honest party over a fixed time period?
- *Number of Connections*. How many peer connections must a party maintain? While memory constraints are less of a concern, maintaining a high number of connections increases bandwidth demands.
- *Data per Node*. What portion of the data must each node store?

## 4 Our Construction

In this section, we present our construction and analyze it. We start with an informal overview. Then, we formally define a necessary building block. Finally, we specify our protocol and analyze it.

## 4.1 Overview

In this section, we explain the intuition behind our protocol, assuming a fixed handle  $h \in \mathcal{H}$  for simplicity. We start with a simple protocol and then introduce our protocol as a generalization that improves on it in terms of bandwidth requirements. This first solution is (almost) identical to the PANDAS proposal [AKP<sup>+</sup>24] (see Section 7.1).

**Starting Point: A Clique Network.** Robust distributed hash tables typically rely on honest-majority assumptions, and each query generates a large polylogarithmic number of messages and incurring polylogarithmic latency. Our first goal is to overcome these limitations. To achieve this, we split the file from  $\Sigma^m$  to be stored into  $k_2$  chunks<sup>7</sup> of equal size, i.e., each chunk containing  $m/k_2$  symbols. When a party joins, it is assigned responsibility to store a random chunk. We make the following assumptions: (1) when an honest party  $\mathcal{P}$  joins, all other honest parties learn about  $\mathcal{P}$  within a bounded time; (2) one can publicly determine for which chunk a given party  $\mathcal{P}$  is responsible, e.g., by assigning the chunks via a random oracle. With these assumptions, a simple protocol implementing distributed arrays with constant latency exists: to store or get a symbol at index  $i$ , one first identifies the chunk  $j \in [k_2]$  in which this symbol resides. Then, one identifies all parties responsible for chunk  $j$  and contacts them (in parallel). In the case of get, one takes the response satisfying the predicate<sup>8</sup>. By our two assumptions, it must be that all honest parties that are responsible for chunk  $j$  are contacted. Therefore, as long as *at least one* honest party is responsible for the chunk, the query succeeds. For any fixed chunk, the probability that no honest party is responsible is at most  $(1 - 1/k_2)^N$ , where  $N$  denotes (a lower bound on) the number of honest parties. If we assume  $N \geq \Omega(\lambda k_2)$ , we get a correct response with overwhelming probability in  $\lambda$ . Notably, the number of malicious parties does not matter at all, and so we have eliminated the need for an honest-majority assumption. Robustness is determined solely by the absolute number of honest parties, while the presence of adversarial parties only affects efficiency.

The downside of this protocol is that every party needs to know the identities of *all* other parties. Maintaining this global party list requires significant bandwidth. Consequently, our next goal is to remove this requirement while preserving the protocol's efficiency and robustness.

**Splitting Parties into Rows.** Our first idea towards a solution is to have  $k_1$  rows, each row copying the approach from before. For now, assume these rows are fully independent networks and that each party would be part of one row. In particular, we can think of each party as being in one cell of a  $k_1 \times k_2$  matrix, where each column is responsible for storing one chunk, and nodes within rows form a clique. If a party in row  $i$  wants to access a symbol in a chunk  $j \in [k_2]$ , it would now ask all parties in cell  $(i, j)$  for that symbol. If we can assign parties to rows in a balanced way, then each of these cliques will contain only an  $1/k_1$  fraction of all parties, and so we reduced the bandwidth requirements. Assuming for now that this is exact, then fixing a row and a chunk as before, the probability that no honest party is responsible for that chunk in that row is at most  $(1 - 1/k_2)^{N/k_1}$ . This means that our requirement on  $N$  slightly increases to  $N \geq \Omega(\lambda k_1 k_2)$ . Of course, in an open distributed protocol, there is no omniscient coordinator that could ensure that every row contains exactly  $N/k_1$  honest parties. To address this, we adopt a randomized approach: parties are assigned to *random* rows. Within their rows, they are then assigned to random cells as before. Intuitively, with a sufficiently large number of parties, each row is likely to contain a comparable number of honest parties.

**How to Store Data.** So far, it is not clear how parties in different rows actually get the data in the first place. Specifically, assume a party  $\mathcal{P}$  in row  $i$  wants to store a symbol that resides in chunk  $j \in [k_2]$ . Then, it can of course send it to every party in cell  $(i, j)$ , but because rows are completely independent, there is no way for that data to end up in cell  $(i', j)$  for some  $i' \neq i$ . This means parties in other rows cannot access this data later. To solve this issue, we introduce *column subnets*, i.e., we make it so that columns also form cliques. Then, every party in cell  $(i, j)$  that receives the data from  $\mathcal{P}$  would use this column subnet to further distribute the data to every other node in the column, thereby propagating it into other rows. We visualize the process of storing and retrieving data in Figure 1. To define the full protocol, we need to specify more details, e.g., how to sync data during joining, and what kind of assumptions we make on these column and row subnets.

<sup>7</sup>It will be clear later why we denote the number of chunks by  $k_2$ .

<sup>8</sup>Recall that we assume that the predicate is position-binding, cf. Remark 2.

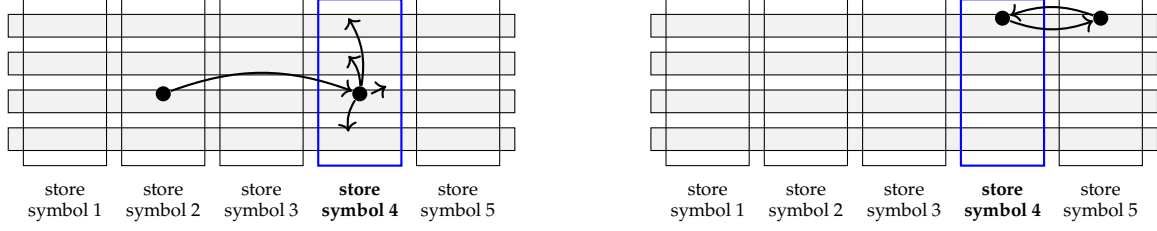


Figure 1: Visualization of our protocol  $\Pi_{\text{RDA}}$  with  $k_1 = 4$  rows and  $k_2 = 5$  columns. Each row and column represents a subnet (i.e., clique) for the subnet discovery protocol  $\Pi_{\text{SD}}$  as per Definition 5. Parties in column  $j \in [k_2]$  store the  $j$ th chunk of the data. For simplicity, we assume that the number of symbols is  $m = k_2$  in this visualization. *Left*: node in column subnet 2 and row subnet 3 storing a symbol that must be stored in column subnet 4. First, the node sends it to *all* nodes in cell (3, 4), then every such node forwards it to all nodes in that column. *Right*: node in column subnet 5 and row subnet 1 querying a symbol that must be stored in column subnet 4. First, the node sends a query to *all* nodes in cell (1, 4). Then, all nodes respond with the symbol.

## 4.2 Subnet Discovery

Before presenting the precise description of our protocol, we introduce an abstract subprotocol that we will use for implementing the row and column subnets. As outlined in our overview, we aim to maintain the intuitive invariant that in each row and column all nodes are fully connected. To model this, we introduce the definition of a *subnet discovery protocol*. This protocol provides two key functionalities: (a) a node can join a subnet through interaction with an existing member, and (b) a node can look up the list of all members of the subnet.

**Definition 5** (Subnet Discovery Protocol). A subnet discovery protocol (for at most  $M$  subnets) is a protocol  $\Pi$  offering the interfaces `CREATESUBNET`, `JOINSUBNET`, and `GETPEERS` with the following syntax:

- Interface `CREATESUBNET(sid, P)`: This interface can be called by any party  $\mathcal{P}$ . It takes a subnet identifier  $\text{sid} \in [M]$  and a set  $\mathbf{P}$  of parties. It does not return any output.
- Interface `JOINSUBNET(sid, P')`: This interface can be called by any party  $\mathcal{P}$ . It takes as input a subnet identifier  $\text{sid} \in [M]$  and a party  $\mathcal{P}'$ . It does not return anything to  $\mathcal{P}$ .
- Interface `GETPEERS(sid)`: This interface can be called by any party  $\mathcal{P}$ . It takes as input a subnet identifier  $\text{sid} \in [M]$ . It returns a set  $\mathbf{P}$  of parties to  $\mathcal{P}$ .
- Interfaces `INIT(P,  $\perp$ , ...,  $\perp$ )` and `JOIN( $\emptyset$ ,  $\perp$ )` as required by Section 2. We make the simplifying assumption that `JOIN` uses no bootstrap nodes (hence the  $\emptyset$ ) and we use no auxiliary data role (hence the  $\perp$ 's). See Remark 5.

*Remark 5* (Multiple subnets). A natural way to realize a subnet discovery protocol for  $M$  subnets is to run  $M$  many concurrent instances (parameterized by  $\text{sid}$ ) of a simplified subnet protocol for just one subnet each. However, our definition given here is more general and allows us to state the required robustness for a *concurrent* execution. Still, it is helpful to think of protocols constructed that way and we will for simplicity only consider subnet discovery protocols  $\Pi$  for which both  $\Pi.\text{INIT}$  and  $\Pi.\text{JOIN}$  complete within a single round,  $\Pi.\text{JOIN}$  does not use bootstrap nodes and we do not have extra input `role` for  $\Pi.\text{INIT}$  and  $\Pi.\text{JOIN}$ .

We now turn to defining robustness for subnet protocols. Essentially, our definition says that if two parties both join a subnet, then they learn about each other eventually. As for the robustness definition of robust distributed arrays, we allow a failure probability  $\delta$  and consider a protocol lifetime  $T$  and a set of admissible join-leave schedules  $\mathcal{S}$ . The definition also uses a delay parameter  $\Delta_{\text{sub}}$  that models the time needed to join the subnets.

**Definition 6** (Robustness). Let  $\Pi$  be a subnet discovery protocol. Consider the experiment specified by our network and adversarial model for an adversary  $\mathcal{A}$  and a join-leave schedule  $S$ . In this experiment, consider the following events:



- Event CreatedSubnet[sid, **P**]: This event occurs, if at time  $\tau = 0$ , all parties  $\mathcal{P} \in \mathbf{P}$  are honest and call the interface `CREATESUBNET(sid, P)`. Furthermore, no other calls to `CREATESUBNET` with this sid are made by honest parties at any point in time, and no calls to `JOINSUBNET` with this sid are made by honest parties at time  $\tau = 0$ .
- Event IsInSubnet $_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau_0 \dots \tau_1]$ : This event occurs for a honest party  $\mathcal{P}$  and time slots  $0 \leq \tau_0 \leq \tau_1$  if  $\mathcal{P}$  is active (at least) from time slots  $\tau_0$  to  $\tau_1$  (both inclusive) and either one of the following holds:
  - CreatedSubnet[sid, **P**] such that  $\mathcal{P} \in \mathbf{P}$ .    – or –
  - At some time  $\tau' \leq \tau_0 - \Delta_{\text{sub}}$ ,  $\mathcal{P}$  calls `JOINSUBNET(sid,  $\mathcal{P}'$ )` for an honest party  $\mathcal{P}'$  with  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}', \tau' \dots (\tau' + \Delta_{\text{sub}})]$ .
- Event IsInSubnet $_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau]$ : This event is a shorthand for  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau \dots \tau]$ .
- Event CalledGetPeers[sid,  $\mathcal{P}, \tau$ ]: This event occurs, if at time  $\tau$ , honest party  $\mathcal{P}$  calls `GETPEERS(sid)`.
- Event GotPeer[sid,  $\mathcal{P}, \mathcal{P}', \tau$ ]: This event occurs, if the event `CalledGetPeers[sid,  $\mathcal{P}, \tau$ ]` occurs and the call `GETPEERS(sid)` returns **P** at time  $\tau$  such that  $\mathcal{P}' \in \mathbf{P}$ .

With these events at hand, we say that  $\Pi$  is  $(\delta, T, \Delta_{\text{sub}}, \mathcal{S})$ -robust, if for every adversary  $\mathcal{A}$  and  $S \in \mathcal{S}$ , the following holds except with probability at most  $\delta$ : For every point in time  $\tau \in [T]$ , every subnet identifier  $\text{sid} \in [M]$ , and every pair of honest parties  $\mathcal{P}, \mathcal{P}'$ , we have:

$$\left( \begin{array}{l} \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau] \\ \wedge \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}', \tau] \\ \wedge \text{CalledGetPeers}[\text{sid}, \mathcal{P}, \tau] \end{array} \right) \implies \text{GotPeer}[\text{sid}, \mathcal{P}, \mathcal{P}', \tau] .$$

To summarize, the definition states that if a subnet has been created properly at the onset of the protocol ( $\tau = 0$ ), and an honest party  $\mathcal{P}$  is in that subnet, then `GETPEERS` returns all other honest parties in that subnet. We show how to implement a basic subnet discovery protocol that satisfies this notion in [Appendix B](#).

**Proposition 2** (One Stays in Subnets). *Let  $\Pi$  be a subnet discovery protocol for  $M$  subnets. Let  $\mathcal{P}$  be an honest party, let  $\tau \geq 0$  be a point in time, and let  $\text{sid} \in [M]$ . Let  $\Delta_{\text{sub}} \geq 0$  be any parameter. Then, we have:*

$$\left( \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau] \wedge \text{Active}[\mathcal{P}, \tau + 1] \right) \implies \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau + 1] .$$

### 4.3 Construction

Let  $k_1, k_2 \in \mathbb{N}$  and assume that we have a subnet discovery protocol  $\Pi_{\text{SD}}$  for  $M = k_1 + k_2$  subnets. Let  $\Delta_{\text{sub}} \geq 2$  be a delay parameter for the subnet discovery protocol and let  $\Delta_{\text{sync}} \geq 2$  be some synchronization delay parameter modeling the time needed to synchronize the stored data (see [Remark 6](#)). Given that, we now specify a distributed array protocol  $\Pi_{\text{RDA}}$  with arbitrary file space  $\Sigma^m$  and arbitrary handle space  $\mathcal{H}$ . For simplicity, we will assume that  $k_2$  divides  $m$ . We will make use of a hash function  $H: \{0, 1\}^* \rightarrow \{0, 1\}$  to assign parties to cells, which is modeled as a random oracle. In the following, we describe our protocol verbally. We give the full specification of  $\Pi_{\text{RDA}}$  as pseudocode in [Figures 2 to 4](#).

**Overall Structure.** The protocol will be presented by talking about  $k_1$  rows and  $k_2$  columns. For a file  $f \in \Sigma^m$ , the first  $m/k_2$  symbols  $f_1, \dots, f_{m/k_2}$  reside in column  $c = 1$ , the second  $m/k_2$  symbols  $f_{m/k_2+1}, \dots, f_{2m/k_2}$  reside in column  $c = 2$ , and so on. That is, we say that the  $i$ th symbol  $f_i$  (for  $i \in [m]$ ) resides in column  $c$ , where  $(c-1)m/k_2 < i \leq cm/k_2$ . Each party  $\mathcal{P}$  will reside in a cell  $(r, c) = \text{Cell}(\mathcal{P}) \in [k_1] \times [k_2]$ , i.e., in row  $r$  and column  $c$ . This cell is determined via the random oracle, i.e.,  $\text{Cell}(\mathcal{P}) = H(\mathcal{P})$ . With this, each other party can determine whether  $\mathcal{P}$  resides in cell  $(r, c)$ . Our protocol utilizes the subnet discovery protocol  $\Pi_{\text{SD}}$  to maintain a subnet for each row and for each column. We use functions `GetRowSid`, `GetColSid` to obtain a unique identifier for each subnet from the row resp. column index. Whenever  $r$  resp.  $c$  denotes the index of a row resp. column, we will use the shorthand notation  $\text{sid}_r := \text{GetRowSid}(r)$  resp.  $\text{sid}_c := \text{GetColSid}(c)$  for the identifier. It will always be clear from context whether such an index refers to a row or to a column. Only a subset of nodes



is utilized as bootstrap nodes. Nodes know whether they are bootstrap nodes via their initial input  $\text{role} \in \{0, 1\}$ , with  $\text{role} = 1$  indicating a bootstrap node. These nodes need to perform more work and will be in every row subnet. Other than that, they will act exactly as normal nodes, e.g., they have their single dedicated cell assigned via Cell.

**Setup and Initial Topology.** Let  $\mathbf{P} = [\mathcal{P}_1, \dots, \mathcal{P}_\ell]$  be the list of initial honest nodes specified by the join-leave-schedule  $S$ . For each such honest node  $\mathcal{P}_i \in \mathbf{P}$ , we have a corresponding bit  $\text{role}_i \in \{0, 1\}$ . Our experiment will call  $\Pi_{\text{RDA}}.\text{INIT}(\mathbf{P}, \text{role}_1, \dots, \text{role}_\ell)$  on each  $\mathcal{P} \in \mathbf{P}$ , which performs the following actions:

- Call  $\Pi_{\text{SD}}.\text{INIT}(\mathbf{P}, \perp, \dots, \perp)$  to run the coordinated setup of the subnet discovery protocol. Also, inherit any message handlers of  $\Pi_{\text{SD}}$ . By this, we mean that  $\Pi_{\text{RDA}}$  should henceforth process all messages intended for  $\Pi_{\text{SD}}$  by forwarding them to  $\Pi_{\text{SD}}$ .
- Let  $\mathbf{P}_r^{\text{row}} \subseteq \mathbf{P}$  be the subset of all parties in row  $r$ , i.e., the set of all  $\mathcal{P}_i$ 's with  $\text{Row}(\mathcal{P}_i) = r$  or  $\text{role}_i = 1$ .
- Let  $\mathbf{P}_c^{\text{col}} \subseteq \mathbf{P}$  be the subset of all parties in column  $c$ , i.e., the set of all  $\mathcal{P}_i$ 's with  $\text{Col}(\mathcal{P}_i) = c$ .
- Create each row subnet  $r \in [k_1]$  by calling  $\Pi_{\text{SD}}.\text{CREATESUBNET}(\text{sid}_r, \mathbf{P}_r^{\text{row}})$ , provided  $\mathcal{P}_{\text{self}} \in \mathbf{P}_r^{\text{row}}$ . The latter is true either for only  $r = \text{Row}(\mathcal{P}_{\text{self}})$  or for all  $r \in [k_1]$ , depending on  $\text{role}$ .
- Party  $\mathcal{P}_{\text{self}}$  creates its corresponding column subnet by calling  $\Pi_{\text{SD}}.\text{CREATESUBNET}(\text{sid}_c, \mathbf{P}_c^{\text{col}})$  for the column  $c = \text{Col}(\mathcal{P}_{\text{self}})$ .
- Party  $\mathcal{P}_{\text{self}}$  initializes an empty map  $\text{Symbols}[\cdot, \cdot]$ .
- Party  $\mathcal{P}_{\text{self}}$  stores  $\text{IsBootstrapNode} := \text{role}_i$ , where  $\mathcal{P}_{\text{self}} = \mathcal{P}_i$ .

Note that in our robustness proof, we will need to ensure that the  $\mathbf{P}_r^{\text{row}}$  and  $\mathbf{P}_c^{\text{col}}$  are non-empty and we require at least one initial bootstrap node if we ever want another node to join.

**Store.** Suppose interface  $\text{STORE}(h, i, x)$  is invoked on party  $\mathcal{P}_{\text{self}}$ . Then, the following steps are taken, but only if  $\text{Pred}(h, i, x) = 1$ :

1. Let  $c := \text{GetColForSymbol}(i) \in [k_2]$  denote the column in which symbol  $i$  resides, and let  $r \in [k_1]$  be the row in which  $\mathcal{P}_{\text{self}}$  resides.
2. Party  $\mathcal{P}_{\text{self}}$  identifies the set  $\mathbf{P}$  of all parties in cell  $(r, c)$  it is aware of:
  - (a) Get all parties  $\mathbf{P}'$  in row  $r$  via  $\Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_r)$ .
  - (b) Consider only the parties  $\mathbf{P} \subseteq \mathbf{P}'$  in column  $c$ , which can be done via H.
3. To each party  $\mathcal{P} \in \mathbf{P}$ , party  $\mathcal{P}_{\text{self}}$  sends a message  $(\text{store}, h, i, x)$ .
4. Each party  $\mathcal{P}'$  that receives  $(\text{store}, h, i, x)$  stores the symbol as  $\text{Symbols}[h, i] := x$ , provided  $\text{Pred}(h, i, x) = 1$ .
5. Each such party  $\mathcal{P}'$  then identifies the set of all parties in column  $c$  and sends  $(\text{store\_fwd}, h, i, x)$  to them.
6. Each party that receives  $(\text{store\_fwd}, h, i, x)$  stores  $\text{Symbols}[h, i] := x$ , provided  $\text{Pred}(h, i, x) = 1$ .

**Get.** Suppose interface  $\text{GET}(h, i)$  is invoked on party  $\mathcal{P}_{\text{self}}$  at time  $\tau$ . The following steps are taken:

1. Let  $c \in [k_2]$  denote the column in which symbol  $i$  resides, and let  $r \in [k_1]$  be the row in which  $\mathcal{P}_{\text{self}}$  resides.
2. Party  $\mathcal{P}_{\text{self}}$  identifies the set  $\mathbf{P}$  of all parties in cell  $(r, c)$  it is aware of, as in STORE.
3. To each party  $\mathcal{P} \in \mathbf{P}$ , party  $\mathcal{P}_{\text{self}}$  sends a message  $(\text{get}, h, i)$ .
4. Each party that receives  $(\text{get}, h, i)$  from  $\mathcal{P}_{\text{self}}$  checks if  $\text{Symbols}[h, i] = \perp$ . If not, it responds with a message  $(\text{get\_rsp}, h, i, x)$  for  $x := \text{Symbols}[h, i]$ .

5. When  $\mathcal{P}_{\text{self}}$  receives the first response ( $\text{get\_rsp}, h, i, x$ ) such that  $\text{Pred}(h, i, x) = 1$ , it outputs  $x$ .
6. If  $\mathcal{P}_{\text{self}}$  does not receive such a response by time slot  $\tau + 2$ , it outputs  $\perp$  in time slot  $\tau + 3$ .

**Join.** Suppose a party  $\mathcal{P}_{\text{self}}$  joins the protocol at time  $\tau$  via parties  $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}$  acting as bootstrap nodes. Let  $\text{role} \in \{0, 1\}$  be the provided bit designating whether  $\mathcal{P}_{\text{self}}$  may be a bootstrap node itself for parties that join in the future. The following steps are taken, where  $(r, c) := \text{Cell}(\mathcal{P}_{\text{self}})$ :

1. Call  $\Pi_{\text{SD}}.\text{JOIN}(\emptyset, \perp)$  and inherit  $\Pi_{\text{SD}}$ 's message handlers.
2. Party  $\mathcal{P}_{\text{self}}$  initializes an empty map  $\text{Symbols}[\cdot, \cdot]$  and memorizes  $\text{IsBootstrapNode} := \text{role}$ .
3. Party  $\mathcal{P}_{\text{self}}$  starts to join (in parallel) the appropriate row subnets via the bootstrap nodes. In detail:
  - (a) If  $\text{role} = 1$ , call  $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_{r'}, \mathcal{P}_i)$  for each row  $r' \in [k_1]$  and each bootstrap node  $\mathcal{P}_i$ .
  - (b) Otherwise, call  $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_r, \mathcal{P}_i)$  for each bootstrap node  $\mathcal{P}_i$ .
4. Set  $\mathbf{P}_c^{\text{col}} := \emptyset$ . To join its column subnet, party  $\mathcal{P}_{\text{self}}$  first sends  $\text{join}$  to each of its bootstrap nodes  $\mathcal{P}_i$  in parallel. Each bootstrap node reacts as follows:
  - (a) Lookup all nodes via the row subnets, i.e., call  $\Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_r)$  for each row  $r \in [k_1]$  and let  $\mathbf{P}$  be the union of the results.
  - (b) Determine  $c := \text{Col}(\mathcal{P})$ , where  $\mathcal{P}$  is the sender of the received  $\text{join}$  message.
  - (c) Let  $\mathbf{P}_c^{\text{col}} \subseteq \mathbf{P}$  be the set of parties  $\mathcal{P}' \in \mathbf{P}$  in column  $c$ .
  - (d) Send  $(\text{join\_rsp}, \mathbf{P}_c^{\text{col}})$  to  $\mathcal{P}$ .
5. Party  $\mathcal{P}_{\text{self}}$  waits, expecting back the message  $(\text{join\_rsp}, \mathbf{P}_c^{(i)})$  from each  $\mathcal{P}_i$  within two time slots.
6. Upon receiving each such message within two time slots:
  - (a) Call  $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_c, \mathcal{P}')$  for every  $\mathcal{P}' \in \mathbf{P}_c^{(i)} \setminus \mathbf{P}_c^{\text{col}}$  at time  $\tau + 2$ .
  - (b)  $\mathbf{P}_c^{\text{col}} := \mathbf{P}_c^{\text{col}} \cup \mathbf{P}_c^{(i)}$ .
7. Wait for  $\Delta_{\text{sub}}$  additional time until  $\tau + 2 + \Delta_{\text{sub}}$ .
8. At time slot  $\tau + 2 + \Delta_{\text{sub}}$ ,  $\mathcal{P}_{\text{self}}$  starts to synchronize data as follows:
  - (a)  $\mathcal{P}$  updates  $\mathbf{P}_c^{\text{col}} = \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_c)$ .
  - (b)  $\mathcal{P}_{\text{self}}$  sends  $\text{sync}$  to every  $\mathcal{P}' \in \mathbf{P}_c^{\text{col}}$ .
  - (c)  $\mathcal{P}_{\text{self}}$  terminates<sup>9</sup>  $\text{JOIN}$  now in time slot  $\tau + 2 + \Delta_{\text{sub}}$ . Note that  $\mathcal{P}_{\text{self}}$  will still handle  $\text{sync\_rsp}$  answers as described below, even though  $\text{JOIN}$  has terminated.
9. When a party  $\mathcal{P}'$  receives  $\text{sync}$ , it does the following
  - (a)  $\mathcal{P}'$  sets
 
$$\mathbf{S} := \{(h, i, \text{Symbols}^{\mathcal{P}'}[h, i]) \mid \text{Symbols}^{\mathcal{P}'}[h, i] \neq \perp\},$$
 where  $\text{Symbols}^{\mathcal{P}'}$  refers to the  $\text{Symbols}$  map stored by  $\mathcal{P}'$ .
  - (b)  $\mathcal{P}'$  responds with  $(\text{sync\_rsp}, \mathbf{S})$  after some arbitrary delay  $T \leq \Delta_{\text{sync}} - 2$ . See [Remark 6](#) for an explanation of the delay. Note that  $\mathbf{S}$  is determined at the time of receiving  $\text{sync}$ .
10. For each message  $(\text{sync\_rsp}, \mathbf{S})$  that  $\mathcal{P}_{\text{self}}$  obtains, it stores  $\text{Symbols}[h, i] = x$  for each  $(h, i, x) \in \mathbf{S}$  with  $\text{Pred}(h, i, x) = 1$ .

---

<sup>9</sup>The significance of formally terminating  $\text{JOIN}$  here before we actually synched the data is that  $\text{FullyJoined}[\mathcal{P}_{\text{self}}, \tau + 2 + \Delta_{\text{sub}}]$  holds. This means that starting from round  $\tau + 2 + \Delta_{\text{sub}}$ , we may already call  $\text{STORE}$  and  $\text{GET}$  and enjoy the guarantees of the robustness definition, even though synchronization of the data has not yet finished.

To summarize, each normal node only joins its row and column subnets, and each bootstrap node ( $\text{role}_i = 1$ ) additionally joins *all* row subnets. After time  $\Delta_{\text{sub}}$ , the new node will receive relevant store and store\_fwd messages and learn about new data; we can call STORE and GET from this point on. To learn data that was stored prior to this point in our column, we send a sync message to other nodes in our column to synchronize that data.

*Remark 6 (Synchronization Delay).* As part of JOIN, a newly joining node  $\mathcal{P}_{\text{self}}$  asks other nodes  $\mathcal{P}'$  in the same column for any data that was stored in the past by sending a sync-query. In practice, the amount of data that  $\mathcal{P}'$  needs to send back to  $\mathcal{P}_{\text{self}}$  here might be very large. In our network model, we have assumed that *any* message arrives by the end of the next round, which is unrealistic in the case of large data. To accommodate for that fact in a realistic way, we have artificially introduced a delay  $\Delta_{\text{sync}}$  here. We will provide more discussion on  $\Delta_{\text{sync}}$  in Section 5.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Interface</b> <math>\text{INIT}(\mathbf{P}, \text{role}_1, \dots, \text{role}_\ell)</math></p> <p><i>// Initialize Subnet protocol</i></p> <p>01 <math>\Pi_{\text{SD}}.\text{INIT}(\mathbf{P}, \text{role}_1, \dots, \text{role}_\ell)</math></p> <p>02 <b>inherit message handlers from</b> <math>\Pi_{\text{SD}}</math></p> <p>03 <b>parse</b> <math>\mathbf{P} = [\mathcal{P}_1, \dots, \mathcal{P}_\ell]</math></p> <p><i>// create row subnets</i></p> <p>04 <b>for</b> <math>r \in [k_1]</math>:</p> <p>05   <math>\mathbf{P}_r^{\text{row}} := \{\mathcal{P}_i \mid \text{Row}(\mathcal{P}_i) = r \vee \text{role}_i = 1\}</math></p> <p>06   <b>if</b> <math>\mathcal{P}_{\text{self}} \in \mathbf{P}_r^{\text{row}}</math>:</p> <p>07     <math>\Pi_{\text{SD}}.\text{CREATESUBNET}(\text{sid}_r, \mathbf{P}_r^{\text{row}})</math></p> <p><i>// create column subnets</i></p> <p>08 <math>c := \text{Col}(\mathcal{P}_{\text{self}})</math></p> <p>09   <math>\mathbf{P}_c^{\text{col}} := \{\mathcal{P}_i \mid \text{Col}(\mathcal{P}_i) = c\}</math></p> <p><i>// note: <math>\mathcal{P}_{\text{self}} \in \mathbf{P}_c^{\text{col}}</math></i></p> <p>10 <math>\Pi_{\text{SD}}.\text{CREATESUBNET}(\text{sid}_c, \mathbf{P}_c^{\text{col}})</math></p> <p><i>// Initialize Symbols map</i></p> <p>11 <math>\text{Symbols}[\cdot, \cdot] := \emptyset</math></p> <p><i>// Memorize whether we can act as bootstrap node</i></p> <p>12 <math>\text{IsBootstrapNode} := \text{role}_i</math> <b>where</b> <math>\mathcal{P}_{\text{self}} = \mathcal{P}_i</math>.</p> <p><b>Alg</b> <math>\text{GetPeersInCell}(r, c)</math></p> <p><i>// get peers in row r</i></p> <p>13 <math>\mathbf{P}' := \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_r)</math></p> <p><i>// only return the ones in column c</i></p> <p>14 <b>return</b> <math>\mathbf{P} := \{\mathcal{P} \in \mathbf{P}' \mid \text{Cell}(\mathcal{P}) = (r, c)\}</math></p> | <p><b>Alg</b> <math>\text{Cell}(\mathcal{P})</math></p> <p><i>// cell in which party resides</i></p> <p>15 <b>return</b> <math>\text{H}(\mathcal{P})</math></p> <p><b>Alg</b> <math>\text{Row}(\mathcal{P})</math></p> <p><i>// row in which party resides</i></p> <p>16 <math>(r, c) := \text{Cell}(\mathcal{P})</math></p> <p>17 <b>return</b> <math>r</math></p> <p><b>Alg</b> <math>\text{Col}(\mathcal{P})</math></p> <p><i>// column in which party resides</i></p> <p>18 <math>(r, c) := \text{Cell}(\mathcal{P})</math></p> <p>19 <b>return</b> <math>c</math></p> <p><b>Alg</b> <math>\text{GetRowSid}(r)</math></p> <p><i>// label rows from 1 to <math>k_1</math></i></p> <p>20 <b>return</b> <math>r</math></p> <p><b>Alg</b> <math>\text{GetColSid}(c)</math></p> <p><i>// label columns from <math>k_1 + 1</math> to <math>k_1 + k_2</math></i></p> <p>21 <b>return</b> <math>k_1 + c</math></p> <p><b>Alg</b> <math>\text{GetColForSymbol}(i)</math></p> <p>22 <b>return</b> <math>c</math> <b>s.t.</b> <math>(c - 1)m/k_2 &lt; i \leq cm/k_2</math></p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 2: Initialization and helper algorithms for our distributed array protocol  $\Pi_{\text{RDA}}$ . It makes use of a subnet discovery protocol  $\Pi_{\text{SD}}$  for  $M = k_1 + k_2$  and a random oracle  $\text{H}: \{0, 1\}^* \rightarrow [k_1] \times [k_2]$ . We present code for joining in Figure 3 and interfaces STORE and GET in Figure 4.

## 4.4 Robustness

We now show that our construction  $\Pi_{\text{RDA}}$  satisfies the robustness notion defined in Definition 4. First, we observe that  $\Pi_{\text{RDA}}$  uses the subnet protocol  $\Pi_{\text{SD}}$  in a way that precisely simulates a run of the experiment specified in Definition 1. Furthermore, in this simulated experiment, honest parties are active in  $\Pi_{\text{SD}}$  iff they are active in  $\Pi_{\text{RDA}}$ . Parties are initial parties in  $\Pi_{\text{SD}}$  if and only if they are initial parties in  $\Pi_{\text{RDA}}$ . The event FullyJoined, which refers to JOIN having terminated, always means that  $\Pi_{\text{RDA}}.\text{JOIN}$  has terminated. This allows us to meaningfully use the events from Definition 2 and Definition 6 and we will talk about active parties without the need to distinguish between  $\Pi_{\text{RDA}}$  and  $\Pi_{\text{SD}}$ . Before stating the precise theorem, we give an intuition for the conditions we require for our theorem to hold: Roughly speaking, we wish that the underlying subnet protocol is robust and there are honest parties in each row, column and cell.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Interface JOIN</b> ( $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}, \text{role}$ )<br>01 <b>let</b> $\tau$ <b>be the time when JOIN is called</b><br>02 $\Pi_{\text{SD}}.\text{JOIN}(\emptyset, \perp)$<br>03 <b>inherit message handlers from</b> $\Pi_{\text{SD}}$<br>04 $(r, c) := \text{Cell}(\mathcal{P}_{\text{self}})$<br>05 $\text{Symbols}[\cdot, \cdot] := \emptyset, \text{IsBootstrapNode} := \text{role}$<br>06 <b>if</b> $\text{IsBootstrapNode} = 1$ :<br><i>// join all row subnet via all bootstrap nodes</i><br>07 <b>for</b> $i \in [t], r' \in [k_1]$ :<br>08 $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_{r'}, \mathcal{P}_i)$<br>09 <b>else</b> :<br><i>// join own row subnet via all bootstrap nodes</i><br>10 <b>for</b> $i \in [t]$ : $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_r, \mathcal{P}_i)$ .<br><i>// ask bootstrap nodes for column peers</i><br>11 $\mathbf{P}_c^{\text{col}} := \emptyset$ .<br>12 <b>for</b> $i \in [t]$ : <b>send</b> join <b>to</b> $\mathcal{P}_i$<br>13 <b>on receiving</b> (join_rsp, $\mathbf{P}_c^{(i)}$ ) <b>from each</b> $\mathcal{P}_i$<br><b>within 2 rounds, do in round</b> $\tau + 2$ :<br>14 <b>for</b> $\mathcal{P}' \in \mathbf{P}_c^{(i)} \setminus \mathbf{P}_c^{\text{col}}$ :<br>15 $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_c, \mathcal{P}')$<br>16 $\mathbf{P}_c^{\text{col}} := \mathbf{P}_c^{\text{col}} \cup \mathbf{P}_c^{(i)}$ .<br><i>// Wait until we can be sure other nodes find <math>\mathcal{P}_{\text{self}}</math>,</i><br><i>// so relevant new data will be learnt via STORE</i><br>17 <b>wait for</b> $\Delta_{\text{sub}}$ <b>until</b> $\tau + 2 + \Delta_{\text{sub}}$<br><i>// sync old data via column subnet</i><br><i>// ask everyone for their data</i><br>18 $\mathbf{P}_c^{\text{col}} := \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_c)$<br>19 <b>for</b> $\mathcal{P}' \in \mathbf{P}_c^{\text{col}}$ : <b>send</b> sync <b>to</b> $\mathcal{P}'$<br>20 <b>terminate JOIN</b> | <b>On Message sync from</b> $\mathcal{P}$<br>21 $\mathbf{S} := \{(h, i, \text{Symbols}[h, i]) \mid \text{Symbols}[h, i] \neq \perp\}$<br><i>// model that this may be slow.</i><br><i>// see Remark 6</i><br>22 <b>wait for arbitrary time</b> $T \leq \Delta_{\text{sync}} - 2$<br>23 <b>send</b> (sync_rsp, $\mathbf{S}$ ) <b>to</b> $\mathcal{P}$<br><br><b>On Message (sync_rsp, <math>\mathbf{S}</math>) from</b> $\mathcal{P}$<br>24 <b>for</b> $(h, i, x) \in \mathbf{S}$ :<br>25 <b>if</b> $\text{Pred}(h, i, x) = 1$ : $\text{Symbols}[h, i] := x$<br><br><b>On Message join from</b> $\mathcal{P}$<br>26 <b>if</b> $\text{IsBootstrapNode} = 0$ :<br>27 <b>return</b><br>28 $c := \text{Col}(\mathcal{P})$<br><i>// collect all nodes we know</i><br>29 $\mathbf{P} := \emptyset$<br>30 <b>for</b> $r \in [k_1]$ :<br>31 $\mathbf{P} := \mathbf{P} \cup \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_r)$<br><i>// respond with nodes in appropriate column</i><br>32 $\mathbf{P}_c^{\text{col}} := \{\mathcal{P}' \in \mathbf{P} \mid \text{Col}(\mathcal{P}') = c\}$<br>33 <b>send</b> (join_rsp, $\mathbf{P}_c$ ) <b>to</b> $\mathcal{P}$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 3: Joining instructions for our distributed array protocol  $\Pi_{\text{RDA}}$  with file space  $\Sigma^m$  and handle space  $\mathcal{H}$ , for a predicate  $\text{Pred}: \mathcal{H} \times [m] \times \Sigma \rightarrow \{0, 1\}$ . It makes use of a subnet discovery protocol  $\Pi_{\text{SD}}$  for  $M = k_1 + k_2$  and a random oracle  $\text{H}: \{0, 1\}^* \rightarrow [k_1] \times [k_2]$ . The party that is executing the instructions is referred to as  $\mathcal{P}_{\text{self}}$ . We present the initialization code in Figure 2, interfaces STORE and GET in Figure 4.

**Overlap times.** A closer look shows that just requiring there to be at least one active honest party assigned to each row, column and cell is not quite enough: we require there to be sufficient overlap time  $\Delta_{\text{overlap}}$  between potentially leaving old honest parties and newly arriving parties. We model this overlap by asking that active honest parties  $\mathcal{P}$  exist that remain active for at least  $\Delta_{\text{overlap}}$  more slots. This way, if a new party joins, its lifetime overlaps at least  $\Delta_{\text{overlap}}$  time slots with  $\mathcal{P}$ . The overlap time is needed both for joining the subnets and for actually handing over the data. The old party needs to have been around for a sufficient time in order to have properly joined the subnet itself and finished retrieving data from its peers. Then the old party needs to remain online for roughly the same amount of time to help new parties join their subnets and retrieve data from the old party. For that reason, the required overlap is roughly twice that of  $\Delta_{\text{sub}}$  resp.  $\Delta_{\text{sync}}$ .

**Failure recovery.** Our conditions for rows, columns and cells differ: assume that in one of our *subnets*, there is no honest party with sufficient overlap: any future party asked to join that subnet will be “tainted” by joining solely via adversarial or honest “tainted” parties. Consequently, this subnet will not provide any guarantees anymore and our protocol cannot recover from this. By contrast, having no honest party in a given bad *cell* is a *transient* failure case that our protocol can recover from. Such failures will give rise to sets  $\text{Corr}_{\mathcal{P}}^{\tau}$  as afforded by Definition 4 and we merely need to rule out that too many cells are simultaneously bad at any given time.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Interface STORE(<math>h, i, x</math>)</b><br>01 <b>if</b> $\text{Pred}(h, i, x) = 0$ : <b>return</b><br><i>// column in which the symbol should be stored</i><br>02 $c := \text{GetColForSymbol}(i)$<br><i>// my own row</i><br>03 $r := \text{Row}(\mathcal{P}_{\text{self}})$<br><i>// get all peers in cell</i><br>04 $\mathbf{P} := \text{GetPeersInCell}(r, c)$<br><i>// send them the symbol</i><br>05 <b>for</b> $\mathcal{P} \in \mathbf{P}$ : <b>send</b> $(\text{store}, h, i, x)$ <b>to</b> $\mathcal{P}$<br><b>On Message</b> $(\text{store}, h, i, x)$<br>06 <b>if</b> $\text{Pred}(h, i, x) = 0$ : <b>return</b><br><i>// store this symbol</i><br>07 $\text{Symbols}[h, i] := x$<br><i>// forward to everyone in column</i><br>08 $c := \text{GetColForSymbol}(i)$<br>09 $\mathbf{P} := \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_c)$<br>10 <b>for</b> $\mathcal{P} \in \mathbf{P}$ : <b>send</b> $(\text{store\_fwd}, h, i, x)$ <b>to</b> $\mathcal{P}$<br><b>On Message</b> $(\text{store\_fwd}, h, i, x)$<br>11 <b>if</b> $\text{Pred}(h, i, x) = 0$ : <b>return</b><br><i>// only store this symbol</i><br>12 $\text{Symbols}[h, i] := x$ | <b>Interface GET(<math>h, i</math>)</b><br>13 <b>let</b> $\tau$ <b>be the current time slot</b><br><i>// column in which symbol <math>i</math> is stored</i><br>14 $c := \text{GetColForSymbol}(i)$<br><i>// my own row</i><br>15 $r := \text{Row}(\mathcal{P}_{\text{self}})$<br><i>// get all peers in cell</i><br>16 $\mathbf{P} := \text{GetPeersInCell}(r, c)$<br><i>// ask them for symbol</i><br>17 <b>for</b> $\mathcal{P} \in \mathbf{P}$ : <b>send</b> $(\text{get}, h, i)$ <b>to</b> $\mathcal{P}$<br><i>// return the first valid response</i><br>18 <b>on receiving</b> $(\text{get\_rsp}, h, i, x)$ <b>at</b> $\tau + 2$<br><b>s.t.</b> $\text{Pred}(h, i, x) = 1$ :<br>19 <b>return</b> $x$<br>20 <b>if no such response within two rounds:</b><br>21 <b>return</b> $\perp$ <b>at time</b> $\tau + 3$<br><b>On Message</b> $(\text{get}, h, i)$ <b>from</b> $\mathcal{P}$<br><i>// only respond if I store this symbol</i><br>22 <b>if</b> $\text{Symbols}[h, i] = \perp$ : <b>return</b><br>23 $x := \text{Symbols}[h, i]$<br>24 <b>send</b> $(\text{get\_rsp}, h, i, x)$ <b>to</b> $\mathcal{P}$ |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 4: Interfaces STORE and GET of  $\Pi_{\text{RDA}}$  with file space  $\Sigma^m$  and handle space  $\mathcal{H}$ , for a predicate  $\text{Pred}: \mathcal{H} \times [m] \times \Sigma \rightarrow \{0, 1\}$ . It makes use of a subnet discovery protocol  $\Pi_{\text{SD}}$  for  $M = k_1 + k_2$  and a random oracle  $\text{H}: \{0, 1\}^* \rightarrow [k_1] \times [k_2]$ . The party that is executing the instructions is referred to as  $\mathcal{P}_{\text{self}}$ . Initialization and helper algorithms are defined in Figure 2, and code for joining in Figure 3.

**Organization.** Our robustness analysis is organized as follows: we first define several good events that capture the conditions under which we will show robustness. Then we will define our class of admissible join-leave schedules. With that definition at hand, we are able to formally state [Theorem 1](#), which asserts that our construction satisfies our robustness notion. The proof of that theorem has two main technical parts: in the first part, we show that construction satisfies what we want from the robustness notion (i.e., that we can retrieve data that was previously stored), provided certain good events occur. This first part does not use probabilities. Then, in the second part, we show that the good events from the first part actually occur with sufficiently high probability, provided that our join-leave schedule is admissible. Combined, these parts then prove [Theorem 1](#).

To simplify notation, we consider  $\Delta_{\text{sub}} \geq 2$  and  $\Delta_{\text{sync}} \geq 2$  fixed constants in the analysis and often do not write the dependency of events on these constants explicitly. For instance, we write  $\text{IsInSubnet}_{\text{sid}}[\mathcal{P}, \tau]$  and  $\text{IsInSubnet}_{\text{sid}}[\mathcal{P}, \tau_0 \dots \tau_1]$  instead of  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau]$  and  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau_0 \dots \tau_1]$ .

#### 4.4.1 Good Events and Admissible Schedules

We start by defining some good events that capture the conditions under which we will show that our protocol does not fail.

**Definition 7** (Good Events). We define the following good events:

- Event  $\text{SubnProtGoodUntil}[T]$ : This event occurs if the property we want from the robustness definition of  $\Pi_{\text{SD}}$  holds until time  $T$ , i.e., for every time  $0 \leq \tau \leq T$ , pairs of honest parties  $\mathcal{P}, \mathcal{P}'$ , subnet identifier  $\text{sid}$ , the following holds: if  $\text{IsInSubnet}_{\text{sid}}[\mathcal{P}, \tau]$ ,  $\text{IsInSubnet}_{\text{sid}}[\mathcal{P}', \tau]$ ,  $\text{CalledGetPeers}[\text{sid}, \mathcal{P}, \tau]$ , then  $\text{GotPeer}[\text{sid}, \mathcal{P}, \mathcal{P}', \tau]$ .

- Event  $\text{ColumnGoodUntil}^\Delta[c, T]$ : This event occurs for column  $c \in [k_2]$  if for every  $0 \leq \tau \leq T$ , there exists an honest party  $\mathcal{P}$  with  $\text{Col}(\mathcal{P}) = c$  and  $\text{Active}[\mathcal{P}, \tau \dots \tau + \Delta]$ .
- Event  $\text{GoodCell}[r, c, \tau_1 \dots \tau_2]$ : This event occurs for row  $r \in [k_1]$ , column  $c \in [k_2]$  and times  $\tau_1 \leq \tau_2$  if there exists an honest party  $\mathcal{P}$  with  $\text{Cell}(\mathcal{P}) = (r, c)$  and  $\text{Active}[\mathcal{P}, \max\{0, \tau_1 - \Delta_{\text{sub}} - 2\} \dots \tau_2]$ .

We remark that  $\text{ColumnGoodUntil}^\Delta[c, T]$  captures the condition about column subnets we explained above, with overlap time  $\Delta$ . We use the event  $\text{GoodCell}[r, c, \tau_1 \dots \tau_2]$  to express the conditions for individual cells. The shift of  $\tau_1$  by  $\Delta_{\text{sub}} + 2$  in the latter definition is done in order to imply that  $\text{GetPeersInCell}(r, c)$  will find an honest node in the subnets if called at a time between  $\tau_1$  and  $\tau_2$ . See [Corollary 3](#) below. Observe that we defined no event for the row subnets: for the rows, we require that there exists honest nodes with sufficient overlap *among the bootstrap nodes used to join* rather than among all nodes. This is more conveniently expressed as a condition on the join-leave schedule  $S$ , defined below as  $S$  *uses good bootstrap nodes* in [Definition 8](#).

**Definition 8** (Admissible Schedules). Let  $S$  be a join-leave schedule. We define the following terminology:

- We say that  $S$  *guarantees  $N$  honest parties with overlap  $\Delta_{\text{overlap}}$* , if for all time slots  $\tau$ , there are at least  $N$  honest parties  $\mathcal{P}$  which are active (at least) from  $\tau$  to  $\tau + \Delta_{\text{overlap}}$  (both inclusive).
- We say that  $S$  *respects bootstrap nodes* if the following holds:
  - The auxiliary data `role` provided to parties in `JOIN` or `INIT` is in  $\{0, 1\}$ . We will denote parties for which `role` = 1 as *prospective bootstrap nodes*.
  - Whenever  $S$  schedules a party  $\mathcal{P}$  to join via honest bootstrap nodes  $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}$ , each  $\mathcal{P}_i$  is a prospective bootstrap node.
- We say that  $S$  *uses good bootstrap nodes* if for every honest party joining at time  $\tau$  via bootstrap nodes  $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}$ , there exists an honest bootstrap node  $\mathcal{P}_i$  for which the following both hold:
  - Either  $\tau \geq \Delta_{\text{sub}}$  and  $\mathcal{P}_i$  is active at time  $\tau - \Delta_{\text{sub}}$  or  $\tau < \Delta_{\text{sub}}$  and  $\mathcal{P}_i$  is one of the initial parties.
  - $\mathcal{P}_i$  is active at time  $\tau + \Delta_{\text{sub}}$ .

Finally, we define the admissible join-leave schedules for  $\Pi_{\text{RDA}}$ :

Let  $N$  be a positive integer,  $\mathcal{S}_{\text{SD}}$  be a set of admissible schedules for the underlying subnet protocol and  $\Delta_{\text{overlap}} \geq \max\{2\Delta_{\text{sub}} + 2, 2\Delta_{\text{sync}} + \Delta_{\text{sub}} + 2\}$ . We define the set of  $(N, \Delta_{\text{overlap}}, \mathcal{S}_{\text{SD}})$ -admissible schedules for  $\Pi_{\text{RDA}}$  as the set of join-leave schedules which respect bootstrap nodes, use good bootstrap nodes, guarantee  $N$  honest parties with overlap  $\Delta_{\text{overlap}}$  and which are in  $\mathcal{S}_{\text{SD}}$ .

With this definition at hand, we now formally state the robustness property that we will prove. Recall that we assume that  $k_2$  divides  $m$ .

**Theorem 1 (Robustness of  $\Pi_{\text{RDA}}$ ).** Let  $\text{Pred}: \mathcal{H} \times [m] \times \Sigma \rightarrow \{0, 1\}$  be position-binding, cf. [Remark 2](#). Assume that  $\Pi_{\text{SD}}$  is  $(\delta_{\text{SD}}, T_{\text{SD}}, \Delta_{\text{sub}}, \mathcal{S}_{\text{SD}})$ -robust according to [Definition 6](#). Let  $N$  be a positive integer and  $\Delta_{\text{overlap}} \geq \Delta_{\text{overlap}, \min}$ , where  $\Delta_{\text{overlap}, \min} = \max\{2\Delta_{\text{sub}} + 2, 2\Delta_{\text{sync}} + \Delta_{\text{sub}} + 2\}$ . Then,  $\Pi_{\text{RDA}}$  is  $(\text{Pred}, \varepsilon, \delta, T, \Delta_{\text{store}}, \Delta_{\text{get}}, \mathcal{S})$ -robust according to [Definition 4](#), where

- *Admissible Schedules.* The set  $\mathcal{S}$  is defined as the set of  $(N, \Delta_{\text{overlap}}, \mathcal{S}_{\text{SD}})$ -admissible schedules in [Definition 8](#).
- *Lifetime.* We have  $T = T_{\text{SD}} - 1$ .
- *Store-Get Delay.* We have  $\Delta_{\text{store}} = 2$ .
- *Get Delay.* We have  $\Delta_{\text{get}} = 2$ .
- *Corrupted Symbol Fraction.* We may choose any  $0 < \varepsilon < 1$  (but  $\delta$  depends on it).



- **Corruption Sets.** We have corruption sets  $\text{Corr}_{\mathcal{P}}^{\tau}$  as follows: For an honest party  $\mathcal{P}$  and time slot  $\tau$ , set

$$\text{Corr}_{\mathcal{P}}^{\tau} = \{i \in [m] \mid \neg \text{GoodCell}[r, c, \max\{0, \tau - \Delta_{\text{sync}}\} \dots (\tau + 1)] \text{ for } c = \text{GetColForSymbol}(i)\}$$

where  $r = \text{Row}(\mathcal{P})$ .

- **Error.** We have an error probability  $\delta$  bounded by

$$\delta \leq \delta_{\text{SD}} + \left\lceil \frac{T + 2}{\Delta_{\text{overlap}} - \Delta_{\text{overlap}, \min} + 1} \right\rceil \cdot \left( k_1 2^{h(\varepsilon)k_2} e^{-\varepsilon \frac{N}{k_1}} + k_2 e^{-\frac{N}{k_2}} \right),$$

where  $h(\varepsilon) = -\varepsilon \log_2(\varepsilon) - (1 - \varepsilon) \log_2(1 - \varepsilon)$  is the binary entropy function.

Let us give some explanation for the individual constituents of the error bound in the above [Theorem 1](#). We refer to the proof of the theorem for more details. Clearly, the  $\delta_{\text{SD}}$ -term comes from the possibility of the subnet protocol failing. The  $\lceil (T + 2)/(\Delta_{\text{overlap}} - \Delta_{\text{overlap}, \min} + 1) \rceil$ -term comes from a union bound over the lifetime  $T$ ; the denominator  $\Delta_{\text{overlap}} - \Delta_{\text{overlap}, \min} + 1$  in that expression is an optimization that uses the fact that the failure events for different times are not independent. The expression  $k_1 2^{h(\varepsilon)k_2} e^{-\varepsilon N/k_1}$  accounts for the bad event that there are too many failing (i.e. without an active honest party with sufficient overlap) cells in any row. The expression  $k_2 e^{-N/k_2}$  accounts for the bad event that in some column, there is no honest party with sufficient overlap.

Below, we prove [Theorem 1](#) by a series of lemmas, using the strategy outlined at the beginning of the section.

#### 4.4.2 Our Protocol Works, Provided Good Events Occur

In the first part of our proof, we will show that our protocol  $\Pi_{\text{RDA}}$  works (i.e. we can successfully query data that was previously stored), provided certain good event occur. For this, we will show a series of technical lemmas, roughly showing (in order) that under appropriate conditions

1. Joining the subnets works.
2. After calling  $\Pi_{\text{RDA}}.\text{STORE}$ , the data is stored by nodes in the appropriate column subnet.
3. Data stored in a column subnet is retained, even if parties leave and join.
4. Calling  $\Pi_{\text{RDA}}.\text{GET}$  will successfully retrieve data stored in the column.
5. We can get data that we stored (i.e. the property we want) by combining the above.

**Joining the subnets works.** Let us start by analyzing the joining of row and column subnets.

**Lemma 1** (Joining the rows and columns). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes as in [Definition 8](#). Consider a run of our protocol  $\Pi_{\text{RDA}}$ . Assume  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T]$  and  $\text{SubnProtGoodUntil}[T + 1]$  hold for some lifetime  $T$ . Let  $\mathcal{P}$  be an honest party joining (via  $\text{JOIN}$  or  $\text{INIT}$ ) our protocol  $\Pi_{\text{RDA}}$  at time  $\tau_{\text{join}} \geq 0$  with  $\text{role} \in \{0, 1\}$ . Let  $(r, c) = \text{Cell}(\mathcal{P})$ . Let  $\tau$  be some time where  $\mathcal{P}$  is active. Then the following hold if  $\tau_{\text{join}} > 0$  (i.e.  $\mathcal{P}$  is not one of the initial parties):*

- The event  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau]$  occurs if and only if  $\tau \geq \tau_{\text{join}} + \Delta_{\text{sub}}$ .
- If  $\text{role} = 1$ , then for all row subnets  $\text{sid}_{r'}$ , we have  $\text{IsInSubnet}_{\text{sid}_{r'}}[\mathcal{P}, \tau]$  if and only if  $\tau \geq \tau_{\text{join}} + \Delta_{\text{sub}}$ .
- If  $\tau_{\text{join}} \leq T$ , then  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$  occurs if and only if  $\tau \geq \tau_{\text{join}} + \Delta_{\text{sub}} + 2$ .

In case  $\tau_{\text{join}} = 0$  (i.e.  $\mathcal{P}$  is an initial party), we can strengthen this to

- $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$  and  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau]$  hold unconditionally.
- If  $\text{role} = 1$ , then  $\text{IsInSubnet}_{\text{sid}_{r'}}[\mathcal{P}, \tau]$  holds for all row subnets  $\text{sid}_{r'}$ .

Before we prove [Lemma 1](#), we state three corollaries: fully joined nodes are in their appropriate subnets, ColumnGoodUntil guarantees honest parties with IsInSubnet (rather than just active), and GoodCell means we can find honest parties in their subnets (rather than just active) via GetPeersInCell.

**Corollary 1** (FullyJoined and IsInSubnet). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Consider a run of our protocol  $\Pi_{\text{RDA}}$ . Assume  $\text{SubnProtGoodUntil}[T + 1]$  and  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T]$  hold for some lifetime  $T$ . Let  $\mathcal{P}$  be an honest party with  $\text{Cell}(\mathcal{P}) = (r, c)$ . Let  $0 \leq \tau \leq T$  be any time slot. Then we have the following:*

- $\text{FullyJoined}[\mathcal{P}, \tau] \implies \text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau]$ .
- $\text{FullyJoined}[\mathcal{P}, \tau] \iff \text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$ .

*Proof of Corollary 1.* Let  $\tau_{\text{join}}$  be the time when  $\mathcal{P}$  joins. If  $\tau < \tau_{\text{join}}$ , the statements hold true, since none of the events can occur. So let  $\tau \geq \tau_{\text{join}}$ . In this case,  $\tau \leq T$  implies  $\tau_{\text{join}} \leq T$ . Now, the corollary follows from [Lemma 1](#) and the fact that  $\Pi_{\text{RDA}}.\text{JOIN}$  takes exactly  $2 + \Delta_{\text{sub}}$  rounds and  $\Pi_{\text{RDA}}.\text{INIT}$  finishes the same round it starts.  $\square$

**Corollary 2** (Strengthened ColumnGoodUntil). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Consider a run of our protocol  $\Pi_{\text{RDA}}$ . Let  $\Delta \geq 2\Delta_{\text{sub}} + 2$  and consider some lifetime  $T \geq 0$ . Assume that  $\text{SubnProtGoodUntil}[T]$  and  $\text{ColumnGoodUntil}^\Delta[c, T]$  hold for some column  $c \in [k_2]$ .*

*Then for any  $0 \leq \tau \leq T$ , there exists an honest party  $\mathcal{P}$  with  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau \dots (\tau + \Delta')]$ , where  $\Delta' := \Delta - \Delta_{\text{sub}} - 2$ .*

*Proof of Corollary 2.* Since  $\text{ColumnGoodUntil}^\Delta[c, T]$  holds, there exists an honest initial party  $\mathcal{P}_0$  with  $\text{Col}(\mathcal{P}_0) = c$  and  $\text{Active}[\mathcal{P}_0, 0 \dots \Delta]$ . By [Lemma 1](#), we get  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}_0, 0 \dots \Delta]$ , which shows the claim for  $0 \leq \tau \leq \Delta_{\text{sub}} + 2$ .

So assume  $\Delta_{\text{sub}} + 2 < \tau \leq T$ . Again, since  $\text{ColumnGoodUntil}^\Delta[c, T]$  holds, there exists some party  $\mathcal{P}$  with  $\text{Col}(\mathcal{P}) = c$  and  $\text{Active}[\mathcal{P}, (\tau - \Delta_{\text{sub}} - 2) \dots (\tau - \Delta_{\text{sub}} - 2 + \Delta)]$ , i.e.  $\text{Active}[\mathcal{P}, (\tau - \Delta_{\text{sub}} - 2) \dots (\tau + \Delta')]$ . Applying [Lemma 1](#) (with  $T$  set to  $T - \Delta_{\text{sub}}$ ) gives  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau \dots (\tau + \Delta')]$ .  $\square$

**Corollary 3** (Strengthened GoodCell). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes and consider a run of our protocol  $\Pi_{\text{RDA}}$ . Let  $T$  be some lifetime bound,  $r \in [k_1]$  a row,  $c \in [k_2]$  a column and  $\Delta \geq 2\Delta_{\text{sub}} + 2$ . Assume  $\text{SubnProtGoodUntil}[T]$  and  $\text{ColumnGoodUntil}^\Delta[c, T]$  hold. Then the following hold for  $\tau_1 \leq \tau_2$  with  $\tau_1 \leq T$ .*

1. *If  $\text{GoodCell}[r, c, \tau_1 \dots \tau_2]$ , then there exists an honest  $\mathcal{P}$  with  $\text{Cell}(\mathcal{P}) = (r, c)$  and  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau_1 \dots \tau_2]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau_1 \dots \tau_2]$ .*
2. *If  $\text{GoodCell}[r, c, \tau_1 \dots \tau_2]$  and some honest party  $\mathcal{P}'$  with  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}', \tau]$  calls  $\mathbf{P} := \text{GetPeersInCell}(r, c)$  at some time  $\tau_1 \leq \tau \leq \tau_2$  with  $\tau \leq T$ , then there exists a  $\mathcal{P} \in \mathbf{P}$  with  $\mathcal{P}$  as in Item 1.*

*Proof of Corollary 3.* By definition,  $\text{GoodCell}[r, c, \tau_1 \dots \tau_2]$  means that there exists an honest party  $\mathcal{P}$  with  $\text{Cell}(\mathcal{P}) = (r, c)$  and  $\text{Active}[\mathcal{P}, \max\{0, \tau_1 - \Delta_{\text{sub}} - 2\} \dots \tau_2]$ . This means that  $\mathcal{P}$  must have joined at time  $\tau_{\text{join}} \leq \max\{0, \tau_1 - \Delta_{\text{sub}} - 2\}$ . If  $\tau_1 < \Delta_{\text{sub}} + 2$ , then  $\mathcal{P}$  is an initial party and we get  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, 0 \dots \tau_2]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, 0 \dots \tau_2]$  by [Lemma 1](#). Hence,  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau_1 \dots \tau_2]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau_1 \dots \tau_2]$ . Otherwise, we must have  $\tau_1 > \Delta_{\text{sub}} + 2$ , so  $\tau_{\text{join}} \leq \tau_1 - \Delta_{\text{sub}} - 2$ . This implies  $\tau_{\text{join}} + 1 \leq T$  and we can again use [Lemma 1](#) (with  $T$  set to  $T - 1$ ) to deduce that  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau_1 \dots \tau_2]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau_1 \dots \tau_2]$ .  $\square$

Item 2 is a direct consequence of the definition of GetPeersInCell and SubnProtGoodUntil.  $\square$

Let us now prove [Lemma 1](#):

*Proof.* For the “only if” part of the statements, we start by observing that the only places where we call  $\Pi_{\text{SD}}.\text{JOINSUBNET}$  resp.  $\Pi_{\text{SD}}.\text{CREATESUBNET}$  are in  $\Pi_{\text{RDA}}.\text{JOIN}$  resp.  $\Pi_{\text{RDA}}.\text{INIT}$ . For  $\tau_{\text{join}} = 0$ , the “only if”-statement is trivial. For  $\tau_{\text{join}} > 0$ , observe that  $\mathcal{P}$  calls  $\Pi_{\text{SD}}.\text{JOINSUBNET}$  for row subnets at time  $\tau_{\text{join}}$  and for column subnets at time  $\tau_{\text{join}} + 2$ . The “only if”-claim then follows from the definition of IsInSubnet.

The proof for the “if” part works by induction over  $\tau_{\text{join}} \geq 0$ .

Let us start with the  $\tau_{\text{join}} = 0$  case, so  $\mathcal{P}$  is among the initial parties  $\{\mathcal{P}_1, \dots, \mathcal{P}_\ell\}$  with roles  $\text{role}_1, \dots, \text{role}_\ell$ . Let  $\mathbf{P}_{r'}^{\text{row}} := \{\mathcal{P}_i \mid \text{Row}(\mathcal{P}_i) = r' \vee \text{role}_i = 1\}$  be as in  $\text{INIT}$  for any  $r' \in [k_1]$ . Also, let  $\mathbf{P}_{c'}^{\text{col}} := \{\mathcal{P}_i \mid \text{Col}(\mathcal{P}_i) = c'\}$  for any  $c' \in [k_2]$ . In our protocol, the only calls to  $\Pi_{\text{SD}}.\text{CREATESUBNET}$  made by honest parties are at time  $\tau_{\text{join}} = 0$  and made by exactly the parties from  $\mathbf{P}_{r'}^{\text{row}}$  resp.  $\mathbf{P}_{c'}^{\text{col}}$  with argument  $\mathbf{P}_{r'}^{\text{row}}$  resp.  $\mathbf{P}_{c'}^{\text{col}}$ . Since  $\mathcal{P} \in \mathbf{P}_{r'}^{\text{row}}$ , this implies  $\text{CreatedSubnet}[\text{sid}_r, \mathbf{P}_{r'}^{\text{row}}]$  and  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, 0]$ . Similarly,  $\mathcal{P} \in \mathbf{P}_{c'}^{\text{col}}$ , so we get  $\text{CreatedSubnet}[\text{sid}_c, \mathbf{P}_{c'}^{\text{col}}]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, 0]$ . This implies  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau]$  resp.  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$  for any  $\tau \geq 0$  for which party  $\mathcal{P}$  is still active. If furthermore  $\text{role} = 1$ , then  $\mathcal{P} \in \mathbf{P}_{r'}^{\text{row}}$  for all  $r'$ , so we get  $\text{IsInSubnet}_{\text{sid}_{r'}}[\mathcal{P}, \tau]$  for all  $r' \in [k_1]$  and all  $\tau \geq 0$  for which party  $\mathcal{P}$  is still active.

Let us proceed with the induction step, where we assume that  $\tau_{\text{join}} > 0$  and all statements of the lemma are proven for all  $\tau_0 < \tau_{\text{join}}$ . Since  $\tau_{\text{join}} > 0$ , the party  $\mathcal{P}$  joins by  $\text{JOIN}$  via some bootstrap nodes  $\mathcal{P}_1, \dots, \mathcal{P}_t$ . Since  $S$  respects bootstrap nodes and uses good bootstrap nodes, there exists a prospective bootstrap node  $\mathcal{P}'$  among them that is active at least from time  $\max\{\tau_{\text{join}} - \Delta_{\text{sub}}, 0\}$  to  $\tau_{\text{join}} + \Delta_{\text{sub}}$ . We first claim that  $\text{IsInSubnet}_{\text{sid}_{r'}}[\mathcal{P}', \tau_{\text{join}} \dots (\tau_{\text{join}} + \Delta_{\text{sub}})]$  holds for all row indices  $r \in [k_1]$ : If  $\tau_{\text{join}} \geq \Delta_{\text{sub}}$ , then  $\mathcal{P}'$  must have joined at  $\tau_{\text{join}} - \Delta_{\text{sub}}$  or earlier. Our first claim is thus implied by the induction hypothesis. Otherwise,  $\tau_{\text{join}} < \Delta_{\text{sub}}$  and  $\mathcal{P}'$  is active from time 0, so it must be one of the initial parties. Using the induction hypothesis on  $\mathcal{P}'$  with  $\tau_{\text{join}} = 0$  then also shows our first claim about  $\mathcal{P}'$ .

We now continue with the induction step for the row subnets. If  $\text{role} = 0$ , then party  $\mathcal{P}$  will call  $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_r, \mathcal{P}')$  at time  $\tau_{\text{join}}$  as part of its  $\text{JOIN}$  call. By definition of  $\text{IsInSubnet}$ , this implies  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau]$  for any  $\tau \geq \tau_{\text{join}} + \Delta_{\text{sub}}$  for which  $\mathcal{P}$  is still active. Similarly, if  $\text{role} = 1$ , then party  $\mathcal{P}$  calls  $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_{r'}, \mathcal{P}_i)$  at time  $\tau_{\text{join}}$  for all rows  $r' \in [k_1]$  as part of its  $\text{JOIN}$  call. This implies  $\text{IsInSubnet}_{\text{sid}_{r'}}[\mathcal{P}, \tau]$  for any  $\tau \geq \tau_{\text{join}} + \Delta_{\text{sub}}$  for which  $\mathcal{P}$  is still active, finishing the part of the lemma with the row subnets.

Finally, we prove the induction step for the column subnets. Set  $\tilde{\tau} := \max\{0, \tau_{\text{join}} - \Delta_{\text{sub}}\}$ . Note that since  $\tau_{\text{join}} > 0$ , we have  $\tilde{\tau} < \tau_{\text{join}}$ . Because  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T]$  holds with  $\tilde{\tau} \leq T$ , there exists some honest party  $\tilde{\mathcal{P}}$  with  $\text{Active}[\tilde{\mathcal{P}}, \tilde{\tau} \dots (\tilde{\tau} + 2\Delta_{\text{sub}} + 2)]$  and  $\text{Col}(\tilde{\mathcal{P}}) = c$ . By definition of  $\tilde{\tau}$ , we have

$$\tilde{\tau} + 2\Delta_{\text{sub}} + 2 \geq \tau_{\text{join}} - \Delta_{\text{sub}} + 2\Delta_{\text{sub}} + 2 = \tau_{\text{join}} + \Delta_{\text{sub}} + 2.$$

So  $\tilde{\mathcal{P}}$  joined at  $\tilde{\tau}$  or earlier and stays active until at least  $\tau_{\text{join}} + \Delta_{\text{sub}} + 2$ .

Let  $\tilde{r} := \text{Row}(\tilde{\mathcal{P}})$  be the row assigned to  $\tilde{\mathcal{P}}$ . In case  $\tilde{\tau} > 0$ , using the induction hypothesis for the rows, applied to  $\tilde{\tau} < \tau$ , gives us  $\text{IsInSubnet}_{\text{sid}_{\tilde{r}}}[\tilde{\mathcal{P}}, (\tilde{\tau} + \Delta_{\text{sub}}) \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)]$ . Using the induction hypothesis for the columns gives  $\text{IsInSubnet}_{\text{sid}_c}[\tilde{\mathcal{P}}, (\tilde{\tau} + \Delta_{\text{sub}} + 2) \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)]$ . We can rewrite those using the definition of  $\tilde{\tau}$ , for  $\tilde{\tau} > 0$  as

$$\text{IsInSubnet}_{\text{sid}_{\tilde{r}}}[\tilde{\mathcal{P}}, \tau_{\text{join}} \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)] \text{ and } \text{IsInSubnet}_{\text{sid}_c}[\tilde{\mathcal{P}}, (\tau_{\text{join}} + 2) \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)] .$$

If  $\tilde{\tau} = 0$ , we know that  $\tilde{\mathcal{P}}$  is one of the initial parties and the exact same statements  $\text{IsInSubnet}_{\text{sid}_{\tilde{r}}}[\tilde{\mathcal{P}}, \tau_{\text{join}} \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\tilde{\mathcal{P}}, (\tau_{\text{join}} + 2) \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)]$  hold by induction hypothesis applied to  $\tilde{\mathcal{P}}$  and  $\tilde{\tau} = 0$ .

Now, during the  $\text{JOIN}$ -call of  $\mathcal{P}$ , the party  $\mathcal{P}$  sends a  $\text{join}$ -message to  $\mathcal{P}'$  at time  $\tau_{\text{join}}$ . We already established  $\text{IsInSubnet}_{\text{sid}_{\tilde{r}}}[\mathcal{P}', \tau_{\text{join}} \dots (\tau_{\text{join}} + \Delta_{\text{sub}})]$ , so  $\mathcal{P}'$  is still active at time  $\tau_{\text{join}} + 1$  and will receive that message. Then  $\mathcal{P}'$  will call (among others)  $\Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_{\tilde{r}})$  at time  $\tau_{\text{join}} + 1$ . Since  $\text{SubnProtGoodUntil}[T + 1]$ ,  $\tau_{\text{join}} + 1 \leq T + 1$ , and  $\text{IsInSubnet}_{\text{sid}_{\tilde{r}}}[\tilde{\mathcal{P}}, \tau_{\text{join}} + 1]$  hold, this call will return  $\tilde{\mathcal{P}}$  (among others) and be included in the local computation of  $\mathbf{P}_c^{\text{col}}$  by  $\mathcal{P}'$ . Then  $\mathcal{P}'$  will send back a  $(\text{join\_rsp}, \mathbf{P}_c^{\text{col}})$ -message to  $\mathcal{P}$  with  $\tilde{\mathcal{P}} \in \mathbf{P}_c^{\text{col}}$ . This message is received by  $\mathcal{P}$  at time  $\tau_{\text{join}} + 2$ , so  $\mathcal{P}$  will call  $\Pi_{\text{SD}}.\text{JOINSUBNET}(\text{sid}_c, \tilde{\mathcal{P}})$  at time  $\tau_{\text{join}} + 2$ . Since  $\text{IsInSubnet}_{\text{sid}_c}[\tilde{\mathcal{P}}, (\tau_{\text{join}} + 2) \dots (\tau_{\text{join}} + \Delta_{\text{sub}} + 2)]$  holds, we obtain  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$  for any  $\tau \geq \tau_{\text{join}} + 2 + \Delta_{\text{sub}}$  by definition of  $\text{IsInSubnet}$ , finishing the proof.  $\square$

**Storing data works.** We now proceed to show that after calling  $\Pi_{\text{RDA}}.\text{STORE}$ , data is stored by the column subnet. In fact, data will be stored by *all* honest parties in a given column subnet that have been active for a sufficiently long time. We introduce the following events:

**Definition 9** (Storage Events). For  $h \in \mathcal{H}$ ,  $i \in [m]$ ,  $x \in \Sigma$ , we define the following events:

- Event  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau]$ : This event occurs if party  $\mathcal{P}$  is honest, active at time  $\tau$ , and at the beginning of time slot  $\tau$ , we have  $\text{Symbols}[h, i] = x$  for  $\mathcal{P}$ 's local map  $\text{Symbols}$ .
- Event  $\text{WholeColumnStores}_{h,i,x}[c, \tau_0, \tau_1]$  for times  $\tau_0 \leq \tau_1$  and column  $c$ : This event occurs if for every honest party  $\mathcal{P}$  with  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau_0 \dots \tau_1]$ , we have  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_1]$ .
- Event  $\text{StoredInColumn}_{h,i,x}[c, \tau_0 \dots \tau_1]$ : This event occurs for a column  $c$  and times  $\tau_0 \leq \tau_1$  if there exists an honest party  $\mathcal{P}$  with  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_0]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, (\tau_0 - 1) \dots \tau_1]$ .

We remark that if we write a symbol during time slot  $\tau$ , the corresponding event  $\text{InSymbolStorage}$  only holds starting from time  $\tau + 1$ . The meaning of  $\text{InSymbolStorage}$  is that we can *reliably query* the symbol, which is not guaranteed in the same round that we write the symbol, absent intra-slot ordering constraints on event processing. This also means that in the definition of  $\text{StoredInColumn}_{h,i,x}[c, \tau_0 \dots \tau_1]$ , the symbol was stored by  $\mathcal{P}$  in round  $\tau_0 - 1$  or earlier, which guarantees that  $\tau_0 - 1 \geq 0$ .

We start by observing that honest parties will not overwrite data if  $\text{Pred}$  is position-binding:

**Lemma 2.** *Let  $\mathcal{P}$  be an honest party,  $\tau_1 \leq \tau_2$  some time slots,  $h \in \mathcal{H}$ ,  $x \in \Sigma$  and  $i \in [m]$ . Assume that  $\text{Pred}$  is position-binding and consider a run of  $\Pi_{\text{RDA}}$ . Then*

$$\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_1] \wedge \text{Active}[\mathcal{P}, \tau_2] \implies \text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_2] .$$

Furthermore, if  $\mathcal{P}$  stores  $\text{Symbols}[h, i] := x$  at time  $\tau$  and is active at time  $\tau + 1$ , then  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau + 1]$  holds.

*Proof.* In our protocol, each honest party initializes  $\text{Symbols}$  only once as part of  $\text{INIT}$  resp.  $\text{JOIN}$  before any writes are made to it. After that, every write  $\text{Symbols}[h, i] := x$  is guarded by a check that  $\text{Pred}(h, i, x) = 1$ . This implies that no value stored to  $\text{Symbols}$  is ever overwritten by a different value (otherwise, this would directly violate position-binding of  $\text{Pred}$ ). The claims follow.  $\square$

With this, we can state some easy properties of the events defined in [Definition 9](#)

**Proposition 3.** *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Let  $h \in \mathcal{H}$ ,  $i \in [m]$ ,  $x \in \Sigma$ . Assume that  $\text{Pred}$  is position-binding and consider a run of  $\Pi_{\text{RDA}}$ . Then we have*

- (a)  $\text{WholeColumnStores}_{h,i,x}[c, \tau_0, \tau_1] \implies \text{WholeColumnStores}_{h,i,x}[c, \tau_0, \tau_2]$  for all  $\tau_0 \leq \tau_1 \leq \tau_2$ .
- (b) Assume  $\text{ColumnGoodUntil}^\Delta[c, T]$  for some lifetime  $T$  and  $\Delta \geq 2\Delta_{\text{sub}} + 2$ . Set  $\Delta' := \Delta - \Delta_{\text{sub}} - 2$ . For  $0 < \tau_0 \leq \tau_1$ , we have

$$\left( \begin{array}{l} \tau_0 \leq T + 1 \\ \wedge \quad \tau_0 + \Delta' - 1 \geq \tau_1 \\ \wedge \quad \text{WholeColumnStores}_{h,i,x}[c, \tau_0, \tau_1] \end{array} \right) \implies \text{StoredInColumn}_{h,i,x}[c, \tau_1 \dots (\tau_0 + \Delta' - 1)] .$$

*Proof.* Item (a) is a direct consequence of [Lemma 2](#) and the definition of  $\text{WholeColumnStores}$ .

For Item (b),  $\text{ColumnGoodUntil}^\Delta[c, T]$  implies by [Corollary 2](#) that there exists some honest  $\mathcal{P}$  with  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, (\tau_0 - 1) \dots (\tau_0 - 1 + \Delta')]$ .

Since  $\tau_0 - 1 < \tau_1 \leq \tau_0 - 1 + \Delta'$ , we have  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau_1]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, (\tau_1 - 1) \dots (\tau_0 + \Delta' - 1)]$ . By definition,  $\text{WholeColumnStores}_{h,i,x}[c, \tau_0, \tau_1]$  then implies  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_1]$ , which shows  $\text{StoredInColumn}_{h,i,x}[c, \tau_1 \dots (\tau_0 + \Delta' - 1)]$ .  $\square$

**Lemma 3 (Store Works).** *Let  $S$  be an join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Assume  $\text{Pred}$  is position-binding and consider a run of our protocol  $\Pi_{\text{RDA}}$ . Let  $\mathcal{P}_{\text{store}}$  be an honest party with  $\text{Row}(\mathcal{P}_{\text{store}}) = r$ . Let  $x \in \Sigma$ ,  $i \in [m]$ ,  $h \in \mathcal{H}$  for which  $\text{Pred}(h, i, x)$  holds and let  $c := \text{GetColForSymbol}(i)$ . Assume  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T]$  and  $\text{SubnProtGoodUntil}[T + 1]$  hold for some lifetime  $T$ . Then we have for all  $0 \leq \tau \leq T$*

$$\left( \begin{array}{l} \text{GoodCell}[r, c, \tau \dots (\tau + 1)] \\ \wedge \quad \text{FullyJoined}[\mathcal{P}_{\text{store}}, \tau] \\ \wedge \quad \text{CalledStore}_{h,i,x}[\mathcal{P}_{\text{store}}, \tau] \end{array} \right) \implies \text{WholeColumnStores}_{h,i,x}[c, (\tau + 1), (\tau + 3)] .$$

*Proof.* Let  $\mathcal{P}$  be an honest party with  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, (\tau + 1) \dots (\tau + 3)]$ . We need to show that  $\mathcal{P}$  stores the data  $(h, i, x)$  at the beginning of time slot  $\tau + 3$ .

Since  $\text{FullyJoined}[\mathcal{P}_{\text{store}}, \tau]$  holds, we have  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}_{\text{store}}, \tau]$  by [Corollary 1](#). Since  $\text{GoodCell}[r, c, \tau \dots (\tau + 1)]$  holds, there exist an honest party  $\mathcal{P}'$  with  $\text{Cell}(\mathcal{P}') = (r, c)$  with  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}', \tau \dots (\tau + 1)]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}', \tau \dots (\tau + 1)]$  by [Corollary 3](#).

By assumption,  $\mathcal{P}_{\text{store}}$  calls  $\Pi_{\text{RDA}}.\text{STORE}(h, i, x)$  at time  $\tau$ . During this call,  $\mathcal{P}_{\text{store}}$  internally calls  $\mathbf{P} := \text{GetPeersInCell}(r, c)$  at time  $\tau$ . Since  $\text{SubnProtGoodUntil}[T]$  holds, this finds  $\mathcal{P}' \in \mathbf{P}$  by [Corollary 3](#) and so  $\mathcal{P}_{\text{store}}$  sends  $(\text{store}, h, i, x)$  to  $\mathcal{P}'$  at time  $\tau$ .

This peer  $\mathcal{P}'$  receives  $(\text{store}, h, i, x)$  at time  $\tau + 1$  and is still active. Party  $\mathcal{P}'$  calls  $\mathbf{P}' := \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_c)$  at time  $\tau + 1$ . Again,  $\text{SubnProtGoodUntil}[T + 1]$  holds, which implies that  $\mathcal{P} \in \mathbf{P}'$ . So party  $\mathcal{P}'$  sends  $(\text{store\_fwd}, h, i, x)$  to  $\mathcal{P}$ . The party  $\mathcal{P}$  is still active at time  $\tau + 2$  by definition, so it will receive  $(\text{store\_fwd}, h, i, x)$  at time  $\tau + 2$ . It reacts by storing the symbol at time  $\tau + 2$ , so we get  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau + 3]$  by [Lemma 2](#), which finishes the proof.  $\square$

**Retaining Data.** We next show that data that was stored in a column is retained.

**Lemma 4** (Retaining Data). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Consider a run of our protocol  $\Pi_{\text{RDA}}$ . Let  $x \in \Sigma, i \in [m], h \in \mathcal{H}$  for which  $\text{Pred}(h, i, x)$  holds and let  $c := \text{GetColForSymbol}(i)$ . Assume that  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T + 1]$  and  $\text{SubnProtGoodUntil}[T + 2]$  hold for some lifetime  $T$ . Then we have for  $0 \leq \tau \leq T$*

$$\left( \begin{array}{l} \text{WholeColumnStores}_{h,i,x}[c, \tau, \tau + \Delta_{\text{sync}} + 1] \\ \wedge \text{StoredInColumn}_{h,i,x}[c, (\tau + 2) \dots (\tau + \Delta_{\text{sync}})] \end{array} \right) \implies \text{WholeColumnStores}_{h,i,x}[c, \tau + 1, \tau + \Delta_{\text{sync}} + 2]$$

*Proof.* Let  $\mathcal{P}$  be some honest party with  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, (\tau + 1) \dots (\tau + 2 + \Delta_{\text{sync}})]$ . Since honest parties only join their own column, this implies  $\text{Col}(\mathcal{P}) = c$ . We need to show  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau + 2 + \Delta_{\text{sync}}]$ . In the case that we have  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$ , we get  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau + \Delta_{\text{sync}} + 1]$  by assumption and the claim follows from [Lemma 2](#).

So we may assume that  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau]$  does *not* hold. By [Corollary 1](#), this means that  $\text{FullyJoined}[\mathcal{P}, \tau]$  does not hold, whereas  $\text{FullyJoined}[\mathcal{P}, \tau + 1]$  does hold. In particular,  $\mathcal{P}$  cannot be one of the initial parties and since  $\Pi_{\text{RDA}}.\text{JOIN}$  takes exactly  $2 + \Delta_{\text{sub}}$  time slots,  $\mathcal{P}$  must have joined via  $\Pi_{\text{RDA}}.\text{JOIN}$  exactly at time  $\tau - \Delta_{\text{sub}} - 1 > 0$ . Looking at the definition of  $\text{JOIN}$ , we know that in time slot  $\tau + 1$ , the party  $\mathcal{P}$  calls  $\mathbf{P}_c^{\text{col}} := \Pi_{\text{SD}}.\text{GETPEERS}(\text{sid}_c)$ .

Because  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 2) \dots (\tau + \Delta_{\text{sync}})]$  holds, there exists some honest party  $\mathcal{P}'$  with  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}', (\tau + 1) \dots (\tau + \Delta_{\text{sync}})]$  and  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}', \tau + 2]$ . Since  $\text{SubnProtGoodUntil}[T + 2]$ , we deduce that  $\mathcal{P}' \in \mathbf{P}_c^{\text{col}}$ . Looking further at the definition of  $\text{JOIN}$ ,  $\mathcal{P}$  will send  $\text{sync}$  to  $\mathcal{P}'$  in round  $\tau + 1$ , to be received in round  $\tau + 2$ . We know that  $\mathcal{P}'$  is still active at time  $\tau + 2$ . Our protocol specifies that  $\mathcal{P}'$  should then send back some  $(\text{sync\_rsp}, \mathbf{S})$  response to  $\mathcal{P}$  at time  $\tau + 2 + (\Delta_{\text{sync}} - 2)$  or earlier. Since  $\mathcal{P}'$  is still active at time  $\tau + \Delta_{\text{sync}}$ , it will actually send this message. Since  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}', \tau + 2]$ , we have  $(h, i, x) \in \mathbf{S}$ . The  $(\text{sync\_rsp}, \mathbf{S})$  response is received by  $\mathcal{P}$  at time  $\tau + \Delta_{\text{sync}} + 1$  or earlier, upon which  $\mathcal{P}$  stores the symbol at time  $\tau + \Delta_{\text{sync}} + 1$  or earlier. Consequently,  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau + \Delta_{\text{sync}} + 2]$  holds by [Lemma 2](#), finishing the proof.  $\square$

We now combine [Lemma 3](#), [Lemma 4](#) and [Proposition 3](#).

**Lemma 5** (Store works long-term). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes and assume  $\text{Pred}$  is position-binding. Let  $T$  be some lifetime bound and consider a run of our protocol  $\Pi_{\text{RDA}}$ . Let  $x \in \Sigma, i \in [m], h \in \mathcal{H}$  for which  $\text{Pred}(h, i, x)$  holds and let  $c := \text{GetColForSymbol}(i)$ . Assume some honest party  $\mathcal{P}_{\text{store}}$  with  $\text{Row}(\mathcal{P}_{\text{store}}) = r$  stored the symbol at some time  $\tau_{\text{store}} \leq T - 2$ , i.e. assume  $\text{FullyJoined}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}]$  and  $\text{CalledStore}_{h,i,x}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}]$  hold. Further, assume that the following good events hold:*

- $\text{GoodCell}[r, c, \tau_{\text{store}} \dots (\tau_{\text{store}} + 1)]$
- $\text{SubnProtGoodUntil}[T + 1]$
- $\text{ColumnGoodUntil}^{\Delta}[c, T + 1]$  with  $\Delta \geq \max\{2\Delta_{\text{sub}} + 2, 2\Delta_{\text{sync}} + \Delta_{\text{sub}} + 2\}$

Then we have  $\text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{store}} + 1, \tau_{\text{store}} + 3]$  and we have

$$\text{WholeColumnStores}_{h,i,x}[c, \tau, (\tau + \Delta_{\text{sync}} + 1)]$$

for all  $\tau_{\text{store}} + 1 \leq \tau \leq T$ .

*Proof.* First, we get  $\text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{store}} + 1, \tau_{\text{store}} + 3]$  by Lemma 3, noting that the conditions of this lemma are satisfied. We now show the main claim by induction over  $\tau$ :

We start the induction at  $\tau = \tau_{\text{store}} + 1$ . Here, the claim follows from  $\text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{store}} + 1, \tau_{\text{store}} + 3]$  and Proposition 3(a).

For the induction step with  $\tau \geq \tau_{\text{store}} + 2$ , we assume that  $\text{WholeColumnStores}_{h,i,x}[c, \tau', (\tau' + \Delta_{\text{sync}} + 1)]$  holds for all  $\tau_{\text{store}} + 1 \leq \tau' < \tau$ . Our first claim is that  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 1) \dots (\tau + \Delta_{\text{sync}} - 1)]$  holds, for which we need to make a case distinction, depending on whether  $\tau \leq \tau_{\text{store}} + \Delta_{\text{sync}}$  or not.

In the case  $\tau \leq \tau_{\text{store}} + \Delta_{\text{sync}}$ , we use  $\text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{store}} + 1, \tau_{\text{store}} + 3]$ . By Proposition 3(a), this implies  $\text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{store}} + 1, \tau + 1]$ , because  $\tau \geq \tau_{\text{store}} + 2$ . We want to use Proposition 3(b) (with  $\tau_0 = \tau_{\text{store}} + 1, \tau_1 = \tau + 1$ ). We know that  $\text{ColumnGoodUntil}^\Delta[c, T]$  by assumption with  $\Delta \geq 2\Delta_{\text{sub}} + 2$ . Set  $\Delta' := \Delta - \Delta_{\text{sub}} - 2$ , so  $\Delta' \geq 2\Delta_{\text{sync}}$ . Observe that  $\tau_{\text{store}} + 1 \leq T + 1$  and

$$\tau_{\text{store}} + 1 + \Delta' - 1 = \tau_{\text{store}} + \Delta' \geq \tau_{\text{store}} + 2\Delta_{\text{sync}} \geq \tau + \Delta_{\text{sync}} \geq \tau + 1.$$

This means the conditions of Proposition 3(b) are satisfied and we obtain  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 1) \dots (\tau_{\text{store}} + \Delta')]$ . We showed above that  $\tau_{\text{store}} + \Delta' \geq \tau + \Delta_{\text{sync}}$ , so we can weaken this to  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 1) \dots (\tau + \Delta_{\text{sync}} - 1)]$ , as required.

So consider the other case  $\tau \geq \tau_{\text{store}} + \Delta_{\text{sync}} + 1$ . So  $\tau - \Delta_{\text{sync}} \geq \tau_{\text{store}} + 1$ . By induction hypothesis (with  $\tau' = \tau - \Delta_{\text{sync}}$ ), we know that  $\text{WholeColumnStores}_{h,i,x}[c, \tau - \Delta_{\text{sync}}, \tau + 1]$  holds. Again, we want to use Proposition 3(b) (with  $\tau_0 = \tau - \Delta_{\text{sync}}, \tau_1 = \tau + 1$ ). We have  $\tau - \Delta_{\text{sync}} \leq T + 1$  and  $\tau - \Delta_{\text{sync}} + \Delta' - 1 \geq \tau + \Delta_{\text{sync}} - 1 \geq \tau + 1$ , so the conditions are satisfied and we obtain  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 1) \dots (\tau - \Delta_{\text{sync}} + \Delta' - 1)]$ . Since  $\Delta' \geq 2\Delta_{\text{sync}}$ , we can weaken this to  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 1) \dots (\tau + \Delta_{\text{sync}} - 1)]$ . So we showed this first claim in both cases.

By induction hypothesis (with  $\tau' = \tau - 1$ ), we know that  $\text{WholeColumnStores}_{h,i,x}[c, \tau - 1, \tau + \Delta_{\text{sync}}]$  holds. Also, we know  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T + 1]$ ,  $\text{SubnProtGoodUntil}[T + 1]$  by assumption.

Combining this with  $\text{StoredInColumn}_{h,i,x}[c, (\tau + 1) \dots (\tau + \Delta_{\text{sync}} - 1)]$  allows us to use Lemma 4 (with  $\tau = \tau - 1, T = T - 1$ ) to finally obtain  $\text{WholeColumnStores}_{h,i,x}[c, \tau, \tau + \Delta_{\text{sync}} + 1]$ , finishing the proof.  $\square$

**Retrieving data works.** We finally show that  $\Pi_{\text{RDA}}.\text{GET}$  allows us to retrieve stored data.

**Lemma 6** (Get works). *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Assume  $\text{Pred}$  is position-binding and consider a run of  $\Pi_{\text{RDA}}$ . Let  $x \in \Sigma, i \in [m], h \in \mathcal{H}$  for which  $\text{Pred}(h, i, x)$  holds and let  $c := \text{GetColForSymbol}(i)$ . Assume that  $\text{SubnProtGoodUntil}[T]$  and  $\text{ColumnGoodUntil}^{2\Delta_{\text{sub}}+2}[c, T]$  hold for some lifetime  $T$ . Let  $\mathcal{P}_{\text{get}}$  be some fully joined honest party in row  $r = \text{Row}(\mathcal{P}_{\text{get}})$  that tries to retrieve the symbol at time  $0 \leq \tau_{\text{get}} \leq T$ , i.e.,  $\text{FullyJoined}[\mathcal{P}_{\text{get}}, \tau_{\text{get}} \dots (\tau_{\text{get}} + 2)]$  and  $\text{CalledGet}_{h,i}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$  hold. Then if any one of the three options*

- $\tau_{\text{get}} \geq \Delta_{\text{sync}}, \text{GoodCell}[r, c, (\tau_{\text{get}} - \Delta_{\text{sync}}) \dots (\tau_{\text{get}} + 1)]$  and  $\text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{get}} - \Delta_{\text{sync}}, \tau_{\text{get}} + 1]$ , or
- $\tau_{\text{get}} \geq \Delta_{\text{sync}}, \text{GoodCell}[r, c, (\tau_{\text{get}} - \Delta_{\text{sync}}) \dots (\tau_{\text{get}} + 1)]$  and  $\text{WholeColumnStores}_{h,i,x}[c, \tau', \tau' + 2]$  for some  $\tau_{\text{get}} - \Delta_{\text{sync}} \leq \tau' \leq \tau_{\text{get}} - 1$ , or
- $\tau_{\text{get}} < \Delta_{\text{sync}}, \text{GoodCell}[r, c, 0 \dots (\tau_{\text{get}} + 1)]$  and  $\text{WholeColumnStores}_{h,i,x}[c, \tau', \tau' + 2]$  for some  $0 \leq \tau' \leq \tau_{\text{get}} - 1$

holds, we get

$$\text{GotResult}_{h,i,x}^{\Delta_{\text{get}}}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$$

with  $\Delta_{\text{get}} = 2$ .



*Proof.* By [Corollary 3](#), there exists some honest party  $\mathcal{P}$  with  $\text{Cell}(\mathcal{P}) = (r, c)$ ,  $\text{IsInSubnet}_{\text{sid}_r}[\mathcal{P}, \tau_{\text{get}}]$  and  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \tau_{\text{get}}]$  that stays active until (at least)  $\tau_{\text{get}} + 1$ . Further,  $\text{IsInSubnet}_{\text{sid}_c}[\mathcal{P}, \max\{0, \tau_{\text{get}} - \Delta_{\text{sync}}\}]$ . In either of the three cases, we have  $\text{WholeColumnStores}_{h,i,x}[c, \tau_0, \tau_1]$  with  $\max\{0, \tau_{\text{get}} - \Delta_{\text{sync}}\} \leq \tau_0 \leq \tau_1 \leq \tau_{\text{get}} + 1$  for some  $\tau_0, \tau_1$ . This implies  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_{\text{get}} + 1]$  by [Proposition 3\(a\)](#) and the definition of  $\text{WholeColumnStores}$ .

Now,  $\mathcal{P}_{\text{get}}$  calls  $\Pi_{\text{RDA}}.\text{GET}(h, i)$  at time  $\tau_{\text{get}}$ . As part of this call, it retrieves  $\mathbf{P} := \text{GetPeersInCell}(r, c)$ . Since  $\text{SubnProtGoodUntil}[T]$  holds with  $\tau_{\text{get}} \leq T$ , we have  $\mathcal{P} \in \mathbf{P}$ , again by [Corollary 3](#).

Then,  $\mathcal{P}_{\text{get}}$  sends  $(\text{get}, h, i)$  to  $\mathcal{P}$ , which is retrieved by  $\mathcal{P}$  at time  $\tau_{\text{get}} + 1$ , where we are guaranteed that  $\mathcal{P}$  is still active. Since  $\text{InSymbolStorage}_{h,i,x}[\mathcal{P}, \tau_{\text{get}} + 1]$  holds,  $\mathcal{P}$  will respond with  $(\text{get\_rsp}, h, i, x)$ . This response is then retrieved by  $\mathcal{P}_{\text{get}}$  at time  $\tau_{\text{get}} + 2$  (where it is still active by assumption), and  $\mathcal{P}_{\text{get}}$  outputs  $x$ , finishing the proof.  $\square$

**Retrieving Stored Data.** We now combine the above lemmas to show that we can retrieve data that we stored, i.e. we prove that the property we want for robustness holds, provided good events hold.

**Lemma 7.** *Let  $S$  be a join-leave schedule that respects bootstrap nodes and uses good bootstrap nodes. Assume  $\text{Pred}$  is position-binding and consider a run of  $\Pi_{\text{RDA}}$ . Let  $x \in \Sigma, i \in [m], h \in \mathcal{H}$  for which  $\text{Pred}(h, i, x)$  holds, let  $T$  be some lifetime and let  $c := \text{GetColForSymbol}(i)$ . Let  $\mathcal{P}_{\text{get}}$  and  $\mathcal{P}_{\text{store}}$  be honest parties with  $\text{CalledStore}_{h,i,x}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}]$ ,  $\text{FullyJoined}[\mathcal{P}_{\text{store}}, \tau_{\text{store}}]$ ,  $\text{CalledGet}_{h,i}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$  and  $\text{FullyJoined}[\mathcal{P}_{\text{get}}, \tau_{\text{get}} \dots (\tau_{\text{get}} + 2)]$ , where  $0 \leq \tau_{\text{store}}$  and  $\tau_{\text{store}} + 2 \leq \tau_{\text{get}} \leq T$ . Set  $r_{\text{store}} := \text{Row}(\mathcal{P}_{\text{store}})$  and  $r_{\text{get}} := \text{Row}(\mathcal{P}_{\text{get}})$ . Then, if all of*

- $\text{SubnProtGoodUntil}[T + 1]$  and
- $\text{ColumnGoodUntil}^\Delta[c, T + 1]$ , where  $\Delta \geq \max\{2\Delta_{\text{sub}} + 2, 2\Delta_{\text{sync}} + \Delta_{\text{sub}} + 2\}$  and
- $\text{GoodCell}[r_{\text{store}}, c, \tau_{\text{store}} \dots (\tau_{\text{store}} + 1)]$  and
- $\text{GoodCell}[r_{\text{get}}, c, \max\{0, \tau_{\text{get}} - \Delta_{\text{sync}}\} \dots (\tau_{\text{get}} + 1)]$

*hold, we have  $\text{GotResult}_{h,i,x}^{\Delta_{\text{get}}}[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$  with  $\Delta_{\text{get}} = 2$ .*

*Proof.* Observe that the conditions of [Lemma 5](#) are satisfied, so we have

$$\begin{aligned} & \text{WholeColumnStores}_{h,i,x}[c, \tau_{\text{store}} + 1, \tau_{\text{store}} + 3] \quad \text{and} \\ & \text{WholeColumnStores}_{h,i,x}[c, \tau, (\tau + \Delta_{\text{sync}} + 1)] \quad \text{for all } \tau_{\text{store}} + 1 \leq \tau \leq T. \end{aligned}$$

We now show that the conditions of [Lemma 6](#) are satisfied, for which we distinguish 3 cases that match the conditions of [Lemma 6](#).

Assume first that  $\tau_{\text{get}} < \Delta_{\text{sync}}$ . Then  $\text{WholeColumnStores}_{h,i,x}[c, \tau', \tau' + 2]$  holds for some  $0 \leq \tau' \leq \tau_{\text{get}} - 1$  by setting  $\tau' = \tau_{\text{store}} + 1$ . So [Lemma 6](#) is applicable in that case.

Otherwise,  $\tau_{\text{get}} \geq \Delta_{\text{sync}}$ . Assume first that  $\tau_{\text{get}} \geq \tau_{\text{store}} + \Delta_{\text{sync}} + 1$ , so  $\tau_{\text{store}} + 1 \leq \tau_{\text{get}} - \Delta_{\text{sync}} \leq T$ . Then we know that  $\text{WholeColumnStores}_{h,i,x}[c, (\tau_{\text{get}} - \Delta_{\text{sync}}), (\tau_{\text{get}} + 1)]$  holds, which shows that [Lemma 6](#) is applicable.

The remaining case is  $\tau_{\text{get}} \geq \Delta_{\text{sync}}$  and  $\tau_{\text{get}} \leq \tau_{\text{store}} + \Delta_{\text{sync}}$ . Here, we have  $\text{WholeColumnStores}_{h,i,x}[c, \tau', \tau' + 2]$  for some  $\tau_{\text{get}} - \Delta_{\text{sync}} \leq \tau' \leq \tau_{\text{get}} - 1$  by setting  $\tau' = \tau_{\text{store}} + 1$ . So again, [Lemma 6](#) is applicable.

By applying [Lemma 6](#), we thereby obtain  $\text{GotResult}_{h,i,x}^2[\mathcal{P}_{\text{get}}, \tau_{\text{get}}]$  in every case.  $\square$

#### 4.4.3 Good Events Occur for Admissible Schedules

In the second part of our proof, we show that the good events that appeared as conditions in [Lemma 7](#) hold true with sufficiently high probability, provided we have an admissible schedule.

**Lemma 8** (Probability of Bad Columns). *Let  $S$  be a join-leave schedule that guarantees  $N$  honest parties with overlap  $\Delta_{\text{overlap}}$ . Then, for any column  $c \in [k_2]$ , lifetime bound  $T$  and  $\Delta \leq \Delta_{\text{overlap}}$ , we have*

$$\Pr \left[ \text{ColumnGoodUntil}^\Delta[c, T] \right] \geq 1 - \left\lceil \frac{T + 1}{\Delta_{\text{overlap}} - \Delta - 1} \right\rceil \cdot e^{-\frac{N}{k_2}},$$

*where the randomness is over the choice of the random oracle. Further,*

$$\Pr \left[ \text{ColumnGoodUntil}^\Delta[c, T] \text{ for all } c \in [k_2] \right] \geq 1 - \left\lceil \frac{T + 1}{\Delta_{\text{overlap}} - \Delta - 1} \right\rceil \cdot k_2 \cdot e^{-\frac{N}{k_2}}.$$

*Proof.* First, we note that the join-leave schedule and the random oracle fully determine whether  $\text{ColumnGoodUntil}^\Delta[c, T]$  holds or not. For any time slots  $\tau$ , let  $A_\tau$  denote the event that there exist no honest party  $\mathcal{P}$  with  $\text{Active}[\mathcal{P}, \tau \dots \tau + \Delta]$  and  $\text{Col}(\mathcal{P}) = c$ . By definition,  $\text{ColumnGoodUntil}^\Delta[c, T]$  holds iff none of the  $T + 1$  many events  $A_0, \dots, A_T$  holds.

Consider any given  $\tau$ . By assumption on the join-leave schedule, we have  $N$  honest parties  $\mathcal{P}$  with  $\text{Active}[\mathcal{P}, \tau \dots (\tau + \Delta_{\text{overlap}})]$ . The probability (over the choice of random oracle) that none of these  $N$  parties is assigned to column  $c \in [k_2]$  is given by

$$\Pr[\text{None of the } N \text{ parties in column } c] = \left(1 - \frac{1}{k_2}\right)^N < e^{-\frac{N}{k_2}}.$$

If any one of those  $N$  parties is assigned to column  $c$ , then none of the  $\Delta_{\text{overlap}} - \Delta + 1$  many events  $A_\tau, \dots, A_{\tau + \Delta_{\text{overlap}} - \Delta}$  holds, so we have

$$\Pr[A_\tau \vee \dots \vee A_{\tau + \Delta_{\text{overlap}} - \Delta}] < e^{-\frac{N}{k_2}}.$$

We get the first result by dividing  $\{0, \dots, T\}$  into  $(T + 1)/(\Delta_{\text{overlap}} - \Delta + 1)$  intervals of length  $\Delta_{\text{overlap}} - \Delta + 1$  and taking a union bound over them. The claim is obtained by taking a union bound over the columns.  $\square$

**Lemma 9** (Probability of Bad Cells). *Let  $S$  be a join-leave schedule that guarantees  $N$  honest parties with overlap  $\Delta_{\text{overlap}}$ . Let  $T$  be a lifetime bound,  $0 \leq \varepsilon \leq 1$  be some fraction and consider some  $\Delta$  with  $\Delta + \Delta_{\text{sub}} \leq \Delta_{\text{overlap}}$ . Let  $\text{SmallCorruption}^\Delta[\varepsilon, T]$  be the event that for every row  $r$  and time  $0 \leq \tau \leq T$ ,  $\text{GoodCell}[r, c, \tau \dots (\tau + \Delta)]$  holds for all but  $\varepsilon \cdot k_2$  many columns  $c$ .*

*Then we have*

$$\Pr[\text{SmallCorruption}^\Delta[\varepsilon, T]] \geq 1 - \left\lceil \frac{T + 1}{\Delta_{\text{overlap}} - \Delta - \Delta_{\text{sub}} + 1} \right\rceil \cdot 2^{h(\varepsilon)k_2} \cdot k_1 \cdot e^{-\frac{\varepsilon N}{k_1}},$$

where  $h(\varepsilon) = -\varepsilon \log_2(\varepsilon) - (1 - \varepsilon) \log_2(1 - \varepsilon)$  is the binary entropy function and the randomness is over the choice of the random oracle.

*Proof.* Again, we note that the join-leave schedule and the random oracle fully determine whether any  $\text{GoodCell}$  events hold or not. Consider some row  $r \in [k_1]$  and consider some time slot  $\tau$ . By assumption, we are guaranteed that there exists a set  $\mathbf{P}$  of least  $N$  honest parties  $\mathcal{P}$  with  $\text{Active}[\mathcal{P}, \max\{0, \tau - 2 - \Delta_{\text{sub}}\} \dots (\max\{0, \tau - 2 - \Delta_{\text{sub}}\} + \Delta_{\text{overlap}})]$ .

For any given subset  $C \subset [k_2]$  of the columns with size  $|C| = \varepsilon k_2$ , the probability that none of  $\mathbf{P}$  is assigned to  $(r, c)$  with  $c \in C$  is

$$\Pr[\text{no } \mathcal{P} \in \mathbf{P} \text{ has } \text{Cell}(\mathcal{P}) \in \{r\} \times C] = \left(1 - \frac{\varepsilon k_2}{k_1 k_2}\right)^N.$$

Taking a union bound over all  $\binom{k_2}{\varepsilon k_2}$  possible choices of  $C$  gives

$$\Pr\left[\left|\{c \text{ with } \text{Cell}(\mathcal{P}) = (r, c) \text{ for some } \mathcal{P} \in \mathbf{P}\}\right| < (1 - \varepsilon)k_2\right] \leq \binom{k_2}{\varepsilon k_2} \left(1 - \frac{\varepsilon}{k_1}\right)^N.$$

Since  $\binom{k_2}{\varepsilon k_2} \leq 2^{h(\varepsilon)k_2}$  and  $(1 - \frac{\varepsilon}{k_1})^N < e^{-\frac{\varepsilon N}{k_1}}$  by Lemma 12 and Lemma 13, we get

$$\Pr\left[\left|\{c \text{ with } \text{Cell}(\mathcal{P}) = (r, c) \text{ for some } \mathcal{P} \in \mathbf{P}\}\right| < (1 - \varepsilon)k_2\right] \leq 2^{h(\varepsilon)k_2} e^{-\frac{\varepsilon N}{k_1}}.$$

By definition, if  $\left|\{c \text{ with } \text{Cell}(\mathcal{P}) = (r, c) \text{ for some } \mathcal{P} \in \mathbf{P}\}\right| \geq (1 - \varepsilon)k_2$ , then  $\text{GoodCell}[r, c, \tau \dots \tau + \Delta]$  must hold for all except at most  $\varepsilon k_2$  many  $c \in [k_2]$ . In fact, analogous to the argument in Lemma 9,  $\text{GoodCell}[r, c, \tau' \dots \tau' + \Delta]$  holds for the  $\Delta_{\text{overlap}} - \Delta_{\text{sub}} - \Delta + 1$  many consecutive times  $\tau' \in \{\tau, \dots, \tau + \Delta_{\text{overlap}} - \Delta_{\text{sub}} - \Delta\}$ . Splitting the  $T + 1$  time slots from 0 to  $T$  into such interval of length  $\Delta_{\text{overlap}} - \Delta_{\text{sub}} - \Delta + 1$  and taking a union bound and also taking a union bound over the possible choices of  $r$  then gives the claim.  $\square$

#### 4.4.4 Proof of Theorem 1

We now finally piece together the lemmas from above to prove Theorem 1.

*Proof of Theorem 1.* First, we need to bound the sizes of the sets  $\text{Corr}_{\mathcal{P}}^\tau$ . Since  $\text{GetColForSymbol}$  distributes the symbols evenly among the columns and  $k_2$  divides  $m$ , we have

$$\frac{|\text{Corr}_{\mathcal{P}}^\tau|}{m} = \frac{|\{c \mid \neg \text{GoodCell}[r, c, \max\{0, \tau - \Delta_{\text{sync}}\} \dots (\tau + 1)]\}|}{k_2},$$

where  $r = \text{Row}(\mathcal{P})$ . So by Lemma 9, we have  $|\text{Corr}_{\mathcal{P}}^\tau| \leq \varepsilon m$  for all honest parties  $\mathcal{P}$  and all  $0 \leq \tau \leq T$ , except with probability at most

$$\Pr[\exists 0 \leq \tau \leq T, \text{ honest } \mathcal{P} \text{ with } |\text{Corr}_{\mathcal{P}}^\tau| > \varepsilon m] < \left[ \frac{T+1}{\Delta_{\text{overlap}} - (\Delta_{\text{sync}} + 1) - \Delta_{\text{sub}} + 1} \right] 2^{h(\varepsilon)k_2} k_1 e^{-\frac{\varepsilon N}{k_1}}$$

Second, we need to show the correctness if not corrupted property, for which we use Lemma 7. Let  $r_{\text{store}} = \text{Row}(\mathcal{P}_{\text{store}})$ ,  $r_{\text{get}} = \text{Row}(\mathcal{P}_{\text{get}})$  and  $c = \text{GetColForSymbol}(i)$ . Let us argue that the conditions of this lemma are satisfied except with small probability:

- By definition, we have  $\Pr[\text{SubnProtGoodUntil}[T+1]] = \Pr[\text{SubnProtGoodUntil}[T_{\text{SD}}]] \geq 1 - \delta_{\text{SD}}$ .
- By Lemma 8, we have

$$\Pr[\text{ColumnGoodUntil}^{\Delta_{\text{overlap}, \min}}[c, T+1] \text{ for all } c] \geq 1 - \left[ \frac{T+2}{1 + \Delta_{\text{overlap}} - \Delta_{\text{overlap}, \min}} \right] \cdot k_2 e^{-\frac{N}{k_2}}$$

- By definition, since  $i \notin \text{Corr}_{\mathcal{P}_{\text{store}}}^{\tau_{\text{store}}}$ , we have  $\text{GoodCell}[r_{\text{store}}, c, \max\{0, \tau_{\text{store}} - \Delta_{\text{sync}}\} \dots (\tau_{\text{store}} + 1)]$ , which implies  $\text{GoodCell}[r_{\text{store}}, c, \tau_{\text{store}} \dots (\tau_{\text{store}} + 1)]$ .
- By definition, since  $i \notin \text{Corr}_{\mathcal{P}_{\text{get}}}^{\tau_{\text{get}}}$ , we have  $\text{GoodCell}[r_{\text{get}}, c, \max\{0, \tau_{\text{get}} - \Delta_{\text{sync}}\} \dots (\tau_{\text{get}} + 1)]$

This means we can apply Lemma 7, which directly gives us the correctness if not corrupted property with error  $\delta$  bounded by

$$\begin{aligned} \delta &< \delta_{\text{SD}} + \left[ \frac{T+1}{\Delta_{\text{overlap}} - \Delta_{\text{sync}} - \Delta_{\text{sub}}} \right] 2^{h(\varepsilon)k_2} k_1 e^{-\frac{\varepsilon N}{k_1}} + \left[ \frac{T+2}{1 + \Delta_{\text{overlap}} - \Delta_{\text{overlap}, \min}} \right] \cdot k_2 e^{-\frac{N}{k_2}} \\ &< \delta_{\text{SD}} + \left[ \frac{T+2}{1 + \Delta_{\text{overlap}} - \Delta_{\text{overlap}, \min}} \right] \cdot \left( 2^{h(\varepsilon)k_2} k_1 e^{-\frac{\varepsilon N}{k_1}} + k_2 e^{-\frac{N}{k_2}} \right), \end{aligned}$$

finishing the proof.  $\square$

## 4.5 Efficiency

We now discuss the efficiency of our protocol. Note that while the number of adversarial parties has no impact on the robustness of our protocol, efficiency depends on the total number of parties, and therefore on the number of adversarial parties. Throughout this section, we denote the (upper bound on) the number of simultaneously active honest parties by  $n_{\text{hon}}^{\max}$  and the (upper bound on) the number of total simultaneously active<sup>10</sup> parties by  $n^{\max} \geq n_{\text{hon}}^{\max}$ . We assume that the subnet discovery protocol is instantiated as in Appendix B, with the optimizations described there.

**Latency and Round Complexity.** When an honest party calls  $\text{GET}$ , the latency is at most 3. Precisely, a call to  $\text{GET}$  made at time  $\tau$  returns either a valid  $x$  at time  $\tau + 2$  or  $\perp$  at time  $\tau + 3$ . For interface  $\text{STORE}$ , the user calling  $\text{STORE}$  at  $\tau$  does not expect a response and the call returns immediately. By the end of round  $\tau + 2$ , all messages caused by this call have been received. For  $\text{JOIN}$  called at time  $\tau$ , the call terminates at  $\tau + 2 + \Delta_{\text{sub}}$ . Synchronization of data may take longer, but we emphasize that as soon as the call is terminated, the party can already participate in the protocol via  $\text{GET}$  and  $\text{STORE}$ .

<sup>10</sup>Recall that we defined *active* for honest parties by them having joined. Since malicious parties do not call  $\text{JOIN}$  or  $\text{INIT}$ , *active* for them shall mean that honest parties may keep a connection to it.

**Data Per Node.** We first examine how much data a node needs to store. For simplicity, assume that exactly one file  $(x_1, \dots, x_m) \in \Sigma^m$  is stored. Recall that each party resides in exactly one column and that nodes in a column are responsible for storing  $m/k_2$  symbols. Therefore, the data storage requirement for each node is  $m/k_2$ .

**Connections Per Node.** Now, let us consider how many connections to peers a node needs to keep. We consider a fixed point in time  $\tau$ . Our goal is to derive a bound for the expected number of connections that a fixed node  $\mathcal{P}$  has to maintain, taking the randomness over the random oracle. We also assume that  $\mathcal{P}$  is not a bootstrap node ( $\text{role} = 0$ ). When joining,  $\mathcal{P}$  needs to connect to each of its supplied bootstrap nodes, but those connections can be dropped after two rounds and we ignore those. Let  $\text{Cell}(\mathcal{P}) = (r, c)$ . Let  $\text{Peers}$  be the number of peers  $\mathcal{P}$  has. As argued above, we may assume that  $\mathcal{P}$  will only possibly keep a connection to another  $\mathcal{P}'$  if  $\text{Row}(\mathcal{P}') = r$  or  $\text{Col}(\mathcal{P}') = c$  and  $\mathcal{P}'$  is active.  $\text{Row}(\mathcal{P}') = r \vee \text{Row}(\mathcal{P}') = c$  happens with probability (over the choice of random oracle)  $(k_1 + k_2 - 1)/(k_1 k_2)$ . Since there are at most  $n^{\max} - 1$  such parties  $\mathcal{P}'$ , we can bound the expected number of connections as

$$\mathbb{E}[|\text{Peers}|] \leq (n^{\max} - 1) \frac{k_1 + k_2 - 1}{k_1 k_2} \leq \frac{n^{\max} - 1}{k_1} + \frac{n^{\max} - 1}{k_2}.$$

**Communication Complexity.** For communication complexity, we consider the expected total communication that is caused by honest parties calling protocol interfaces. We do only count the outgoing data of honest parties, as one can never limit the amount of data that dishonest parties send. For simplicity, we denote the maximum size of a single message (with prefix `get`, `get_rsp` or `store`, `store_fwd`, `join`) by  $L_{\text{msg}}$ . Note that  $L_{\text{msg}}$  depends on the parameters  $|\mathcal{H}|$ ,  $m$ , and  $|\Sigma|$ . For messages of the form  $(\text{join\_rsp}, \mathbf{P}_c^{(i)})$ , we treat them as  $|\mathbf{P}_c^{(i)}|$  messages of size  $L_{\text{msg}}$ .

First, consider an honest party  $\mathcal{P}$  calling  $\text{GET}(h, i)$ . Here,  $\mathcal{P}$  identifies a cell  $(r, c)$ , where column  $c$  is responsible for storing the  $i$ th symbol, then sends a `get`-message to all nodes in that cell. The nodes respond with a `get_rsp`-message. We use  $|\cdot|$  to denote the length of strings, and denote by  $\text{NodesInCell}_{r,c}$  (resp.  $\text{HonNodesInCell}_{r,c}$ ) the random variable modeling the number of nodes (resp. honest nodes) in cell  $(r, c)$ . Then, the expected (over the choice of the random oracle) communication complexity is bounded by

$$\mathbb{E}[\text{NodesInCell}_{r,c}] \cdot L_{\text{msg}} + \mathbb{E}[\text{HonNodesInCell}_{r,c}] \cdot L_{\text{msg}} \leq \left( \frac{n^{\max}}{k_1 k_2} + \frac{n_{\text{hon}}^{\max}}{k_1 k_2} \right) \cdot L_{\text{msg}},$$

where the first summand accounts for `get`-messages and the second for `get_rsp`-responses of honest parties.

Next, we consider an honest party  $\mathcal{P}$  calling  $\text{STORE}(h, i, x)$ . Again,  $\mathcal{P}$  identifies all parties in an appropriate cell  $(r, c)$  and then sends a `store`-message. All parties in the cell then send `store_fwd`-messages to all parties in column  $c$ . Denoting the number of parties in column  $c$  by  $\text{NodesCol}_c$ , the following random variable models the communication complexity:

$$\text{NodesInCell}_{r,c} \cdot L_{\text{msg}} + \text{HonNodesInCell}_{r,c} \cdot \text{NodesCol}_c \cdot L_{\text{msg}}.$$

To get an upper bound on the expectation, we use [Corollary 4](#) with  $Y = \text{HonNodesInCell}_{r,c}$ ,  $X = \text{NodesCol}_c$ , and  $p = 1/k_2$ . Then, as soon as  $n^{\max} \geq 3k_2 \ln k_2$ , the expected communication becomes

$$\left( \frac{n^{\max}}{k_1 k_2} + \frac{3n_{\text{hon}}^{\max} n^{\max}}{k_1 k_2^2} \right) \cdot L_{\text{msg}}.$$

Finally, we consider an honest party  $\mathcal{P}$  with  $\text{role} = 0$  joining the network. We ignore communication complexity from data synchronization, as this depends on the size of the data. Assume that it joins via  $t$  bootstrap nodes  $\{\mathcal{P}_1, \dots, \mathcal{P}_t\}$ . As we only count outgoing messages of honest parties, the worst case is when these  $t$  bootstrap nodes are all honest. Let  $n^{\text{BS}}$  denote the total number of currently active bootstrap nodes.

Denote by  $\text{ComRowSub}(S)$  resp.  $\text{ComColSub}(S)$  the communication complexity of joining a row resp. column subnet containing  $S$  nodes, where for the row subnets  $\text{sid}_r$  we do not count (bootstrap) nodes  $\mathcal{P}'$  with  $\text{Row}(\mathcal{P}') \neq r$ . With our protocol from [Appendix B](#), we have

$$\text{ComColSub}(S) \leq L_{\text{msg}}(2 + 2S).$$

Using the optimizations that we have outlined in the end of [Appendix B](#), the `join_subn_pull_rsp`-messages to parties do not contain bootstrap nodes, and so we have

$$\text{ComRowSub}(S) \leq L_{\text{msg}}(2 + 2(S + n^{\text{BS}}) - n^{\text{BS}}) = L_{\text{msg}}(2 + 2S + n^{\text{BS}}).$$

Denoting the number of parties in row  $r$  by  $\text{NodesRow}_r$ , we can bound the communication complexity induced by an honest party with `role` = 0 joining by

$$\begin{aligned} & t \cdot \text{ComRowSub}(\text{NodesRow}_r) + t(L_{\text{msg}} + \text{NodesCol}_c \cdot L_{\text{msg}}) + \text{NodesCol}_c \cdot \text{ComColSub}(\text{NodesCol}_c) \\ & \leq L_{\text{msg}}(3t + 2t \cdot \text{NodesRow}_r + (t + 2) \cdot \text{NodesCol}_c + 2 \cdot \text{NodesCol}_c^2 + tn^{\text{BS}}). \end{aligned}$$

We get the expected value of this using linearity of expectation and [Lemma 10](#) for the quadratic term. Namely, the expected communication is bounded by

$$\left( 3t + tn^{\text{BS}} + \frac{tn^{\text{max}}}{k_1} + \frac{(t + 4)n^{\text{max}}k_2 - 2n^{\text{max}} + (n^{\text{max}})^2}{k_2^2} \right) \cdot L_{\text{msg}}.$$

## 5 Extensions and Practical Considerations

In practical deployments of our protocol, various modifications and extensions may be beneficial. This section informally discusses several such enhancements.

**Virtual Nodes.** The robustness of our protocol relies on probabilistic guarantees that depend heavily on the number of honest nodes. We can artificially increase this number by making each node simulate  $K \geq 1$  *virtual nodes*. This enables us to apply our analysis to a larger effective set of honest nodes, increasing their count from  $N$  to  $N' = N \cdot K$ . A key advantage of this approach is reduced variance in the probabilistic processes involved, leading to improved protocol parameters such as smaller  $\delta$  and  $\varepsilon$ . Of course, this comes at the cost of increased storage requirements per node, as each node participates in multiple columns. This tradeoff can be beneficial for system designers who can afford additional storage but cannot increase  $N$ . The use of virtual nodes to improve variance is a well-established technique, e.g., used in consistent hashing [\[KLL<sup>+</sup>97\]](#).

**Sending Leave Messages.** An honest party may additionally send leave messages before leaving to all of its peers. In this way, peers can free memory and do not need to keep connections open. Importantly, such leave messages must be authenticated, as otherwise an adversary could use them to break connections between two honest peers.

**Efficient Data Synchronization.** Our protocol requires nodes in the same column to synchronize stored data, particularly when a new node joins or when new data is added. For simplicity, we have described the naive approach where each node transmits all of its stored data to its peers. However, more efficient methods exist: Set reconciliation techniques [\[MTZ03, BKW24, YGA24\]](#) can significantly reduce communication overhead in such settings. These methods can be integrated into our protocol.

**Synchronization Time.** The synchronization delay,  $\Delta_{\text{sync}}$ , influences the admissibility condition of our join-leave schedules ([Definition 8](#)). Further, while our theoretical description of our protocol required that parties have explicit knowledge of  $\Delta_{\text{sync}}$  when responding to sync-messages, parties executing the protocol in practice do not need to know the exact value of  $\Delta_{\text{sync}}$ . Instead, they should respond to sync-messages as quickly as possible. Consequently, if  $\Delta_{\text{sync}}$  increases, slightly stronger honesty assumptions are required, but the protocol's code remains unchanged. This property is particularly relevant in practice, as synchronization time may increase over time with the growth of stored data. Hence, we can think of  $\Delta_{\text{sync}}$  as depending on the time, which also means the admissibility condition changes over time.

**Lazy Cell Messages.** Many operations in our protocol involve querying *all* nodes in a cell in parallel. For example, in interface `GET`, a node requests data from all nodes in a cell simultaneously. A more bandwidth-efficient alternative is to query nodes sequentially, stopping once data satisfying the predicate is retrieved, or to query in small batches, proceeding incrementally. A practical heuristic is to prioritize nodes based on past responsiveness, contacting the most responsive nodes first. This approach introduces a bandwidth-latency tradeoff that can be adjusted to suit different system requirements.

**Less Powerful Bootstrap Nodes.** In our protocol description, we assumed that parties take on either the role of a normal node or a bootstrap node. We defined bootstrap nodes to be present in every row. In this way, when a party wishes to join a cell  $(r, c)$ , it can join its row subnet  $r$  via the bootstrap node. Further, it can join its column subnet  $c$  through nodes in column  $c$  obtained from the bootstrap node. Consequently, bootstrap nodes are more powerful and have higher bandwidth requirements.

If, in practice, one wishes to avoid relying on such powerful bootstrap nodes, a slight modification to the join process can address this issue. Specifically, suppose a party  $\mathcal{P}$  joins the protocol through a (normal) node<sup>11</sup>  $\mathcal{P}'$ . Assume that  $\mathcal{P}$  intends to join cell  $(r, c)$  while  $\mathcal{P}'$  resides in cell  $(r', c')$ . Then,  $\mathcal{P}$  can request from  $\mathcal{P}'$  a list of all nodes in cell  $(r, c')$  and use them to join its row subnet  $r$ . Similarly,  $\mathcal{P}$  can request from  $\mathcal{P}'$  a list of all nodes in cell  $(r', c)$  and use them to join its column subnet  $c$ . This approach works as long as  $\mathcal{P}'$  is honest and the cells  $(r, c')$  and  $(r', c)$  each contain at least one honest party.

## 6 Benchmarks

In this section, we derive concrete numbers from our robustness analysis and present some simulation results. The code for generating our plots can be found at

<https://github.com/b-wagn/rda-experiments>.

### 6.1 Concrete Numbers from Analytical Results

The robustness theorem (Theorem 1) shows a relation between the error  $\delta$ , the maximum fraction of corrupted symbols  $\varepsilon$ , the minimum number of honest parties  $N$ , and the number of rows and columns  $k_1$  and  $k_2$ . Also, by Section 4.5, there is a relation between  $k_1$  and  $k_2$  and the efficiency of our protocol. We show these dependencies in Figures 5 and 6, and explain the plots next.

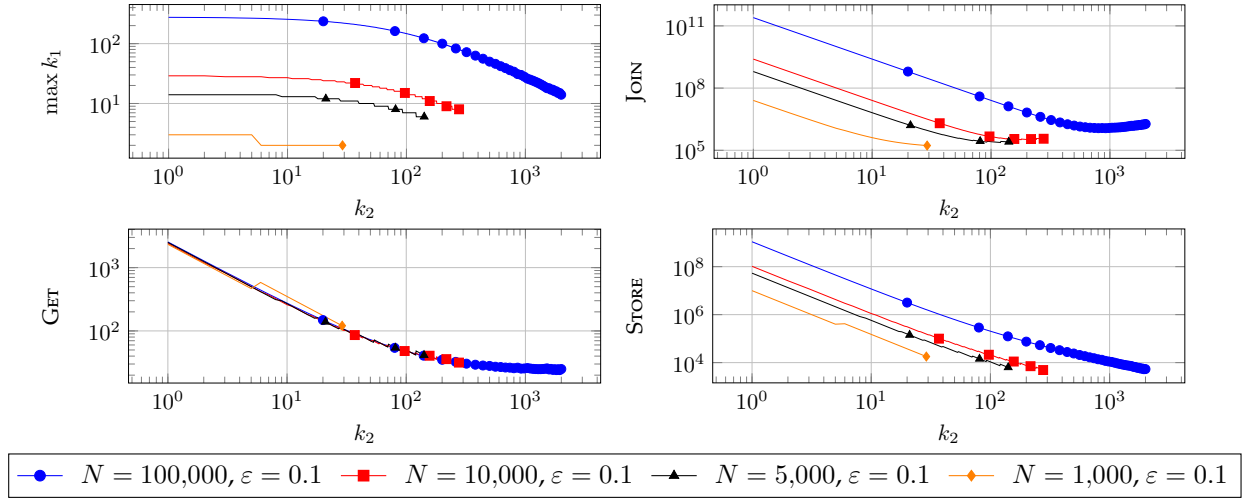


Figure 5: Trade-off between  $k_1$  (number of rows) and  $k_2$  (number of columns) and resulting complexities for  $\varepsilon = 0.1$ . The complexities show the expected total message complexity when an honest party calls the respective interfaces. All plots assume  $\delta \leq 10^{-9}$ , and the given  $N$ . Scales are logarithmic.

**Setting and Assumptions.** The subnet discovery protocol in Appendix B is perfectly robust, i.e., we can assume  $\delta_{SD} = 0$  in Theorem 1. We assume that one round in the synchronous model takes 4 seconds and target a protocol lifetime of 10 years (which determines the value for  $T$ ). We set  $\Delta_{\text{overlap}, \min} = 450$ , which means we assume that data can be synchronized in at most roughly 15 minutes. We set  $\Delta_{\text{overlap}}$  by assuming the average honest nodes stays online for 6 hours. We target an error probability of at most  $\delta \leq 10^{-9}$ . For these values, we then take the following approach to arrive at the plots in Figures 5

<sup>11</sup>The reader may recall that a party can join via multiple nodes, but for simplicity, we describe the process assuming it joins via a single honest node.



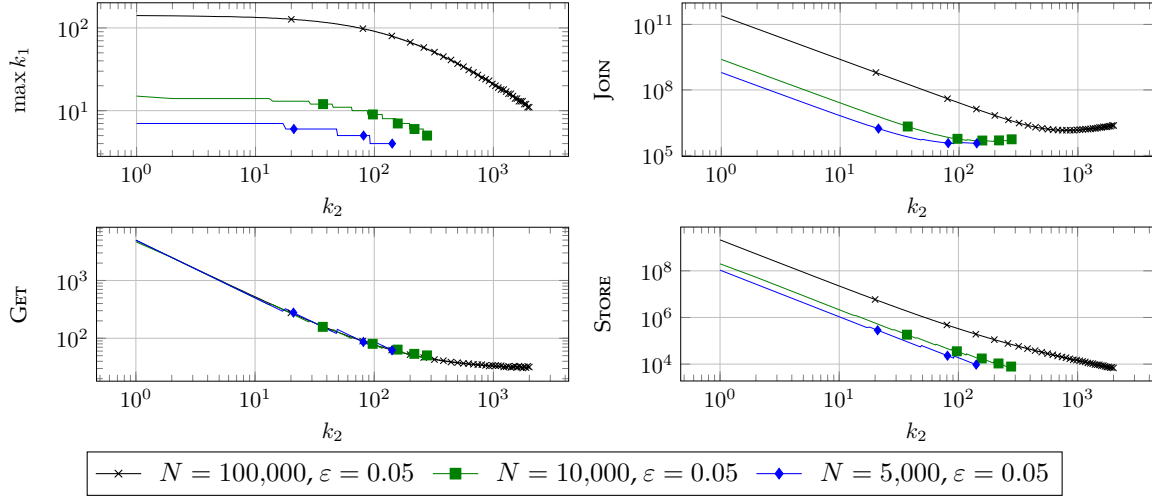


Figure 6: Trade-off between  $k_1$  (number of rows) and  $k_2$  (number of columns) and resulting complexities for  $\varepsilon = 0.05$ . The complexities show the expected total message complexity when an honest party calls the respective interfaces. All plots assume  $\delta \leq 10^{-9}$  and the given  $N$ . Scales are logarithmic.

and 6: we consider a variety of combinations of  $N$  and  $\varepsilon$ , and for each of those, we do the following: (1) we generate a trade-off curve between  $k_1$  and  $k_2$ . This curve shows for each  $k_2$  (number of columns) the maximum  $k_1$  (number of rows) for which the bound in [Theorem 1](#) yields the desired  $\delta$ ; (2) we then generate plots showing the expected communication complexity of GET and JOIN (as analyzed in [Section 4.5](#)) depending on  $k_2$ , viewing  $k_1$  as a function of  $k_2$  using (1). For (2), we assume  $L_{\text{msg}} = 1$ , i.e., we plot message complexity. For the complexity of JOIN, we assume  $t = 50$  bootstrap nodes, and  $n^{\text{BS}} = 100$ . We assume that  $n_{\text{hon}}^{\text{max}} = 2N$  and  $n^{\text{max}} = 5N$ .

**Interpretation.** We can interpret these plots by first observing that a given number of honest nodes  $N$  can only support a limited number of cells, assuming that (most) of the cells have to contain one honest node. This creates the tradeoff between  $k_1$  and  $k_2$  as shown in the plots; the total number of cells is  $k_1 \cdot k_2$  (top left graph). Intuitively, one might think that the plot would represent the hyperbola  $k_1 \cdot k_2 \approx N$ . However, with a little thought experiment we can see why the fact that we fix an error threshold (for example,  $\varepsilon = 0.1$ ) creates a deviation from the hyperbola:

- If we choose  $k_1 = 1$ , then the bound  $\varepsilon$  applies to the total number of cells that are corrupted, i.e., contain no honest party.
- However, choosing a larger number of rows  $k_1$  means that the bound has to apply to every individual row, and not just to all  $k_1 \cdot k_2$  cells globally.

The result is that the number of cells  $k_1 \cdot k_2$  is maximized for  $k_1 = 1$ . This is relevant for operations for which the cost is inversely proportional to  $k_1 \cdot k_2$ , such as GET.

To choose a particular number for  $k_1$  and  $k_2$  with the goal of maximizing the efficiency of the protocol, i.e., the total message complexity, we look at the complexity of GET and STORE for storing data as well as the complexity of JOIN, which can be seen as the cost of maintaining the networking topology.

- GET and STORE are always minimized by choosing  $k_1 = 1$  and maximizing  $k_2$ . This is a result of the above observation that this maximizes the number of cells  $k_1 \cdot k_2$ .
- JOIN complexity has a minimum where  $k_1 \approx k_2$ , representing the point where the cost of joining rows and columns is balanced.

So, depending on the relative complexity of the two operations, we can choose  $k_1$  and  $k_2$  to minimize the total message complexity. In applications where storing and retrieving data is very common and therefore dominates complexity, we probably want to choose  $k_1$  to be small while maximizing  $k_2$ . However, if JOIN is expensive (for example, because there are a lot of nodes, or joining and leaving is frequent), we should choose a parameter set closer to the minimum of JOIN complexity, i.e.,  $k_1 \approx k_2$ .

## 6.2 Results from Simulation

We complement our analytical results with simulations of our protocol (excluding extensions).

**Simulation Setup.** We fix a specific join-leave schedule and values for the parameters  $k_1$  and  $k_2$ , and simulate the assignment of honest parties to cells. At each time step  $\tau$ , we compute the number of connections a party  $\mathcal{P}$  has, denoted by  $|\text{Peers}_{\mathcal{P}}^{\tau}|$ , assuming that a party is immediately connected to all parties in the same row and column. We also evaluate the fraction of corrupted symbols for each party, denoted as  $|\text{Corr}_{\mathcal{P}}^{\tau}|/m$ , where a symbol in column  $c$  is considered corrupted for a party in row  $r$  if the cell  $(r, c)$  contains no honest party. This simulation is intentionally simplified: it does not account for timing constraints or overlap times. Nonetheless, it captures the key characteristics of our protocol. Importantly, due to the protocol’s design, we do not need to model the behavior of dishonest parties. For our plots, we fix the number of columns to  $k_2 = 100$  and vary the number of rows  $k_1$ . The simulated join-leave schedule begins with 20 initial parties. Then follows a warm-up phase in which 2,480 additional parties join, one per round, until a total of 2,500 parties are active. After this phase, we introduce churn: in each subsequent step, 50 parties leave (in FIFO order) and 50 new parties join. Consequently, each party remains in the system for exactly  $2,500/50 = 50$  time steps.

**Results.** Our simulation results are presented in Figure 7, where we plot  $\max_{\mathcal{P}} |\text{Corr}_{\mathcal{P}}^{\tau}|/m$  and  $\max_{\mathcal{P}} |\text{Peers}_{\mathcal{P}}^{\tau}|$  as functions of time  $\tau$ . One key observation is that after the warm-up phase, the fraction of corrupted symbols stabilizes. Notably, errors do not accumulate over time. We further observe that the fraction of corrupted symbols decreases as the number of rows  $k_1$  is reduced. This is expected, as fewer rows imply fewer total cells, increasing the probability that a cell contains an honest party. In contrast, the number of peers (and thus bandwidth usage) increases with fewer rows. Hence, our design demonstrates a trade-off between robustness (fewer corrupted symbols) and bandwidth (number of peer connections).

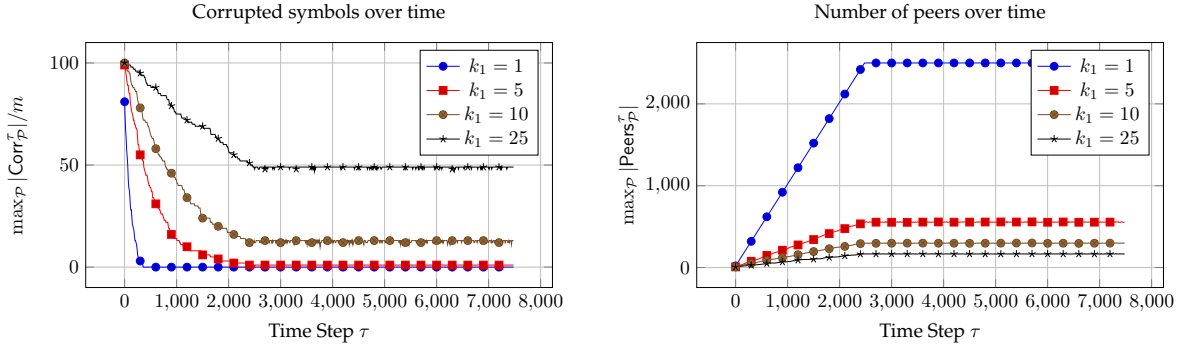


Figure 7: Simulation results for  $k_2 = 100$  columns and different number of rows  $k_1$ . We simulated a join-leave schedule that leads to 2,500 honest parties (after a warmup phase) and each party staying for 50 time steps.

## 7 More Related Work

In this section, we provide a more extensive overview of related work. To summarize, our work is the first to rigorously address the networking aspect of DAS. Our robust distributed arrays share similarities with distributed hash tables but differ significantly in their design goals. Namely, in contrast to solutions based on distributed hash tables, our construction achieves simplicity, low query latency, and provable security even under dishonest majority.

### 7.1 Data Availability Sampling

Our primary motivation for introducing and studying robust distributed arrays is *data availability sampling* (DAS), first introduced by Al-Bassam et al. [ASBK21]. To provide context for our definition, we review the fundamentals of DAS in Section 3.1. Here, we discuss related work in the context of DAS.

**Cryptography for DAS.** The cryptographic and encoding aspects of DAS were first formally defined as a cryptographic primitive in [HASW23]. Since then, these cryptographic aspects have been explored further [HASW24, WZ24, EMA25, EA25]. Notably, all of these works assume an abstract, black-box networking layer. Prior to our work, there was no formal definition of the networking requirements or a construction that provably satisfies them.

**Networking for DAS.** While there has been no rigorous formalization of networking in the context of DAS, several recent studies have explored potential approaches and identified key challenges. Król et al. [KAR<sup>+</sup>23] have (informally) identified functionality and efficiency requirements for a DAS networking layer, such as permissionlessness and low-latency storage requests. They have also discussed why classical networking solutions, including distributed hash tables (DHTs) like Kademlia [MM02], are not directly applicable to DAS without significant modifications. Cortes-Goicoechea et al. [CKR<sup>+</sup>24] specifically analyze the suitability of Kademlia for DAS in terms of efficiency, though not security. Using a custom DHT simulator, they evaluate Kademlia’s performance and conclude that while query efficiency is reasonable, the workload associated with storing the data is prohibitive for DAS applications. A notable proposal for DAS networking is PANDAS [AKP<sup>+</sup>24]. This proposal assumes that each node maintains an (almost) complete list of all other nodes through continuous background discovery. Nodes are deterministically assigned to specific chunks of encoded data using a hash function. In combination, this ensures that data can be retrieved in a single hop with minimal latency. Conceptually, PANDAS can be viewed as a special case of our proposed solution with exactly one row. However, there are differences: PANDAS assumes that node-to-chunk mappings change with each slot, whereas we propose a stable assignment where nodes remain within their designated cells until they leave the network. Most importantly, however, PANDAS lacks a formal security definition or proof.

## 7.2 Distributed Hash Tables

In many ways, distributed arrays bear similarities to distributed hash tables (DHTs). Therefore, we discuss DHTs and how they relate to distributed arrays and our design goals.

**Distributed Hash Tables (DHTs).** A *hash table* is a classical data structure that implements a *key-value store*, dynamically and efficiently maintaining a partial mapping  $T: \mathcal{U} \rightarrow \mathcal{V}$  between a large universe of keys  $\mathcal{U}$  and a domain of values  $\mathcal{V}$ . It supports an *insert* operation, which assigns a value  $v \in \mathcal{V}$  to a key  $k \in \mathcal{U}$  (i.e.,  $T(k) := v$ ), and a *query* operation, which retrieves  $T(k) \in \mathcal{V} \cup \{\perp\}$  given a key  $k \in \mathcal{U}$ . A *distributed hash table* extends this concept by distributing key-value pairs across multiple nodes, ensuring efficient storage and retrieval by contacting only a few nodes. DHTs function effectively even when nodes dynamically join or leave and operate with only a local view of the network topology. Prominent examples of DHTs include CAN [RFH<sup>+</sup>01], Pastry [RD01], Chord [SMK<sup>+</sup>01], and Kademlia [MM02], which leverage ideas from consistent hashing [KLL<sup>+</sup>97].

**Flat Key Space vs. Indices.** A key distinction between DHTs and our distributed arrays lies in the structure of the key space. In a DHT, key-value pairs can have arbitrary keys from a large universe  $\mathcal{U}$ , with no structural constraints (e.g.,  $\mathcal{U} = \{0, 1\}^{256}$ ). The actual set of inserted keys forms only a small subset of  $\mathcal{U}$ . By contrast, in our distributed arrays, parties query indices  $i \in [m]$ . That is, keys are always small integers with a predefined structure.

**Orthogonal Design Goals.** While DHTs could theoretically implement distributed arrays, this approach is unsuitable for our context. First, classical DHTs [RFH<sup>+</sup>01, RD01, SMK<sup>+</sup>01, MM02] lack provable robustness against adversarial behavior (though some robust variants exist, see below). Second, DHTs prioritize minimizing the number of stored peers, at the cost of (poly-)logarithmic query latencies. This is because when these solutions have been developed over two decades ago, memory was expensive, as already discussed in [KAR<sup>+</sup>23]. In contrast, in our modern setting we can assume each node having sufficient memory to store *almost all* other peers, though without requiring full peer storage to avoid high bandwidth costs. Given the importance of minimizing query latency in DAS, our design goals fundamentally differ from those of DHTs. Although recent work [ZV23] optimizes Kademlia’s concrete latency, it still lacks formal security analysis.

**Kelips DHT.** An exception to the typical  $\Omega(\text{polylog}(N))$  query latency in DHTs is Kelips [GBL<sup>+</sup>03], which achieves  $\Theta(1)$  query latency at the expense of increased memory usage and background communication. This design choice aligns closely with our goals. Moreover, both our construction and Kelips employ

a similar conceptual framework: Kelips randomly partitions nodes into *affinity groups* – akin to our construction’s rows – with each node maintaining an (almost complete) list of peers within its affinity group and partial connections to other groups. In this way, Kelips enables queries in *two* hops, whereas our approach achieves *one*-hop queries by leveraging additional connections along columns (cf. Figure 1). Furthermore, unlike Kelips, our construction provides a formal security proof and rigorous definition.

**Adversarially Robust DHTs.** To add robustness in presence of byzantine adversaries, Baumgart and Mies introduced S/Kademlia [BM07], a variant of the well-known Kademlia DHT. Their work systematically analyzes multiple attack vectors against Kademlia and integrates a set of ad-hoc countermeasures into the protocol. While these modifications improve practical robustness, they do not provide formal guarantees of security.

A significant line of research aims to establish a more rigorous understanding of security in DHTs, ultimately converging on *quorum-based* solutions. Castro et al. [CDG<sup>+</sup>02] have observed in 2002 that while network overlays such as DHTs offer resilience against *random crash failures*, they lack guarantees against coordinated malicious failures. Their robustness definition informally asserts that any honest query for a specific key should reach all honest nodes responsible for storing that key. Around the same time, Fiat and Saia [FS02] proposed a content-addressable network resilient to *malicious node deletions*. Their approach groups nodes into *supernodes* (or *quorums*), which collectively act as vertices in a butterfly network, facilitating message routing through this structure. A simpler construction by Naor and Wieder [NW03] also provides robustness against random crash failures and makes use of overlapping quorums induced by a continuous graph.

Beyond random crash failures, Fiat, Saia, and Young introduced the *byzantine join adversary* (also known as *adaptive join-leave attacks*) and adapted Chord to be resilient against it [FSY05]. This adversary can adaptively insert malicious nodes into the network, allowing them to choose their identifiers and thus their positions within the DHT topology. Fiat, Saia, and Young’s approach replaces each Chord node with a quorum (they call it a *swarm*) of logarithmically many nodes. As long as each quorum maintains a (strong) *honest majority*, the protocol remains secure. Several subsequent works have extended this approach [AS04, Sch05, AS06b, AS06a, AS07, SF12, YKGK13, GHK13] and studied how this local honest majority condition can be achieved. However, such quorum-based techniques exhibit three key limitations making them unsuitable for use in DAS: (1) achieving a (strong) honest majority within quorums necessitates a (strong) honest majority among all nodes; (2) quorum management requires complex sub-protocols, such as distributed coin flips [AS06a] or distributed key generation [YKGK13]; (3) similar to classical DHTs, these protocols have (poly-)logarithmic latency, and they introduce additional polylogarithmic overheads, particularly in terms of message complexity. Jaiyeola et al. [JPS<sup>+</sup>18] proposed a method that reduces quorum sizes (and consequently overhead) to  $\Theta(\log \log N)$ . To achieve this, they need to limit the adversary’s join rate and relax the robustness requirements to hold only for a large fraction of the nodes.

### 7.3 Further Related Work

Here, we discuss further related work.

**Distributed Data Structures.** We have already discussed distributed hash tables as a fundamental example of a distributed data structure. There is a rich literature on distributed data structures (e.g., [AHL<sup>+</sup>13, AY15, BS20]). We refer the reader to these works for further references. In the following, we highlight two examples that share similarities with our robust distributed arrays:

Skip graphs, introduced by Aspnes and Shah [AS03], serve as a distributed counterpart to balanced search trees, offering efficient range and successor queries – capabilities that traditional distributed hash tables lack. Moreover, their construction is resilient to a substantial fraction of nodes crashing. Over time, skip graphs have been extensively analyzed and extended [GNS06, GMS08, JRS<sup>+</sup>09, AW09, HG17].

To the best of our knowledge, only one prior work explicitly considers distributed arrays [FSSH09]. Similar to skip graphs, their motivation is the observation that distributed hash tables do not maintain locality in a logical array structure. Specifically, if an array  $a[0], \dots, a[n-1]$  is stored in a DHT using key-value pairs  $(i, a[i])$ , then consecutive elements  $a[i]$  and  $a[i+1]$  are unlikely to be stored near each other. To mitigate this, their approach modifies Chord [SMK<sup>+</sup>01] by employing a reverse-bit order mapping. As a result, this enables more efficient sequential access, sorted array searches, and more. Compared to our work, the key distinctions are: (1) their approach does not define or guarantee any

robustness, and (2) our focus is not primarily on locality features such as efficiency of sequential access. Still, our constructions supports some limited form of locality: symbols stored within the same column are stored on the same node.

**Random Node Sampling.** One potential approach to constructing networking protocols for DAS is the use of peer discovery protocols, a method widely considered by practitioners [Rya23, Chu24]. In this approach, each node employs a peer discovery protocol to establish connections with a sufficiently large number,  $L$ , of peers. When a node needs to query a specific position, it relies on the assumption that at least one of these  $L$  peers both stores the required position and behaves honestly. Existing analyses of this approach [Chu24] assume that peers are sampled uniformly at random and independently from the overall peer set. Under this assumption, if there are sufficiently many honest nodes and  $L$  is large enough, most positions will be adequately covered. In essence, these analyses implicitly assume that the peer discovery protocol securely implements a *random node sampling functionality*. However, peer discovery protocols used in practice do not necessarily fulfill this requirement.

There is existing literature on byzantine-resistant random node sampling [BGK<sup>+</sup>08, ABG10, ABFT21, ZV24]. However, some of these works lack formal analysis, while for others, it remains unclear whether they truly instantiate the required functionality – particularly regarding whether the sampled nodes are independent.



## References

- [ABFT21] Alex Auvolat, Yérom-David Bromberg, Davide Frey, and François Taïani. BASALT: A rock-solid foundation for epidemic consensus algorithms in very large, very open networks. *CoRR*, abs/2102.04063, 2021. (Cited on Page 37.)
- [ABG10] Emmanuelle Anceaume, Yann Busnel, and Sébastien Gambs. Uniform and ergodic sampling in unstructured peer-to-peer systems with malicious nodes. In Chenyang Lu, Toshimitsu Masuzawa, and Mohamed Mosbah, editors, *Principles of Distributed Systems - 14th International Conference, OPODIS 2010, Tozeur, Tunisia, December 14-17, 2010. Proceedings*, volume 6490 of *Lecture Notes in Computer Science*, pages 64–78. Springer, 2010. (Cited on Page 37.)
- [AHL<sup>+</sup>13] Chen Avin, Bernhard Haeupler, Zvi Lotker, Christian Scheideler, and Stefan Schmid. Locally self-adjusting tree networks. In *27th IEEE International Symposium on Parallel and Distributed Processing, IPDPS 2013, Cambridge, MA, USA, May 20-24, 2013*, pages 395–406. IEEE Computer Society, 2013. (Cited on Page 36.)
- [AKP<sup>+</sup>24] Onur Ascigil, Michał Król, Matthieu Pigaglio, Sergi Reñe, Etienne Rivière, and Ramin Sadre. PANDAS: A Practical Approach for Next-Generation Data Availability Sampling. <https://ethresear.ch/t/pandas-a-practical-approach-for-next-generation-data-availability-s> 2024. [Accessed 21-02-2025]. (Cited on Page 4, 12, 35.)
- [AS03] James Aspnes and Gauri Shah. Skip graphs. In *14th SODA*, pages 384–393. ACM-SIAM, January 2003. (Cited on Page 36.)
- [AS04] Baruch Awerbuch and Christian Scheideler. Group spreading: A protocol for provably secure distributed name service. In Josep Díaz, Juhani Karhumäki, Arto Lepistö, and Donald Sannella, editors, *Automata, Languages and Programming: 31st International Colloquium, ICALP 2004, Turku, Finland, July 12-16, 2004. Proceedings*, volume 3142 of *Lecture Notes in Computer Science*, pages 183–195. Springer, 2004. (Cited on Page 4, 36.)
- [AS06a] Baruch Awerbuch and Christian Scheideler. Robust random number generation for peer-to-peer systems. In Alexander A. Shvartsman, editor, *Principles of Distributed Systems, 10th International Conference, OPODIS 2006, Bordeaux, France, December 12-15, 2006, Proceedings*, volume 4305 of *Lecture Notes in Computer Science*, pages 275–289. Springer, 2006. (Cited on Page 4, 36.)
- [AS06b] Baruch Awerbuch and Christian Scheideler. Towards a scalable and robust DHT. In Phillip B. Gibbons and Uzi Vishkin, editors, *SPAA 2006: Proceedings of the 18th Annual ACM Symposium on Parallelism in Algorithms and Architectures, Cambridge, Massachusetts, USA, July 30 - August 2, 2006*, pages 318–327. ACM, 2006. (Cited on Page 4, 36.)
- [AS07] Baruch Awerbuch and Christian Scheideler. Towards scalable and robust overlay networks. In John R. Douceur and Roger Wattenhofer, editors, *6th International workshop on Peer-To-Peer Systems, IPTPS 2007, Bellevue, WA, USA, February 26-27, 2007*, 2007. (Cited on Page 4, 36.)
- [ASBK21] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. Fraud and data availability proofs: Detecting invalid blocks in light clients. In Nikita Borisov and Claudia Díaz, editors, *FC 2021, Part II*, volume 12675 of *LNCS*, pages 279–298. Springer, Berlin, Heidelberg, March 2021. (Cited on Page 3, 9, 34.)
- [AW09] James Aspnes and Udi Wieder. The expansion and mixing time of skip graphs with applications. *Distributed Comput.*, 21(6):385–393, 2009. (Cited on Page 36.)
- [AY15] Kota Abe and Mikio Yoshiday. Constructing distributed doubly linked lists without distributed locking. In Yong Liu, Marinho P. Barcellos, Jay R. Lorch, and Anyi Wang, editors, *2015 IEEE International Conference on Peer-to-Peer Computing, P2P 2015, Boston, MA, USA, September 21-25, 2015*, pages 1–10. IEEE, 2015. (Cited on Page 36.)



- [BGK<sup>+</sup>08] Edward Bortnikov, Maxim Gurevich, Idit Keidar, Gabriel Kliot, and Alexander Shraer. Brahms: byzantine resilient random membership sampling. In Rida A. Bazzi and Boaz Patt-Shamir, editors, *Proceedings of the Twenty-Seventh Annual ACM Symposium on Principles of Distributed Computing, PODC 2008, Toronto, Canada, August 18-21, 2008*, pages 145–154. ACM, 2008. (Cited on Page 37.)
- [BKW24] Djamal Belazzougui, Gregory Kucherov, and Stefan Walzer. Better space-time-robustness trade-offs for set reconciliation. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *ICALP 2024*, volume 297 of *LIPIcs*, pages 20:1–20:19. Schloss Dagstuhl, July 2024. (Cited on Page 31.)
- [BM07] Ingmar Baumgart and Sebastian Mies. S/kademlia: A practicable approach towards secure key-based routing. In *13th International Conference on Parallel and Distributed Systems, ICPADS 2007, Hsinchu, Taiwan, December 5-7, 2007*, pages 1–8. IEEE Computer Society, 2007. (Cited on Page 4, 36.)
- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993. (Cited on Page 5.)
- [BS20] Ryohei Banno and Kazuyuki Shudo. Skip suffix array: A partial match retrieval method on structured overlay networks. In *2020 International Conference on Information Networking, ICOIN 2020, Barcelona, Spain, January 7-10, 2020*, pages 487–492. IEEE, 2020. (Cited on Page 36.)
- [CDG<sup>+</sup>02] Miguel Castro, Peter Druschel, Ayalvadi J. Ganesh, Antony I. T. Rowstron, and Dan S. Wallach. Secure routing for structured peer-to-peer overlay networks. In David E. Culler and Peter Druschel, editors, *5th Symposium on Operating System Design and Implementation (OSDI 2002), Boston, Massachusetts, USA, December 9-11, 2002*. USENIX Association, 2002. (Cited on Page 36.)
- [Chu24] Suphanat Chunhapanaya. Number of peers you need for peer sampling in PeerDAS (EIP-7594). <https://ethresear.ch/t/number-of-peers-you-need-for-peer-sampling-in-peerdas-eip-7594/205>. 2024. [Accessed 25-02-2025]. (Cited on Page 37.)
- [CKR<sup>+</sup>24] Mikel Cortes-Goicoechea, Csaba Király, Dmitriy Ryajov, Jose Luis Muñoz-Tapia, and Leonardo Bautista-Gomez. Scalability limitations of kademlia dhts when enabling data availability sampling in Ethereum. *CoRR*, abs/2402.09993, 2024. (Cited on Page 4, 35.)
- [EA25] Alex Evans and Guillermo Angeris. The accidental computer: Polynomial commitments from data availability. 2025. (Cited on Page 3, 35.)
- [EMA25] Alex Evans, Nicolas Mohnblatt, and Guillermo Angeris. ZODA: Zero-overhead data availability. *Cryptology ePrint Archive*, Paper 2025/034, 2025. (Cited on Page 3, 35.)
- [FS02] Amos Fiat and Jared Saia. Censorship resistant peer-to-peer content addressable networks. In David Eppstein, editor, *13th SODA*, pages 94–103. ACM-SIAM, January 2002. (Cited on Page 4, 36.)
- [FSSH09] Daisuke Fukuchi, Christian Sommer, Yuichi Sei, and Shinichi Honiden. Distributed arrays: A P2P data structure for efficient logical arrays. In *INFOCOM 2009. 28th IEEE International Conference on Computer Communications, Joint Conference of the IEEE Computer and Communications Societies, 19-25 April 2009, Rio de Janeiro, Brazil*, pages 1458–1466. IEEE, 2009. (Cited on Page 36.)
- [FSY05] Amos Fiat, Jared Saia, and Maxwell Young. Making chord robust to byzantine attacks. In Gerth Stølting Brodal and Stefano Leonardi, editors, *Algorithms - ESA 2005, 13th Annual European Symposium, Palma de Mallorca, Spain, October 3-6, 2005, Proceedings*, volume 3669 of *Lecture Notes in Computer Science*, pages 803–814. Springer, 2005. (Cited on Page 4, 36.)

- [GBL<sup>+</sup>03] Indranil Gupta, Kenneth P. Birman, Prakash Linga, Alan J. Demers, and Robbert van Renesse. Kelips: Building an efficient and stable P2P DHT through increased memory and background overhead. In M. Frans Kaashoek and Ion Stoica, editors, *Peer-to-Peer Systems II, Second International Workshop, IPTPS 2003, Berkeley, CA, USA, February 21-22, 2003, Revised Papers*, volume 2735 of *Lecture Notes in Computer Science*, pages 160–169. Springer, 2003. (Cited on Page 35.)
- [GHK13] Rachid Guerraoui, Florian Huc, and Anne-Marie Kermarrec. Highly dynamic distributed computing with byzantine failures. In Panagiota Fatourou and Gadi Taubenfeld, editors, *ACM Symposium on Principles of Distributed Computing, PODC '13, Montreal, QC, Canada, July 22-24, 2013*, pages 176–183. ACM, 2013. (Cited on Page 4, 36.)
- [GMS08] Alejandra N. González-Beltrán, Peter Milligan, and Paul Sage. Range queries over skip tree graphs. *Comput. Commun.*, 31(2):358–374, 2008. (Cited on Page 36.)
- [GNS06] Michael T. Goodrich, Michael J. Nelson, and Jonathan Z. Sun. The rainbow skip graph: a fault-tolerant constant-degree distributed data structure. In *17th SODA*, pages 384–393. ACM-SIAM, January 2006. (Cited on Page 36.)
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. (Cited on Page 3.)
- [HASW23] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. Foundations of data availability sampling. *Cryptology ePrint Archive*, Paper 2023/1079, 2023. <https://eprint.iacr.org/2023/1079>. (Cited on Page 3, 9, 11, 35.)
- [HASW24] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. FRIDA: Data availability sampling from FRI. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VI*, volume 14925 of *LNCS*, pages 289–324. Springer, Cham, August 2024. (Cited on Page 3, 35.)
- [HG17] Sikder Huq and Sukumar Ghosh. Locally self-adjusting skip graphs. In Kisung Lee and Ling Liu, editors, *37th IEEE International Conference on Distributed Computing Systems, ICDCS 2017, Atlanta, GA, USA, June 5-8, 2017*, pages 805–815. IEEE Computer Society, 2017. (Cited on Page 36.)
- [JPS<sup>+</sup>18] Mercy O. Jaiyeola, Kyle Patron, Jared Saia, Maxwell Young, and Qian M. Zhou. Tiny groups tackle byzantine adversaries. In *2018 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2018, Vancouver, BC, Canada, May 21-25, 2018*, pages 1030–1039. IEEE Computer Society, 2018. (Cited on Page 36.)
- [JRS<sup>+</sup>09] Riko Jacob, Andréa W. Richa, Christian Scheideler, Stefan Schmid, and Hanjo Täubig. A distributed polylogarithmic time algorithm for self-stabilizing skip graphs. In Srikanta Tirthapura and Lorenzo Alvisi, editors, *28th ACM PODC*, pages 131–140. ACM, August 2009. (Cited on Page 36.)
- [KAR<sup>+</sup>23] Michal Król, Onur Ascigil, Sergi Rene, Etienne Rivière, Matthieu Pigaglio, Kaleem Peeroo, Vladimir Stankovic, Ramin Sadre, and Felix Lange. Data availability sampling in ethereum: Analysis of P2P networking requirements. *CoRR*, abs/2306.11456, 2023. (Cited on Page 4, 35.)
- [Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *24th ACM STOC*, pages 723–732. ACM Press, May 1992. (Cited on Page 3.)
- [KLL<sup>+</sup>97] David R. Karger, Eric Lehman, Frank Thomson Leighton, Rina Panigrahy, Matthew S. Levine, and Daniel Lewin. Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web. In *29th ACM STOC*, pages 654–663. ACM Press, May 1997. (Cited on Page 31, 35.)
- [Mic00] Silvio Micali. Computationally sound proofs. *SIAM J. Comput.*, 30(4):1253–1298, 2000. (Cited on Page 3.)

- [MM02] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In Peter Druschel, M. Frans Kaashoek, and Antony I. T. Rowstron, editors, *Peer-to-Peer Systems, First International Workshop, IPTPS 2002, Cambridge, MA, USA, March 7-8, 2002, Revised Papers*, volume 2429 of *Lecture Notes in Computer Science*, pages 53–65. Springer, 2002. (Cited on Page 4, 35.)
- [MTZ03] Yaron Minsky, Ari Trachtenberg, and Richard Zippel. Set reconciliation with nearly optimal communication complexity. *IEEE Trans. Inf. Theory*, 49(9):2213–2218, 2003. (Cited on Page 31.)
- [MU05] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005. (Cited on Page 42.)
- [NW03] Moni Naor and Udi Wieder. A simple fault tolerant distributed hash table. In M. Frans Kaashoek and Ion Stoica, editors, *Peer-to-Peer Systems II, Second International Workshop, IPTPS 2003, Berkeley, CA, USA, February 21-22, 2003, Revised Papers*, volume 2735 of *Lecture Notes in Computer Science*, pages 88–97. Springer, 2003. (Cited on Page 4, 36.)
- [RD01] Antony I. T. Rowstron and Peter Druschel. Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems. In Rachid Guerraoui, editor, *Middleware 2001, IFIP/ACM International Conference on Distributed Systems Platforms Heidelberg, Germany, November 12-16, 2001, Proceedings*, volume 2218 of *Lecture Notes in Computer Science*, pages 329–350. Springer, 2001. (Cited on Page 4, 35.)
- [RFH<sup>+</sup>01] Sylvia Ratnasamy, Paul Francis, Mark Handley, Richard M. Karp, and Scott Shenker. A scalable content-addressable network. In Rene L. Cruz and George Varghese, editors, *Proceedings of the ACM SIGCOMM 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, August 27-31, 2001, San Diego, CA, USA*, pages 161–172. ACM, 2001. (Cited on Page 4, 35.)
- [Rya23] Danny Ryan. PeerDAS – a simpler DAS approach using battle-tested p2p components. <https://ethresear.ch/t/peerdas-a-simpler-das-approach-using-battle-tested-p2p-components/> 2023. [Accessed 25-02-2025]. (Cited on Page 37.)
- [Sch05] Christian Scheideler. How to spread adversarial nodes? Rotate! In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 704–713. ACM Press, May 2005. (Cited on Page 4, 36.)
- [SF12] Siddhartha Sen and Michael J. Freedman. Commensal cuckoo: secure group partitioning for large-scale services. *ACM SIGOPS Oper. Syst. Rev.*, 46(1):33–39, 2012. (Cited on Page 4, 36.)
- [SMK<sup>+</sup>01] Ion Stoica, Robert Tappan Morris, David R. Karger, M. Frans Kaashoek, and Hari Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications. In Rene L. Cruz and George Varghese, editors, *Proceedings of the ACM SIGCOMM 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, August 27-31, 2001, San Diego, CA, USA*, pages 149–160. ACM, 2001. (Cited on Page 4, 35, 36.)
- [WZ24] Benedikt Wagner and Arantxa Zapico. A documentation of ethereum’s PeerDAS. Cryptology ePrint Archive, Paper 2024/1362, 2024. (Cited on Page 3, 35.)
- [YGA24] Lei Yang, Yossi Gilad, and Mohammad Alizadeh. Practical rateless set reconciliation. In *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024*, pages 595–612. ACM, 2024. (Cited on Page 31.)
- [YKGK13] Maxwell Young, Aniket Kate, Ian Goldberg, and Martin Karsten. Towards practical communication in byzantine-resistant dhts. *IEEE/ACM Trans. Netw.*, 21(1):190–203, 2013. (Cited on Page 4, 36.)
- [ZV23] Yunqi Zhang and Shaileshh Bojja Venkatakrishnan. Kadabra: Adapting kademlia for the decentralized web. In Foteini Baldimtsi and Christian Cachin, editors, *FC 2023, Part II*, volume 13951 of *LNCS*, pages 327–345. Springer, Cham, May 2023. (Cited on Page 35.)

[ZV24] Yunqi Zhang and Shaileshh Bojja Venkatakrisnan. Honeybee: Decentralized peer sampling with verifiable random walks for blockchain data sharding. *arXiv preprint arXiv:2402.16201*, 2024. (Cited on Page 37.)

## Appendix

### A Mathematical Tools

**Lemma 10** (Second Moment of Binomial Distribution, Any Textbook). *Let  $X = \sum_{i=1}^n X_i$  be the sum of independent random variables  $X_i \in \{0, 1\}$ , where  $\Pr[X_i = 1] = p$  for all  $i \in [n]$ . Then, we have  $\mathbb{E}[X^2] = np(1-p) + n^2p^2$ .*

**Lemma 11** (Chernoff Bound, taken from [MU05]). *Let  $X_1, \dots, X_\ell$  be independent Poisson trials. Let  $X = \sum_{i \in [\ell]} X_i$  and  $\mu = \mathbb{E}[X]$ . Then, for  $0 < \delta \leq 1$ , we have*

$$\Pr[X \geq (1 + \delta)\mu] \leq e^{-\mu\delta^2/3}.$$

**Corollary 4.** *Let  $X, Y$  be random variables, not necessarily independent. Assume that  $X$  follows a binomial distribution with  $n$  trials and parameter  $p$ . Then, we have*

$$\mathbb{E}[XY] \leq \mathbb{E}[Y](ne^{-np/3} + 2np).$$

*In particular, if  $n \geq -3 \ln(p)/p$ , then*

$$\mathbb{E}[XY] \leq 3np \cdot \mathbb{E}[Y].$$

*Proof.* This follows directly by using conditional expectation and applying Lemma 11 to random variable  $X$  with  $\delta = 1$  and  $\mu = np$ .  $\square$

The following two facts are standard:

**Lemma 12** (Exponential Decay). *We have  $(1 - x)^N < e^{-xN}$  for all real-valued  $1 > x \geq 0$  and  $N > 0$ .*

*Proof.* Taking  $\ln$  on both sides, this is equivalent to  $N \ln(1 - x) < -xN$ , which follows from the fact that  $\ln$  is convex.  $\square$

**Lemma 13** (Binomial Coefficients and Entropy). *For any  $n > 0$  and  $0 \leq k \leq n$ , we have  $\binom{n}{k} \leq 2^{h(k/n)n}$ , where  $h(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$  is the binary entropy function.*

*Proof.* Consider the probability space, where we uniformly choose exactly  $k$  different items out of  $\{1, \dots, n\}$ . Let  $X_i$  be the indicator variable indicating whether  $i$  was chosen. Denote the Shannon-entropy of (the distribution of) a random variable  $X$  by  $HS(X)$ . By definition,  $HS(X_i) = h(k/n)$ . Then using subadditivity of entropy, we get

$$\log_2 \left( \binom{n}{k} \right) = HS(X_1, \dots, X_n) \leq HS(X_1) + \dots + HS(X_n) = n \cdot h(k/n),$$

which shows the claim.  $\square$

### B Subnet Discovery Protocol

In this section, we present a simple protocol that satisfies Definitions 5 and 6. That is, we give a subnet discovery protocol  $\Pi_{SD}$  for  $M$  subnets. The protocol is specified in Figure 9.

**Protocol Overview.** We give an informal overview of the subnet protocol. For details, we refer to Figure 9. The main idea is simple: if a party  $\mathcal{P}$  wants to join a subnet  $\text{sid}$  via a party  $\hat{\mathcal{P}}$ , it first announces

itself to that party. Then,  $\hat{P}$  forwards this information to all parties in the subnet that it knows. In this way, if  $\mathcal{P}$  joins at time  $\tau$ , all parties that  $\hat{P}$  knows will learn about  $\mathcal{P}$  at latest at the beginning of round  $\tau + 3$ . Further, we need to establish that  $\mathcal{P}$  learns from  $\hat{P}$  about all other parties in the subnet. In a naive approach,  $\hat{P}$  would immediately respond with all peers in the subnet that it knows. However, this approach fails if multiple parties join simultaneously (either all via  $\hat{P}$  or via different parties). To solve this,  $\mathcal{P}$  must give  $\hat{P}$  enough time to learn about all parties that joined around time  $\tau$ . It turns out that waiting until  $\tau + 4$  before pulling the list of peers from  $\hat{P}$  is enough.

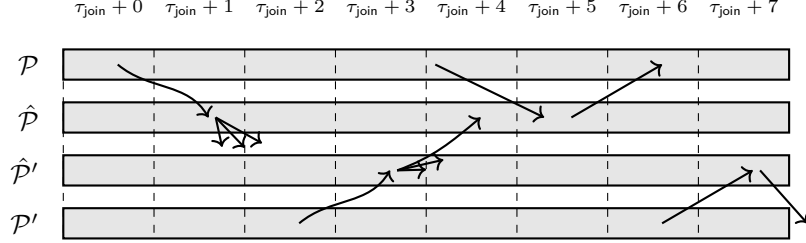


Figure 8: Visualization of joining a subnet in our simple subnet discovery protocol  $\Pi_{SD}$ . Party  $\mathcal{P}$  joins the subnet at time  $\tau_{\text{join}}$  via party  $\hat{P}$ . It sends a message to  $\hat{P}$ , and then  $\hat{P}$  announces to other parties in the subnet that  $\mathcal{P}$  joined. At  $\tau_{\text{join}} + 4$ , party  $\mathcal{P}$  pulls all peers in the subnet from  $\hat{P}$ . By that time,  $\hat{P}$  will have learned about  $\mathcal{P}'$  from  $\hat{P}'$ , assuming that  $\hat{P}$  was one of the peers of  $\hat{P}'$  at time  $\tau_{\text{join}} + 3$ .  $\mathcal{P}'$  joined the subnet via  $\hat{P}'$  at time  $\tau_{\text{join}} + 2$ .

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Interface</b> $\text{INIT}(\mathbf{P}, \text{role}_1, \dots, \text{role}_\ell)$<br><i>// nothing to do here</i><br><b>Interface</b> $\text{JOIN}(\{\mathcal{P}_1, \dots, \mathcal{P}_t\}, \text{role})$<br><i>// initialize all subnets locally</i><br>01 <b>for</b> $\text{sid} \in [M]$ : $\text{Peers}[\text{sid}] := \emptyset$<br><b>Interface</b> $\text{CREATESUBNET}(\text{sid}, \mathbf{P})$<br>02 $\text{Peers}[\text{sid}] := \text{Peers}[\text{sid}] \cup \mathbf{P}$<br><b>Interface</b> $\text{JOINSUBNET}(\text{sid}, \hat{P})$<br>03 $\text{Peers}[\text{sid}] := \text{Peers}[\text{sid}] \cup \{\mathcal{P}_{\text{self}}, \hat{P}\}$<br>04 <b>send</b> $(\text{join\_subn}, \text{sid})$ <b>to</b> $\hat{P}$ ( <b>say, in round</b> $\tau$ )<br>05 <b>in round</b> $\tau + 4$ : <b>send</b> $(\text{join\_subn\_pull}, \text{sid})$ <b>to</b> $\hat{P}$<br>06 <b>receive</b> $(\text{join\_subn\_pull\_rsp}, \text{sid}, \mathbf{P})$ <b>from</b> $\hat{P}$<br>07 $\text{Peers}[\text{sid}] := \text{Peers}[\text{sid}] \cup \mathbf{P}$ | <b>On Message</b> $(\text{join\_subn}, \text{sid})$ <b>from</b> $\mathcal{P}$<br>08 $\mathbf{P} := \text{Peers}[\text{sid}]$<br>09 $\text{Peers}[\text{sid}] := \text{Peers}[\text{sid}] \cup \{\mathcal{P}\}$<br><i>// forward the party (in parallel)</i><br>10 <b>for</b> $\mathcal{P}'' \in \mathbf{P}$ : <b>send</b> $(\text{join\_subn\_fw}, \text{sid}, \mathcal{P})$ <b>to</b> $\mathcal{P}''$<br><b>On Message</b> $(\text{join\_subn\_pull}, \text{sid})$ <b>from</b> $\mathcal{P}$<br>11 $\mathbf{P} := \text{Peers}[\text{sid}]$<br><i>// respond to the party</i><br>12 <b>send</b> $(\text{join\_subn\_pull\_rsp}, \text{sid}, \mathbf{P})$ <b>to</b> $\mathcal{P}$<br><b>On Message</b> $(\text{join\_subn\_fw}, \text{sid}, \mathcal{P})$ <b>from</b> $\hat{P}$<br>13 $\text{Peers}[\text{sid}] := \text{Peers}[\text{sid}] \cup \{\mathcal{P}\}$<br><b>Interface</b> $\text{GETPEERS}(\text{sid})$<br>14 <b>return</b> $\text{Peers}[\text{sid}]$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 9: A subnet discovery protocol  $\Pi_{SD}$  for  $M$  subnets. The party that is executing the instructions is referred to as  $\mathcal{P}_{\text{self}}$ .

**Robustness Analysis.** In the next definition, we introduce a shorthand notation to denote that a party  $\mathcal{P}$  knows another party  $\mathcal{P}'$  as part of a subnet  $\text{sid}$  at time  $\tau$ . Note that the notation represents an event in the experiment of running the protocol.

**Definition 10** (Peer Notation). Consider  $\Pi_{SD}$  as defined in Figure 9. Let  $\mathcal{P}, \mathcal{P}'$  be parties, let  $\tau \geq 0$  be a point in time, and let  $\text{sid} \in [M]$ . Then, we write  $\mathcal{P} \rightsquigarrow_{\text{sid}}^\tau \mathcal{P}'$  to denote the event that at time  $\tau$ ,  $\mathcal{P}$  is active and honest, and the set  $\text{Peers}[\text{sid}]$  locally stored at party  $\mathcal{P}$  contains<sup>12</sup>  $\mathcal{P}'$ .

With this notation, we can now make several observations. The first observation is that if a party knows another party at some point, it will also know this party in the future. This holds because parties never remove elements from sets  $\text{Peers}[\text{sid}]$ . The second observation also follows trivially from our definition of the protocol and the definition of the event. It relates the creation of new subnets and

<sup>12</sup>We mean it is contained in the set over the *entire* round.

our peer notation. The third trivial observation that we make is how our notation relates to interface `GETPEERS`. Our fourth observation is about the mechanics of joining a subnet. It can easily be seen from the definition of the protocol.

**Proposition 4** (Peer Preservation). *Let  $\mathcal{P}, \mathcal{P}'$  be honest parties, let  $\tau \geq 0$  be a point in time, and let  $\text{sid} \in [M]$ . Conditioned on the event that at time  $\tau$  and  $\tau + 1$ ,  $\mathcal{P}$  is active, we have (with probability 1):*

$$\mathcal{P} \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}' \implies \mathcal{P} \rightsquigarrow_{\text{sid}}^{\tau+1} \mathcal{P}'.$$

**Proposition 5** (Subnet Creation). *Let  $\mathcal{P}, \mathcal{P}'$  be honest parties active at time  $\tau = 0$  and  $\mathbf{P}$  be a set with  $\mathcal{P}, \mathcal{P}' \in \mathbf{P}$ . Let  $\text{sid} \in [M]$ . Then, conditioned on event  $\text{CreatedSubnet}[\text{sid}, \mathbf{P}]$ , we have (with probability 1):*

$$\mathcal{P} \rightsquigarrow_{\text{sid}}^0 \mathcal{P}' \wedge \mathcal{P}' \rightsquigarrow_{\text{sid}}^0 \mathcal{P}.$$

**Proposition 6** (Get Peers). *Let  $\mathcal{P}, \mathcal{P}'$  be honest parties, let  $\tau \geq 0$  be a point in time at which  $\mathcal{P}$  is active, and let  $\text{sid} \in [M]$ . Then, with probability 1, we have:*

$$(\mathcal{P} \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}' \wedge \text{CalledGetPeers}[\text{sid}, \mathcal{P}, \tau]) \implies \text{GotPeer}[\text{sid}, \mathcal{P}, \mathcal{P}', \tau].$$

**Proposition 7** (Join Timings). *Let  $\mathcal{P}, \mathcal{P}', \hat{\mathcal{P}}$  be honest parties, let  $\tau \geq 0$  be a point in time, and let  $\text{sid} \in [M]$ . Condition on  $\mathcal{P}$  calling  $\text{JOINSUBNET}(\text{sid}, \hat{\mathcal{P}})$  at time  $\tau$  and  $\hat{\mathcal{P}}$  being active at time  $\tau$ . Then, with probability 1, we have the following two:*

$$\begin{aligned} \text{Active}[\mathcal{P}, \tau + 7] \wedge \hat{\mathcal{P}} \rightsquigarrow_{\text{sid}}^{\tau+5} \mathcal{P}' &\implies \mathcal{P} \rightsquigarrow_{\text{sid}}^{\tau+7} \mathcal{P}', \text{ and} \\ \text{Active}[\mathcal{P}', (\tau + 2) \dots (\tau + 3)] \wedge \hat{\mathcal{P}} \rightsquigarrow_{\text{sid}}^{\tau+1} \mathcal{P}' &\implies \mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau+3} \mathcal{P}. \end{aligned}$$

We will now come to our central lemma, which we then use to show robustness. Before giving the lemma, we introduce some useful terminology, namely, what it means that a node joins a subnet properly. This is exactly as in the definition of event `IsInSubnet`.

**Definition 11** (Joining Subnets Properly). Let  $\Delta_{\text{sub}} = 7$ . Let  $\text{sid} \in [M]$ . Let  $\mathcal{P}$  be an honest party and  $\tau \geq 0$  be a point in time. Then, we say that party  $\mathcal{P}$  joins the subnet  $\text{sid}$  properly at time  $\tau$  if (a)  $\tau = 0$  and  $\text{CreatedSubnet}[\text{sid}, \mathbf{P}]$  such that  $\mathcal{P} \in \mathbf{P}$  or (b)  $\mathcal{P}$  calls  $\text{JOINSUBNET}(\text{sid}, \hat{\mathcal{P}})$  at time  $\tau$  for an honest party  $\hat{\mathcal{P}}$  with  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\hat{\mathcal{P}}, \tau \dots (\tau + \Delta_{\text{sub}})]$ . In case (b) we also say that  $\mathcal{P}$  joins properly via  $\hat{\mathcal{P}}$ .

**Lemma 14** (Peer Propagation). *Let  $\mathcal{P}, \mathcal{P}'$  be honest parties and let  $\text{sid} \in [M]$ . Assume that*

- *No honest party calls interface `JOINSUBNET` with this  $\text{sid}$  in rounds 0 to 7 (inclusive), and*
- *Party  $\mathcal{P}$  joins the subnet  $\text{sid}$  properly at time  $\tau_{\text{join}}$ , and*
- *party  $\mathcal{P}'$  joins the subnet  $\text{sid}$  properly at time  $\tau'_{\text{join}}$  and is active at a point  $\tau \geq \tau'_{\text{join}}$ .*

*Further, assume that at least one of the following three conditions hold:*

- (a)  $\tau \geq \tau_{\text{join}} + 3 \wedge \tau'_{\text{join}} \leq \tau_{\text{join}} - 3$ , or
- (b)  $\tau \geq \tau_{\text{join}} + 3 \wedge \tau'_{\text{join}} \leq \tau - 7$ , or
- (c)  $\tau_{\text{join}} = \tau'_{\text{join}} = 0$ .

*Then, we have  $\mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}$ .*

*Proof.* Before we start with the actual proof, we make the following useful observation, which follows from the assumption that no honest party joins in rounds 1 to 7: If a party  $\mathcal{P}$  joins the subnet  $\text{sid}$  properly at time  $\tau$ , then



- Either,  $\mathcal{P}$  is one of the initial parties, i.e.,  $\tau = 0$  and event  $\text{CreatedSubnet}[\text{sid}, \mathbf{P}]$  occurs such that  $\mathcal{P} \in \mathbf{P}$ , or
- $\mathcal{P}$  calls  $\text{JOINSUBNET}(\text{sid}, \hat{\mathcal{P}})$  at time  $\tau$  for an honest party  $\hat{\mathcal{P}}$  with  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\hat{\mathcal{P}}, \tau \dots (\tau + \Delta_{\text{sub}})]$ , where  $\hat{\mathcal{P}}$  joined the subnet at some time  $\leq \tau - 7$ .

With this in mind, we will now prove the statement via induction on pairs  $(\tau, \tau_{\text{join}})$ , with lexicographic ordering. Essentially, (c) serves as our base case, and for the induction step we consider cases (a) and (b) separately.

*Assuming (c) holds:* If  $\tau_{\text{join}} = \tau'_{\text{join}} = 0$ , then we know that event  $\text{CreatedSubnet}[\text{sid}, \mathbf{P}]$  such that  $\mathcal{P}, \mathcal{P}' \in \mathbf{P}$  holds, and therefore (via Propositions 4 and 5), we get that  $\mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}$ .

*Induction Step, assuming (a) holds:* Our induction hypothesis is that the lemma holds for all pairs  $(\tilde{\tau}, \tilde{\tau}_{\text{join}})$  preceding  $(\tau, \tau_{\text{join}})$  in the lexicographic ordering. Assume that (a) holds. Note that in case (a) we cannot have  $\tau_{\text{join}} = 0$ , as otherwise  $\tau'_{\text{join}} < 0$ . So, we can assume that  $\tau_{\text{join}} > 0$ . By the definition of joining a subnet properly and our observation above, we therefore know that  $\mathcal{P}$  calls  $\text{JOINSUBNET}(\text{sid}, \hat{\mathcal{P}})$  at time  $\tau_{\text{join}}$  for an honest party  $\hat{\mathcal{P}}$  with  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\hat{\mathcal{P}}, \tau_{\text{join}} \dots (\tau_{\text{join}} + \Delta_{\text{sub}})]$ , where  $\hat{\mathcal{P}}$  joined the subnet properly at some time  $\hat{\tau}_{\text{join}} \leq \tau_{\text{join}} - 7$ . Our claim is that  $\hat{\mathcal{P}} \rightsquigarrow_{\text{sid}}^{\tau_{\text{join}}+1} \mathcal{P}'$ , which then implies  $\mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau_{\text{join}}+3} \mathcal{P}$  via Proposition 7, and then  $\mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}$  because of  $\tau \geq \tau_{\text{join}} + 3$  and Proposition 4.

We can show this claim using the induction hypothesis applied to <sup>13</sup>

$$\tilde{\mathcal{P}} := \mathcal{P}', \tilde{\mathcal{P}}' := \hat{\mathcal{P}}, \tilde{\tau}_{\text{join}} := \tau'_{\text{join}}, \tilde{\tau}'_{\text{join}} := \hat{\tau}_{\text{join}}, \tilde{\tau} := \tau_{\text{join}} + 1.$$

To see why we can apply the induction hypothesis, first observe that  $\tilde{\mathcal{P}}' = \hat{\mathcal{P}}$  is active at  $\tilde{\tau} = \tau_{\text{join}} + 1$ , because  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\hat{\mathcal{P}}, \tau_{\text{join}} \dots (\tau_{\text{join}} + \Delta_{\text{sub}})]$ . Second, observe that

$$(\tilde{\tau}, \tilde{\tau}_{\text{join}}) = (\tau_{\text{join}} + 1, \tau'_{\text{join}}) \leq (\tau - 2, \tau_{\text{join}} - 3) < (\tau, \tau_{\text{join}}).$$

Third, one can see that condition (b) is met:

$$\begin{aligned} \tilde{\tau} &= \tau_{\text{join}} + 1 > \tau_{\text{join}} - 3 + 3 \geq \tau'_{\text{join}} + 3 = \tilde{\tau}_{\text{join}} + 3, \\ \text{and } \tilde{\tau}'_{\text{join}} &= \hat{\tau}_{\text{join}} \leq \tau_{\text{join}} - 7 \leq (\tau_{\text{join}} + 1) - 7 = \tilde{\tau} - 7 \end{aligned}$$

*Induction Step, assuming (b) holds:* Again, our induction hypothesis is that the lemma holds for all pairs  $(\tilde{\tau}, \tilde{\tau}_{\text{join}})$  preceding  $(\tau, \tau_{\text{join}})$  in the lexicographic ordering. Now, we assume that (b) holds. First, note that if  $\tau = \tau_{\text{join}} + 3$ , then one can see that the conditions in (a) are met, and we have already proven this case. So, we can assume that  $\tau \geq \tau_{\text{join}} + 4$ . Now, if  $\tau'_{\text{join}} \leq \tau - 8$ , then (b) holds for the same set of parties and  $\tilde{\tau} := \tau - 1$ , so we can apply induction directly. Therefore, we can make the following assumptions:

$$\tau \geq \tau_{\text{join}} + 4 \wedge \tau'_{\text{join}} = \tau - 7.$$

Now, consider the case in which  $\tau'_{\text{join}} = 0$  and thus  $\tau = 7$ . Then, if  $\tau_{\text{join}} = 0$  as well, we are in case (c) and we are done. If  $\tau_{\text{join}} > 0$ , then  $\mathcal{P}$  joined properly via  $\hat{\mathcal{P}}$  at  $\tau_{\text{join}} \leq \tau - 3 = 4$ . But we assume that there is no join in rounds 1 to 7. Therefore, we can extend the assumptions that we make:

$$\tau \geq \tau_{\text{join}} + 4 \wedge \tau'_{\text{join}} = \tau - 7 \wedge \tau'_{\text{join}} > 0.$$

This means that party  $\mathcal{P}'$  joins the subnet sid properly at time  $\tau'_{\text{join}}$  via a party  $\hat{\mathcal{P}}'$ . Denote the time at which  $\hat{\mathcal{P}}$  joined the subnet by  $\hat{\tau}_{\text{join}}$ . In the following, we consider two cases:

... *Case b1:*  $\tau = \tau_{\text{join}} + 4$ . We have already established that  $\tau'_{\text{join}} = \tau - 7$ . In combination, this means that

$$\tau = \tau_{\text{join}} + 4 \geq \tau_{\text{join}} + 3 \wedge \tau'_{\text{join}} = \tau - 7 = (\tau - 4) - 3 = \tau_{\text{join}} - 3.$$

Therefore, this is a special case of (a), which we have already proven.

<sup>13</sup>Values with a tilde indicate that we use the induction hypothesis on these, e.g.,  $\tilde{\mathcal{P}}$  takes the role of  $\mathcal{P}$  in the lemma when applying the hypothesis.

... *Case b2*:  $\tau \geq \tau_{\text{join}} + 5$ . In this case, we use the induction hypothesis to show that  $\hat{\mathcal{P}}' \rightsquigarrow_{\text{sid}}^{\tau-2=\tau'_{\text{join}}+5} \mathcal{P}$ . Note that this implies our desired statement  $\mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}$  via Proposition 7. Concretely, we apply the induction hypothesis to

$$\tilde{\mathcal{P}} := \mathcal{P}, \tilde{\mathcal{P}}' := \hat{\mathcal{P}}', \tilde{\tau}_{\text{join}} := \tau_{\text{join}}, \tilde{\tau}'_{\text{join}} := \tau'_{\text{join}}, \tilde{\tau} := \tau - 2.$$

Again, let us make explicit why we can use the induction hypothesis: For  $\tilde{\mathcal{P}}' = \hat{\mathcal{P}}'$  we know that  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\tilde{\mathcal{P}}', \tau'_{\text{join}} \dots (\tau'_{\text{join}} + \Delta_{\text{sub}})]$ . As  $\tau - 2 = \tau'_{\text{join}} + 5$ , this in particular means that  $\tilde{\mathcal{P}}'$  is active at  $\tilde{\tau} = \tau - 2$ . Further, it is clear that  $(\tilde{\tau}, \tilde{\tau}_{\text{join}}) = (\tau - 2, \tau_{\text{join}})$  lexicographically precedes  $(\tau, \tau_{\text{join}})$ . Finally, the conditions of (b) are satisfied as

$$\begin{aligned} \tilde{\tau} = \tau - 2 &\geq \tau_{\text{join}} + 5 - 2 = \tilde{\tau}_{\text{join}} + 3, \\ \text{and } \tilde{\tau}'_{\text{join}} = \tau'_{\text{join}} &\leq \tau'_{\text{join}} - 7 = \tau - 7 - 7 \leq \tau - 2 - 7 = \tilde{\tau} - 7. \end{aligned}$$

□

**Theorem 2 (Robustness of  $\Pi_{\text{SD}}$ ).** Consider the subnet discovery protocol  $\Pi_{\text{SD}}$  for  $M$  subnets as specified in Figure 9. Let  $\mathcal{S}$  be an arbitrary class of join-leave schedules. Let  $T \in \mathbb{N}$  be arbitrary, and let  $\delta = 0$  and  $\Delta_{\text{sub}} = 7$ . Then,  $\Pi_{\text{SD}}$  is  $(\delta, T, \Delta_{\text{sub}}, \mathcal{S})$ -robust.

*Proof.* Fix any point in time  $\tau \in [T]$ , any subnet identifier  $\text{sid} \in [M]$ , and any pair of honest parties  $\mathcal{P}, \mathcal{P}'$ . We need to show that

$$\left( \begin{array}{l} \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau] \\ \wedge \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}', \tau] \\ \wedge \text{CalledGetPeers}[\text{sid}, \mathcal{P}, \tau] \end{array} \right) \implies \text{GotPeer}[\text{sid}, \mathcal{P}, \mathcal{P}', \tau].$$

To do so, we first argue that

$$\left( \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau] \wedge \text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}', \tau] \right) \xRightarrow{\text{Lemma 14}} \mathcal{P} \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}'. \quad (1)$$

This is because we can apply Lemma 14 as follows: First, one can assume without loss of generality that no honest party calls  $\text{JoinSubnet}$  with  $\text{sid}$  in rounds 1 to 7<sup>14</sup>. Also, if an honest party calls  $\text{JoinSubnet}$  with  $\text{sid}$  at round 0, then our robustness definition does not make any guarantees, and so there is nothing to show.

Now, if  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}, \tau]$ , then we know that party  $\mathcal{P}$  joined the subnet  $\text{sid}$  properly at time  $\tau_{\text{join}} \leq \tau - \Delta_{\text{sub}} \leq \tau - 3$ . Similarly, if  $\text{IsInSubnet}_{\text{sid}}^{\Delta_{\text{sub}}}[\mathcal{P}', \tau]$ , then party  $\mathcal{P}'$  joined the subnet  $\text{sid}$  properly at time  $\tau'_{\text{join}} \leq \tau - \Delta_{\text{sub}} = \tau - 7$ . So, all conditions of Lemma 14 (in particular, Item (b)) are satisfied, which means that  $\mathcal{P}' \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}$ . Similarly, we get  $\mathcal{P} \rightsquigarrow_{\text{sid}}^{\tau} \mathcal{P}'$ . Then, using Proposition 6, we get the desired implication. □

**Optimizations.** We make some optimizations when using this subnet discovery protocol to instantiate our protocol  $\Pi_{\text{RDA}}$ : first, observe that honest parties only ever join row subnets via their bootstrap nodes. Thus, nodes with  $\text{role} = 0$  can ignore  $\text{join\_subn}$  and  $\text{join\_subn\_pull}$  messages for row subnets, as those are supposed to only be sent to bootstrap nodes. In fact, an honest node  $\mathcal{P}$  with  $\text{role} = 0$  and  $\text{Row}(\mathcal{P}) = r$  only uses their local list of row subnet peers for calls  $\mathbf{P} := \text{GetPeersInCell}(r, c)$  for some  $c$  and then sends messages to those parties in  $\mathbf{P}$ . For that reason,  $\mathcal{P}$  only needs to keep track of row subnet peers  $\mathcal{P}'$  with  $\text{Row}(\mathcal{P}') = r$ . Note that  $\mathcal{P}$  will learn about their row subnet peers exclusively via  $\text{join\_subn\_fw}$  and  $\text{join\_subn\_pull\_rsp}$ -messages from parties  $\hat{\mathcal{P}}$  with  $\text{role} = 1$ . We shall assume that those  $\text{join\_subn\_fw}$  and  $\text{join\_subn\_pull\_rsp}$  messages by  $\hat{\mathcal{P}}$  do not contain those peers that  $\mathcal{P}$  would not store anyway. Furthermore, we assume that every honest party  $\mathcal{P}$  ignores column subnet messages originating from (necessarily malicious) parties  $\mathcal{P}'$  with  $\text{Col}(\mathcal{P}') \neq \text{Col}(\mathcal{P})$ .

<sup>14</sup>To see why this is without loss of generality, first assume that Equation (1) holds for some run of the protocol in which no honest party joins a subnet in rounds 1 to 7. Then, one can easily see that it also holds for the modified run in which rounds 1 to 7 are entirely skipped. This is because the only messages sent by the protocol are due to joining a subnet. As every run with arbitrary join-leave schedule can be seen as such a truncation of a run in which no honest party joins a subnet in rounds 1 to 7, we get that Lemma 14 holds for every run, as long as we can show that it holds for such runs with no joins from rounds 1 to 7.

| Event                                                                      | Intuition                                                                        | Reference     |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------|
| Active $[\mathcal{P}, \tau]$                                               | $\mathcal{P}$ is active at time $\tau$                                           | Definition 2  |
| Active $[\mathcal{P}, \tau \dots \tau']$                                   | $\mathcal{P}$ is active from time $\tau$ to time $\tau'$                         | Definition 2  |
| FullyJoined $[\mathcal{P}, \tau]$                                          | $\mathcal{P}$ active at $\tau$ and JOIN already finished at $\tau$               | Definition 2  |
| FullyJoined $[\mathcal{P}, \tau \dots \tau']$                              | $\mathcal{P}$ active from $\tau$ to $\tau'$ and JOIN already finished at $\tau$  | Definition 2  |
| CalledStore $_{h,i,x}[\mathcal{P}, \tau]$                                  | $\mathcal{P}$ calls STORE( $h, i, x$ ) at $\tau$                                 | Definition 4  |
| CalledGet $_{h,i}[\mathcal{P}, \tau]$                                      | $\mathcal{P}$ calls GET( $h, i$ ) at $\tau$                                      | Definition 4  |
| GotResult $_{h,i,x}^{\Delta}[\mathcal{P}, \tau]$                           | With a delay of $\leq \Delta$ , $\mathcal{P}$ got $x$ from call to GET( $h, i$ ) | Definition 4  |
| CreatedSubnet[sid, $\mathbf{P}$ ]                                          | Honest parties in $\mathbf{P}$ created subnet sid at $\tau = 0$                  | Definition 6  |
| IsInSubnet $_{sid}^{\Delta_{sid}^{sub}}[\mathcal{P}, \tau_0 \dots \tau_1]$ | $\mathcal{P}$ active from $\tau_0$ to $\tau_1$ and properly joined subnet sid    | Definition 6  |
| IsInSubnet $_{sid}^{\Delta_{sid}^{sub}}[\mathcal{P}, \tau]$                | $\mathcal{P}$ active at $\tau$ and properly joined subnet sid                    | Definition 6  |
| CalledGetPeers[sid, $\mathcal{P}, \tau$ ]                                  | $\mathcal{P}$ calls GETPEERS(sid) at $\tau$                                      | Definition 6  |
| GotPeer[sid, $\mathcal{P}, \mathcal{P}', \tau$ ]                           | $\mathcal{P}$ got peer $\mathcal{P}'$ from call to GETPEERS(sid)                 | Definition 6  |
| SubnProtGoodUntil $[T]$                                                    | Subnet protocol works until time $T$                                             | Definition 7  |
| ColumnGoodUntil $^{\Delta}[c, T]$                                          | always an honest party in $c$ until $T$ (with overlap)                           | Definition 7  |
| GoodCell $[r, c, \tau_1 \dots \tau_2]$                                     | $\exists$ honest party in cell $(r, c)$ from $\tau_1$ to $\tau_2$                | Definition 7  |
| InSymbolStorage $_{h,i,x}[\mathcal{P}, \tau]$                              | $\mathcal{P}$ stores $(h, i, x)$ at $\tau$                                       | Definition 9  |
| WholeColumnStores $_{h,i,x}[c, \tau_0, \tau_1]$                            | All honest parties in $c$ store $(h, i, x)$                                      | Definition 9  |
| $\mathcal{P} \rightsquigarrow_{sid}^{\tau} \mathcal{P}'$                   | $\mathcal{P}$ knows $\mathcal{P}'$ at $\tau$ in subnet sid                       | Definition 10 |

Figure 10: Table of events used throughout the paper. Events typically assume that  $\mathcal{P}$  is honest. For the precise definitions of the events, the reader should consult the definitions.

This figure "acm-jdslogo.png" is available in "png" format from:

<http://arxiv.org/ps/2504.13757v2>